

Total Outcome Logic: Unified Reasoning for a Taxonomy of Program Logics

JAMES LI, Cornell University, USA

NOAM ZILBERSTEIN, Cornell University, USA

ALEXANDRA SILVA, Cornell University, USA

While there is a long tradition of reasoning about (non)termination in program analysis, specialized logics are typically needed to give different termination criteria. This includes partial correctness, where termination is not guaranteed, and total correctness, where it is guaranteed. We present *Total Outcome Logic* (TOL), a single logic which can express the full spectrum of termination conditions and program properties offered by the aforementioned logics. TOL extends (non)termination and (in)correctness reasoning across different kinds of branching effects, so that a single metatheory powers this reasoning in different kinds of programs, including nondeterministic and probabilistic. We also show that TOL subsumes several recently created taxonomies of (in)correctness logics, so that many different kinds of properties can be proven with a single unified theory.

1 Introduction

The study of program logics is heavily intertwined with the *computational effects* that can occur in the underlying programs. Prime examples of effects include nondeterminism and nontermination. In terms of (non)termination, traditional program logics such as Hoare Logic initially revolved around *partial correctness*—properties that hold *if* the program terminates [Floyd 1967; Hoare 1969]. Partial correctness was attractive, as it offered a simple way to describe loop behaviors via invariants while still providing *safety*—the property that the program never displays undesirable behaviors [Lamport 1977]. Soon after, the notion of *total correctness* was introduced, to additionally guarantee termination [Manna and Pnueli 1974]. More recently, other program logics for proving the possibility of nontermination have been explored too [Raad et al. 2024].

Orthogonally, different logics have been proposed to deal with nondeterminism in various ways. Traditional *correctness* logics use a demonic interpretation of nondeterminism where the postcondition applies to all reachable states [Hoare 1969]. However, interest in *possible correctness* [Hoare 1978] and *incorrectness* [Möller et al. 2021; O’Hearn 2020; Zilberstein et al. 2023] has emerged, which demand angelic nondeterminism (the postcondition applies to *some* reachable state).

To make sense of these different logics, various recent efforts have been made to build taxonomies highlighting the similarities and differences between them [Ascari et al. 2023; Cousot 2024; Verscht and Kaminski 2025; Zhang and Kaminski 2022]. In particular, Verscht and Kaminski [2025] identified the following three dimensions for program logics:

- (1) *Correctness* (being able to reach) vs. *incorrectness* (reachability)
- (2) *Totality* (termination) vs. *partiality* (possible nontermination)
- (3) *Angelic* vs. *demonic* nondeterminism

Though extensive, this characterization is incomplete. As Verscht and Kaminski [2025] already acknowledge: after presenting a taxonomy of 16 logics, they identify two more *in-between* ones! But even more dimensions are missing. For one, nondeterminism need not be treated in a strictly angelic or demonic fashion, but rather in a gradient that can be captured by considering multiple *reachable outcomes*. As demonstrated by Outcome Logic [Zilberstein 2024a,b; Zilberstein et al. 2023, 2024a], this more expressive treatment of nondeterminism is needed in order to build logics and static analyses that unify correctness *and* incorrectness reasoning. In addition, branching may not arise

from nondeterminism at all, but rather from other computational effects such as probabilistic choice. So the natural question arises: *is there a taxonomy that captures all these additional dimensions?*

In this paper we take a different approach than [Verscht and Kaminski \[2025\]](#), [Cousot \[2024\]](#), and [Ascari et al. \[2023\]](#). We present a single expressive logic, which subsumes the aforementioned taxonomies, while also providing a higher degree of expressivity to deal with *in-between* cases. As such, this paper is not only about understanding the similarities and differences between different logics, but also about providing a common framework for reasoning about those properties.

Compositionality is the key to developing these unified reasoning principles. We start with a compositional denotational semantics where sequential composition is defined *monadically* and loops are defined as the least fixed point of a Scott continuous operation. However, our desire to reason about nontermination in conjunction with nondeterminism brings about well-known barriers to compositionality: unbounded nondeterminism makes loop semantics non-compositional [[Apt and Plotkin 1986](#)]. As an illustration, consider the command $x := \star$, which nondeterministically assigns a natural number to the variable x . We now ask, does the following program terminate?

$$x := \star \ ; \ \text{while } x > 0 \text{ do } x := x - 1$$

Based on naive intuitions, the answer appears to be yes; for any natural number n assigned to x , the loop will terminate after precisely n iterations. The problem is that we cannot make this argument *compositionally*. Indeed, a nonterminating trace exists in every finite approximation of the loop. This means that standard techniques for proving termination such as loop variants and ranking functions will not suffice.

Curiously, the issues with unbounded choice do not arise in conjunction with other computational effects such as probabilistic choice. Termination of probabilistic programs has been studied extensively [[Feng et al. 2023](#); [Kaminski 2019](#); [Majumdar and Sathiyarayanan 2025](#); [McIver and Morgan 2005](#); [McIver et al. 2018](#)], and there are many examples of probabilistic programs with infinitely many outcomes, which *almost surely terminate*—they terminate with probability 1. For example, one can write a probabilistic program that computes a geometric distribution, where the probability that $x = n$ is $\frac{1}{2^n}$ for all $n \geq 1$. The probability of termination after n iterations is $1 - \frac{1}{2^n}$, which clearly converges to 1 using a simple compositional proof.

In this paper, we identify the key difference between these types of choice. Probabilistic programs are *conservative*—probability mass is treated like a resource, with the total amount of mass (1) being split between branches of computation and nontermination, so the weight of infinite traces can converge to 0 in the limit. Nondeterminism is *indicative*—there is an infinite amount of total *weight*, and spawning new branches does not consume anything.

Our formalism follows the weighted programming paradigm, where the possible program end states are assigned weights [[Batz et al. 2022](#); [Zilberstein 2024a](#)]. Varying the representations of the weights gives interpretations of different effects. For example, Boolean weights indicate whether or not a state is a possible outcome of a nondeterministic program whereas real-valued weights are used to quantify the probabilities of outcomes in a probabilistic program. Weights are elements of a *semiring*, meaning that they can be added and multiplied to provide semantics to branching and sequential composition, respectively. In conservative weightings—like probabilistic programs—it is possible to have compositional inference rules for termination with unbounded choice, whereas in indicative weightings it is not possible. Our full list of contributions is as follows:

- Building on Outcome Logic [[Zilberstein 2024a](#)], we define a generic program semantics, which can also quantify the weights of infinite traces (Section 2). We show how the ability of each model to make unbounded choices corresponds to particular properties of the underlying semiring.

$$\begin{aligned}
\text{Commands } \ni C &::= \mathbf{skip} \mid \mathbf{assume} \, e \mid a \in \text{Act} \mid C_1 \mathbin{\circ} C_2 \mid C_1 + C_2 \mid C^{(e_1, e_2)} \\
\text{Guards } \ni e &::= b \mid u \in U \\
\text{BExp } \ni b &::= \text{true} \mid \text{false} \mid b_1 \vee b_2 \mid b_1 \wedge b_2 \mid \neg b \mid t \in \text{Test}
\end{aligned}$$

Fig. 1. Syntax of a simple weighted imperative language, parametric on a set of basic program actions Act and a set of weights U .

- We extend Outcome Logic with the ability to reason about nontermination to obtain Total Outcome Logic (Section 3). Unlike prior work, this allows us to prove that programs always terminate, or sometimes do not terminate.
- We show that [Verscht and Kaminski’s \[2025\]](#) entire taxonomy of *Hoare-like* logics can be expressed in Total Outcome Logic (Section 4).
- We show that a second taxonomy—the [Cousot \[2024\] cube](#)—can also be expressed in Total Outcome Logic and compare it to the aforementioned taxonomy of [Verscht and Kaminski](#) (Section 5).
- We present a new taxonomy of correctness and incorrectness logics, showing that Total Outcome Logic gives more flexibility to express reusable specifications, making it a good foundation for static analysis (Section 6).
- We derive proof rules to reason more easily about special cases (Section 7).
- We present a variety of case studies in Section 8 to demonstrate how to reason about termination and nontermination with Total Outcome Logic.

Omitted proofs are included in the appendix.

2 Weighted Program Semantics

We consider a simple *weighted* programming language in the style of [Batz et al. \[2022\]](#) and [Zilberstein \[2024a\]](#). We will present the syntax and semantics in this section, while highlighting our new addition: the ability to quantify weights of infinite traces in a denotational model.

2.1 Syntax

The syntax is presented in Figure 1. The language includes the usual syntax for the program that does nothing **skip**, assertions **assume** e , basic (uninterpreted) program actions $a \in \text{Act}$, and sequential composition $\mathbin{\circ}$. We also include in the language a nondeterministic choice $C_1 + C_2$, which is a program that nondeterministically chooses one of the branches to execute. One difference worth highlighting is that the **assume** e command takes a guard e that is not just a usual Boolean expression. In this language guards can be Boolean—the expressions in BExp are built from a basic set of tests Test using the usual Boolean operations—but they can also be *weights* $u \in U$. The intuition behind **assume** u is that the computation trace in which this is executed is weighed by u . This will become clearer below when we discuss the formal semantics.

Using Boolean tests, we can define syntactic sugar for if statements, shown below. The syntax for a loop $C^{(e_1, e_2)}$, due to [Zilberstein \[2024a\]](#), is slightly unusual as it has two guards: e_1 is the guard for the body of the loop C to continue executing, whereas e_2 is the guard for the exit of the loop. In a typical syntax, loops usually only have Boolean guard b and the exit guard is simply the negation of b . This sort of guarded iteration can be encoded in our language as follows:

$$\mathbf{if} \, b \, \mathbf{then} \, C_1 \, \mathbf{else} \, C_2 \triangleq (\mathbf{assume} \, b \mathbin{\circ} C_1) + (\mathbf{assume} \, \neg b \mathbin{\circ} C_2) \quad \mathbf{while} \, b \, \mathbf{do} \, C \triangleq C^{(b, \neg b)}$$

2.2 Semantics as Weighting Functions

In order to provide the semantics of our language we first need to introduce a few algebraic definitions. To make sense of the weights in the context of computation traces, we will need to

have structure on the set U that enables us to combine different weights along a computation trace and across computation traces. In other words, how do we give semantics to expressions like: $(\text{assume } 3) \wp a \wp (\text{assume } 5)$ or $(\text{assume } 3) \wp a_1 + (\text{assume } 5) \wp a_2$? We will require the set of weights U to have the structure of a *semiring*, which is an algebraic structure that combines two *monoids*.

Definition 2.1 (Monoid). A monoid $\langle U, +, \mathbb{0} \rangle$ consists of a carrier set U , an associative binary operation $+: U \times U \rightarrow U$, and an identity element $\mathbb{0}$ ($u + \mathbb{0} = \mathbb{0} + u = u$). If $+: U \rightarrow U$ is partial, then we say the monoid is *partial*. If $+$ is commutative ($u + v = v + u$), then the monoid is *commutative*.

Definition 2.2 (Semiring). A semiring $\langle U, +, \cdot, \mathbb{0}, \mathbb{1} \rangle$ is an algebraic structure such that:

- (1) $\langle U, +, \mathbb{0} \rangle$ is a commutative monoid and $\langle U, \cdot, \mathbb{1} \rangle$ is a monoid.
- (2) Distributivity: $u \cdot (v + w) = uv + uw$ and $(u + v) \cdot w = uw + vw$
- (3) Annihilation: $\mathbb{0} \cdot x = x \cdot \mathbb{0} = \mathbb{0}$

Example 2.3 (Semirings). The following semirings encode different kinds of computation.

- (1) The Boolean semiring $\langle \mathbb{B}, \vee, \wedge, 0, 1 \rangle$ uses Boolean $\mathbb{B} = \{0, 1\}$ weights, which indicate whether states are possible outcomes. Addition is disjunction and multiplication is conjunction.
- (2) The probabilistic semiring $\langle [0, 1], +, \cdot, 0, 1 \rangle$ uses real-valued probabilities as weights to quantify the likelihoods of outcomes. Addition is standard arithmetic addition, but it is partial since it is undefined if the sum is above 1. Multiplication is standard arithmetic multiplication.
- (3) The natural number semiring $\langle \mathbb{N}^\infty, +, \cdot, 0, 1 \rangle$ uses natural numbers (plus ∞) as weights, with addition and multiplication given by standard arithmetic operations. This encodes nondeterminism, where we count the traces leading to each outcome, *i.e.*, as multisets of outcomes.

For more examples, refer to [Batz et al. \[2022\]](#) and [Zilberstein \[2024a\]](#).

In order to define the semantics we will assume a set of program states Σ and then assign to each command a map from program states to weighted, possibly nonterminating, traces. More precisely, we will build a denotational object in two steps. First, to encode information about nontermination, we expose divergence as a possible outcome of running a program. We will represent this with the element $\cup$, where for any set S , we write $S_\cup \triangleq S \cup \{\cup\}$. Second, we define weighting functions, which assign a weight to all states $\sigma \in \Sigma$, and to nontermination.

Definition 2.4 (Weighting Function). Given a set of program states Σ and a (partial) semiring $\mathcal{A} = \langle U, +, \cdot, \mathbb{0}, \mathbb{1} \rangle$, the set of weighting functions is

$$\mathcal{W}_{\mathcal{A}}(\Sigma_\cup) \triangleq \left\{ m : \Sigma_\cup \rightarrow U \mid |m| \text{ is defined and } \text{supp}(m) \text{ is countable} \right\}$$

The support $\text{supp}(m) \triangleq \{\sigma \mid m(\sigma) \neq \mathbb{0}\}$ is the set of states to which m assigns nonzero weight and the mass of $m \in \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$ written as $|m| \triangleq \sum_{\sigma \in \text{supp}(m)} m(\sigma)$ is the cumulative weight in m .

We take a weighting function to represent a configuration in a computation, which may have branched into many different outcomes, including the nontermination outcome $\cup$. Captured here are two distinct computational effects: weighted branching and nontermination. We can now assign to each program C a denotation $\llbracket C \rrbracket : \Sigma \rightarrow \mathcal{W}_{\mathcal{A}}^\infty(\Sigma_\cup)$, as shown in [Figure 2](#)¹.

We left several operations underspecified in [Figure 2](#), which we will now explain in detail, from simpler to more complex: 1. semantics of guards and branching; 2. monadic structure unit and bind that provides semantics of **skip** and sequential composition; 3. fixpoint semantics of loops.

¹The distinction between $\mathcal{W}_{\mathcal{A}}(\Sigma)$ and $\mathcal{W}_{\mathcal{A}}^\infty(\Sigma)$ will be explained when we discuss the semantics of loops.

$$\begin{aligned}
\llbracket \mathbf{skip} \rrbracket (\sigma) &\triangleq \eta(\sigma) & \llbracket a \rrbracket (\sigma) &\triangleq \llbracket a \rrbracket_{\text{Act}} (\sigma) \\
\llbracket C_1 \circ C_2 \rrbracket (\sigma) &\triangleq \llbracket C_2 \rrbracket^\dagger (\llbracket C_1 \rrbracket (\sigma)) & \llbracket C_1 + C_2 \rrbracket (\sigma) &\triangleq \llbracket C_1 \rrbracket (\sigma) + \llbracket C_2 \rrbracket (\sigma) \\
\llbracket \mathbf{assume } e \rrbracket (\sigma) &\triangleq \llbracket e \rrbracket (\sigma) \cdot \text{unit}(\sigma) & \llbracket C^{(e_1, e_2)} \rrbracket (\sigma) &\triangleq \text{lfp } f. (\Phi_{C^{(e_1, e_2)}}(f))(\sigma) \\
\text{where } \Phi_{C^{(e_1, e_2)}}(f)(\sigma) &\triangleq \llbracket e_1 \rrbracket (\sigma) \cdot f^\dagger(\llbracket C \rrbracket (\sigma)) + \llbracket e_2 \rrbracket (\sigma) \cdot \eta(\sigma)
\end{aligned}$$

Fig. 2. Denotational semantics $\llbracket C \rrbracket : \Sigma \rightarrow \mathcal{W}_{\mathcal{A}}^\infty(\Sigma_\cup)$ of programs, parametric on a set of program states Σ ; an interpretation $\llbracket a \rrbracket_{\text{Act}} : \Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma)$ for atomic actions $a \in \text{Act}$; and $\text{Test} \subseteq 2^\Sigma$, the set of primitive tests.

Guards and Branching. Recall that guards e can be Boolean expressions b or weights $u \in U$. We define the semantics of guard expressions e as $\llbracket e \rrbracket : \Sigma \rightarrow U$, where $\llbracket b \rrbracket (\sigma) \triangleq \llbracket b \rrbracket_{\text{Test}} (\sigma)$ and $\llbracket u \rrbracket (\sigma) \triangleq u$. The semantics of Boolean guards $\llbracket b \rrbracket_{\text{Test}} : \Sigma \rightarrow \{0, 1\}$ is a simple extension of the set-element predicate (where $0, 1$ are the semiring identities), since primitive tests are sets of program states $\text{Test} \subseteq 2^\Sigma$. The full definition is available in Appendix A.

The semantics of **assume** e uses a product operation $\cdot$ on weighting functions whereas a $+$ operation is used in the definition of $\llbracket C_1 + C_2 \rrbracket$. These are operations on weighting functions that are lifted from the semiring $+$ and $\cdot$ pointwise: $(m_1 + m_2)(\sigma) \triangleq m_1(\sigma) + m_2(\sigma)$ and $(u \cdot m)(\sigma) \triangleq u \cdot m(\sigma)$.

Monadic structure of weighting functions. We next turn to the semantics of sequential composition and **skip**, which is achieved using a monad [Manes 1976; Moggi 1991].

Definition 2.5 (Monads). A Kleisli triple $\langle T, \eta, (-)^\dagger \rangle$ in the category **Set** consists of a functor $T : \mathbf{Set} \rightarrow \mathbf{Set}$, and two morphisms $\eta : \text{Id} \Rightarrow T$ and, for any sets X and Y , $(-)^\dagger : (X \rightarrow T(Y)) \rightarrow T(X) \rightarrow T(Y)$ such that $\eta^\dagger = \text{id}$, $f^\dagger \circ \eta = f$, and $f^\dagger \circ g^\dagger = (f^\dagger \circ g^\dagger)$. If such a triple exists, then the functor T is a monad.

For any semiring $\mathcal{A}$, $\langle \mathcal{W}_{\mathcal{A}}, \eta, (-)^\dagger \rangle$ is a Kleisli triple, with functions η and $(-)^\dagger$ defined as follows:

$$\eta(x)(y) \triangleq \begin{cases} 1 & \text{if } x = y \\ 0 & \text{if } x \neq y \end{cases} \quad f^\dagger(m)(y) \triangleq \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(x) \cdot f(x)(y) + m(\cup) \cdot \eta(\cup)(y)$$

Note how the Kleisli extension $f^\dagger(m)$ discriminates between weight allocated to program states (in Σ) and weight allocated to $\cup$ by m . Intuitively, this captures the notion that any program state occupied by a weighted program can be passed along to the next step of the computation, whereas any accrued nontermination persists and must be *carried through*. Although we define the monad operations directly here, $\mathcal{W}_{\mathcal{A}}$ can also be defined via a distributive law [Beck 1969] between the weighting function monad of Outcome Logic [Zilberstein 2024a] and the $+1$ or *error* monad, which is known to compose with all other monads [Lüth and Ghani 2002].

Loops. Finally, we discuss the semantics of iterated computations, which requires $\mathcal{W}_{\mathcal{A}}$ to be a directed complete partial order (dcpo). We use a *fusion order*: $m_1 \sqsubseteq m_2$ if and only if the weight of each regular state increases and the weight of nontermination decreases. This is similar to the fixpoint maximal trace semantics of Cousot [2002] in which finite traces are compared covariantly and infinite traces are compared contravariantly. This order can also be viewed as an *approximation order* [Abramsky and Jung 1995]— $m_1 \sqsubseteq m_2$ means that m_2 contains more information about the program's behavior than m_1 . As such, we initially consider the behavior to be nontermination (bottom), and the approximation gets larger as more and more terminating executions are discovered.

Definition 2.6 (Fusion Order). We define the *fusion order* $\sqsubseteq$ on $\mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$ as:

$$m_1 \sqsubseteq m_2 \quad \text{iff} \quad m_1(\sigma) \leq m_2(\sigma) \text{ for all } \sigma \in \Sigma, \text{ and } m_1(\cup) \geq m_2(\cup)$$

Above, $\leq$ is the *natural order* on $\mathcal{A}$: $u \leq v$ iff $u + w = v$ for some w . If $\langle U, \leq \rangle$ is a partial order, then the semiring is *naturally-ordered*. The semiring is *bounded* if there exists a top element $\top$ such that $u \leq \top$ for all $u \in U$. This top element is sometimes required to be an infinity, as defined below.

Definition 2.7 (Infinity [Golan 1999]). An element w in semiring $\langle U, +, \cdot, 0, 1 \rangle$ is an *infinity* if it is additively absorbing: $u + w = w + u = w$ for all $u \in U$. In addition, w is a *strong infinity* if it is infinite as well as multiplicatively absorbing: $x \cdot w = w \cdot x = w$ for all $x \neq 0$.

We also need a notion of *bounded weighting*, which generalizes bounded nondeterminism, and is crucial for ensuring the standard continuity results for our semantics.

Definition 2.8 (Bounded Weighting). We define the following classes of weighting functions:

- $\mathcal{W}_{\mathcal{A}}^+(\Sigma) \triangleq \{m \in \mathcal{W}_{\mathcal{A}}(\Sigma \cup) \mid \text{supp}(m) \text{ is finite}\}$
- $\mathcal{W}_{\mathcal{A}}^\omega(\Sigma) \triangleq \{m \in \mathcal{W}_{\mathcal{A}}(\Sigma \cup) \mid m(\cup) = \sup_{m' \in E(m)} m'(\cup)\}$.
- $\mathcal{W}_{\mathcal{A}}^\infty(\Sigma) \triangleq \mathcal{W}_{\mathcal{A}}^+(\Sigma) \cup \mathcal{W}_{\mathcal{A}}^\omega(\Sigma)$

For any particular $m \in \mathcal{W}_{\mathcal{A}}(\Sigma \cup)$, let $E(m)$ be the set of weighting functions that assign equal weight to all program states in Σ (i.e., that can only differ in the weight on nontermination $\cup$):

$$E(m) \triangleq \{m' \in \mathcal{W}_{\mathcal{A}}(\Sigma \cup) \mid \forall \sigma \in \Sigma. m'(\sigma) = m(\sigma)\}$$

A mapping $\Sigma \rightarrow \mathcal{W}_{\mathcal{A}}^\infty(\Sigma)$ is said to exhibit **bounded weighting**.

Our notion approximates the operational behavior captured by bounded nondeterminism, which stipulates that a nondeterministic program can only select between finitely many branches at each step of execution. Drawing from the definition of Back [1983] for a powerdomain semantics, this means that running a nondeterministic program must produce a set of outcomes that is either (i) finite or (ii) exhibits nontermination (such that infinite sets of outcomes must be generated by some nonterminating trace). The sets $\mathcal{W}_{\mathcal{A}}^+(\Sigma)$ and $\mathcal{W}_{\mathcal{A}}^\omega(\Sigma)$ are the analogue of Back's two cases, respectively; weightings produced by programs must either:

- (1) Describe a finite set of outcomes (finite support) (that is, be in $\mathcal{W}_{\mathcal{A}}^+(\Sigma)$); or
- (2) Maximize the weight on nontermination $\cup$ (that is, be in $\mathcal{W}_{\mathcal{A}}^\omega(\Sigma)$).

This can also be seen as a generalization of the Plotkin powerdomain, which includes all finite sets of states and infinite states that contain $\cup$ [Plotkin 1976; Søndergaard and Sestoft 1992]. Here, the choice to *maximize* nontermination lies in our approach to tracking the presence of nontermination in weighting functions. In particular, we support two semantic paradigms and correspondingly require the semiring of weights $\mathcal{A}$ to belong to one of two classes.

- (1) **Conservative Weighting:** $\mathcal{A}$ is partial and bounded with *non-idempotent* top element $\top$ such that, for all $x \in X \setminus \{0\}$, $x + \top$ and $\top + x$ are not defined. So, only $\top + 0 = 0 + \top = \top$. In this case, we choose our weighting functions to be drawn from $\mathcal{D} \triangleq \{m \in \mathcal{W}_{\mathcal{A}}^\infty(\Sigma) \mid |m| = \top\}$. Thus, programs under a *conservative weighting* scheme always produce a weighting function of fixed mass $\top$, which is conserved across sequences of programs. Examples include probabilistic and deterministic programs, for which the mass of weighting functions is fixed at 1.
- (2) **Indicative Weighting:** $\mathcal{A}$ is total and bounded with a *strongly infinite* top element $\top$. Since the semiring is total, we can rewrite

$$\mathcal{W}_{\mathcal{A}}^\omega(\Sigma) \triangleq \{m \in \mathcal{W}_{\mathcal{A}}(\Sigma \cup) \mid m(\cup) = \top\}$$

and limit weighting functions to $\mathcal{D} \triangleq \mathcal{W}_{\mathcal{A}}^\infty(\Sigma)$. We refer to this as an *indicative weighting* scheme in that the weight on $\cup$ serves as an indicator for nontermination; we assign a weight of 0 if the computation does not produce a diverging trace, and assign infinite weight otherwise.

At first, the introduction of these restrictions may seem to fetter the generalizability of our approach. However, we note that any semiring of weights $\mathcal{A} = \langle U, +, \cdot, \emptyset, \mathbb{1} \rangle$ that does not belong to either of the above classes can be extended with a strongly infinite element [Golan 1999, p. 166]. In particular, we adjoin to $\mathcal{A}$ an element $\infty \notin U$ to accommodate an *indicative* weighting scheme. Addition and multiplication can be defined such that $\infty + u = u + \infty = \infty$ for all $u \in U$; $\infty \cdot u = u \cdot \infty = \infty$ for all $u \in U \setminus \{\emptyset\}$; and $\infty \cdot \emptyset = \emptyset \cdot \infty = \emptyset$. Doing so limits our (indicative) semantics to a coarser representation of program behavior, foregoing attempts at tracking finer weights with which a program can diverge, and instead only maintaining whether nontermination is possible or not. Nevertheless, this is sufficient for most purposes, and in the case for some semirings, necessary to preserve continuity, which is defined below.

Definition 2.9 (Scott-Continuity). A semiring $\langle U, +, \cdot, \emptyset, \mathbb{1} \rangle$ with order $\leq$ is *Scott-continuous* if for any directed set $D \subseteq U$ (where all pairs of elements in D have a supremum):

$$\sup_{u \in D} (u + v) = (\sup D) + v \quad \sup_{u \in D} (u \cdot v) = (\sup D) \cdot v \quad \sup_{u \in D} (v \cdot u) = v \cdot (\sup D)$$

The semiring is *lower Scott-continuous* if the same operations preserve infima.

For Scott-continuous semirings, we can define an infinite sum operation over an index set I as the supremum of sums over all finite subsets of I .

$$\sum_{i \in I} u_i = \sup \left\{ \sum_{i \in J} u_i \mid J \subseteq I \text{ finite} \right\}$$

This construction yields a *complete* semiring, meaning that the sum operator obeys the following desirable laws [Kuich 2011]:

- (1) For $I = \{i_1, \dots, i_n\}$ finite, $\sum_{i \in I} u_i = u_{i_1} + \dots + u_{i_n}$.
- (2) If $I = \bigcup_{k \in K} J_k$ is a disjoint union of nonempty sets, then $\sum_{k \in K} \sum_{j \in J_k} u_j = \sum_{i \in I} u_i$.
- (3) If $\sum_{i \in I} u_i$ is defined, then $v \cdot \sum_{i \in I} u_i = \sum_{i \in I} v \cdot u_i$ and $\left(\sum_{i \in I} u_i \right) \cdot v = \sum_{i \in I} u_i \cdot v$.

The above definitions are what we need to fully define the denotational semantics for our language, and in particular the semantics of loops. In particular, we require $\mathcal{A} = \langle U, +, \cdot, \emptyset, \mathbb{1} \rangle$ to be a naturally ordered, finitary, Scott-continuous, partial semiring bounded by top element $\top$.

3 Total Outcome Logic

We now leverage our semantics to define *Total Outcome Logic* (TOL), building atop the proof system given by Zilberstein [2024a] to also reason about the possible divergence and certain termination of programs. We will later see that TOL is strictly more expressive than Outcome Logic; by exposing nontermination as a program outcome, TOL is capable of expressing additional properties with termination and nontermination guarantees, including the classical notion of *total correctness*.

3.1 Outcome Assertions

First, we define *outcome assertions*, which serve as pre- and postconditions in TOL. In many logics, pre- and postconditions are represented extensionally as *sets of program states*. Outcome assertions, on the other hand, must operate on a finer level, specifying not only a collection of states but also their weights at the respective point in the computation. We give extensional definitions of outcome assertions $\varphi, \psi \in 2^{\mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)}$ as sets of weighting functions. For any $m \in \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)$, we write $m \models \varphi$ (read as *m satisfies φ*) to mean $m \in \varphi$. Below, we introduce notation for assertions used later

in constructing the logic. We start with basic propositional constructs, which are defined as usual:

$$\begin{aligned} \top &\triangleq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) & \perp &\triangleq \emptyset & \neg\varphi &\triangleq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) \setminus \varphi \\ \varphi \vee \psi &\triangleq \varphi \cup \psi & \varphi \wedge \psi &\triangleq \varphi \cap \psi & \varphi \Rightarrow \psi &\triangleq \neg\varphi \vee \psi \end{aligned}$$

Given an assertion on program states $P \subseteq \Sigma$, we define liftings to indicate that P over-approximates, under-approximates, or exactly characterizes the support of a weighting function with weight u . Similarly, $\uparrow^{(u)}$ denotes that the nontermination outcome $\cup$ occurs with weight $u \cdot \top$.

$$\begin{aligned} [P]^{(u)} &\triangleq \{m \mid |m| = u, \text{supp}(m) \subseteq P\} & [P]^{(u)} &\triangleq \{m \mid |m| = u, \text{supp}(m) \supseteq P\} \\ [P]^{(u)} &\triangleq \{m \mid |m| = u, \text{supp}(m) = P\} & \uparrow^{(u)} &\triangleq \{u \cdot \top \cdot \eta(\cup)\} \end{aligned}$$

When the superscript is omitted, then the weight is exactly 1, i.e., $[P] = [P]^{(1)}$. Operations $u \odot \varphi$ and $\varphi \odot u$ scale the outcome assertion φ with weight u on the left or right. Existential quantification with respect to predicate $\phi : T \rightarrow 2^{\mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)}$ is defined as the union of all $\phi(t)$ for inputs $t \in T$. Then, $m \models \exists x : T. \phi(x)$ precisely when m satisfies $\phi(t)$ for *some* input t .

$$u \odot \varphi \triangleq \{u \cdot m \mid m \in \varphi\} \quad \varphi \odot u \triangleq \{m \cdot u \mid m \in \varphi\} \quad \exists x : T. \phi(x) \triangleq \bigcup_{t \in T} \phi(t)$$

We write the *outcome conjunction* $\varphi \oplus \psi$ to consist of weighting functions m that can be split into components $m = m_1 + m_2$, with m_1 satisfying φ and m_2 satisfying ψ . We define this more generally for a potentially infinite index set T :

$$\bigoplus_{x \in T} \phi(x) \triangleq \left\{ \sum_{t \in T} m_t \mid \forall t \in T. m_t \in \phi(t) \right\}$$

Finally, we assert *identity* to m with $1(m)$, which consists of the singleton $\{m\}$.

3.2 Total Outcome Logic

The judgements of Total Outcome Logic are *outcome triples*, which take the form $\langle \varphi \rangle C \langle \psi \rangle$ for a precondition φ , command C , and postcondition ψ .

Definition 3.1 (Outcome Triples). The validity of outcome triples is defined as follows:

$$\models \langle \varphi \rangle C \langle \psi \rangle \quad \text{iff} \quad \forall m \in \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma). \quad m \models \varphi \implies \llbracket C \rrbracket^{\dagger}(m) \models \psi$$

The main reasoning apparatus in TOL consists of inference rules for deriving triples, shown in Figures 3 and 4. Some of these rules carry over from Outcome Logic [Zilberstein 2024a], while others must be adapted to account for nontermination. We introduce the rules as belonging to three main categories: structural rules, standard command rules, and iteration.

Structural Rules are provided in Figure 3 and hold for arbitrary program commands C , as they capture logical rules that apply to pre- and post-conditions. These include the trivial rules, **FALSE** and **TRUE**, which feature the vacuous precondition $\perp$ and the weakest postcondition $\top$, respectively. **DIV** handles nontermination, stating that assertions $\uparrow^{(u)}$ are preserved across programs.

A given triple can be left-scaled by any weight $u \in U$ using **SCALE**. The rules **DISJ**, **CONJ**, and **CHOICE** allow multiple triples to be conjoined via the connectives $\wedge$, $\vee$, and $\oplus$, respectively. Introduction of existential quantification is accomplished through **EXISTS**. Finally, the standard **CONSEQUENCE** rule allows for strengthening preconditions and weakening postconditions.

Standard Command Rules are provided in Figure 4 to specify how non-iterative program commands transform outcome assertions. **SKIP** and **SEQ** are standard for Hoare-style logics. The **ASSUME** rule has the premise $\varphi \models e = u$, which is defined below:

Definition 3.2 (Assertion Entailment). Given an assertion φ , expression e , and weight $u \in U$, we write $\varphi \models e = u$ (φ entails $e = u$) iff for all $m \models \varphi$, $\cup \notin \text{supp}(m)$ and $\llbracket e \rrbracket(\sigma) = u$ for all $\sigma \in \text{supp}(m)$.

$$\begin{array}{c}
\frac{}{\langle \perp \rangle C \langle \varphi \rangle} \text{FALSE} \quad \frac{}{\langle \varphi \rangle C \langle \top \rangle} \text{TRUE} \quad \frac{}{\langle \uparrow^{(u)} \rangle C \langle \uparrow^{(u)} \rangle} \text{DIV} \quad \frac{\langle \varphi \rangle C \langle \psi \rangle}{\langle u \odot \varphi \rangle C \langle u \odot \psi \rangle} \text{SCALE} \\
\\
\frac{\langle \varphi_1 \rangle C \langle \psi_1 \rangle \quad \langle \varphi_2 \rangle C \langle \psi_2 \rangle}{\langle \varphi_1 \vee \varphi_2 \rangle C \langle \psi_1 \vee \psi_2 \rangle} \text{DISJ} \quad \frac{\langle \varphi_1 \rangle C \langle \psi_1 \rangle \quad \langle \varphi_2 \rangle C \langle \psi_2 \rangle}{\langle \varphi_1 \wedge \varphi_2 \rangle C \langle \psi_1 \wedge \psi_2 \rangle} \text{CONJ} \\
\\
\frac{\forall t \in T. \langle \phi(t) \rangle C \langle \phi'(t) \rangle}{\langle \bigoplus_{x \in T} \phi(x) \rangle C \langle \bigoplus_{x \in T} \phi'(x) \rangle} \text{CHOICE} \quad \frac{\forall t \in T. \langle \phi(t) \rangle C \langle \phi'(t) \rangle}{\langle \exists x : T. \phi(x) \rangle C \langle \exists x : T. \phi'(x) \rangle} \text{EXISTS} \\
\\
\frac{\varphi' \implies \varphi \quad \langle \varphi \rangle C \langle \psi \rangle \quad \psi \implies \psi'}{\langle \varphi' \rangle C \langle \psi' \rangle} \text{CONSEQUENCE}
\end{array}$$

Fig. 3. Structural rules.

That is, for every weighting function $m \models \varphi$ and any $\sigma \in \text{supp}(m)$, e evaluates to u in state σ . For tests $b \in \text{Test}$, we introduce the shorthand $\varphi \models b$ to mean $\varphi \models b = \mathbb{1}$.

In **PLUS**, the side condition $\varphi \models \text{true}$ is equivalent to a sure-termination guarantee: for $m \in \varphi$, $\cup \notin \text{supp}(m)$. If this holds, the triples proved for both branches are combined with outcome conjunction. Sure-termination must be enforced in these cases as the additional effect of nontermination is sequenced differently than for regular weighted states. Similar issues arise in instances of OL with exceptions [Zilberstein et al. 2023, 2024a] and probabilistic nondeterminism [Zilberstein et al. 2025].

Iteration. Our nontermination semantics culminates in the **ITER** rule for iteration commands $C^{(e, e')}$ (Figure 4), which also serves to introduce any assertions pertaining to nontermination. The form of this rule corresponds roughly to an unrolling of the loop and describes sequences of assertions that hold between iterations. These assertions come in three sorts:

- $(\varphi_n)_{n \in \mathbb{N}}$ describe ongoing lines of computation, i.e., those that have not yet terminated or those that have been deemed nonterminating. These include program configurations that are passed to the next iteration of the loop and subsequently transformed. Note that we have as premise that $\varphi_n \models \text{true}$ for every n , so every $m \in \varphi_n$ describes only weights on program states Σ , but not the weight on $\cup$ —we call computations restricted to the states Σ -computations.
- $(\psi_n)_{n \in \mathbb{N}}$ describe the terminating outcomes that are accrued in iteration n . In particular, observe that ψ_n is obtained by filtering φ_n through command **assume** e' : $\langle \varphi_n \rangle \text{ assume } e' \langle \psi_n \rangle$.
- $(\zeta_n)_{n \in \mathbb{N}}$ describe nontermination encountered in the n^{th} iteration (i.e., via nested loops). We write $\zeta_n \models \text{div}$ as a premise to enforce this characterization.

Definition 3.3 (Nonterminating Assertions). For outcome assertion φ , we write $\varphi \models \text{div}$ (read as: φ is nonterminating) iff for all $m \in \varphi$, $\text{supp}(m) \subseteq \{\cup\}$.

The conclusion of the rule has as postcondition two *limiting* assertions ψ_∞ and ζ_∞ . Intuitively, the former describes the aggregate of all terminating outcomes accrued by each ψ_n . We represent this formally by writing $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$, which is defined below:

Definition 3.4 (Converging Assertions). A family $(\psi_n)_{n \in \mathbb{N}}$ of assertions converges to ψ_∞ (written $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$) iff for any $(m_n)_{n \in \mathbb{N}}$, if $m_n \models \psi_n$ for each $n \in \mathbb{N}$, then $\sum_{n \in \mathbb{N}} m_n \models \psi_\infty$.

On the other hand, ζ_∞ describes the degree of nontermination of the loop in the limit. This notion is captured by the semantic assertion $(\varphi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty$, which we take to mean:

Definition 3.5 (Diverging Assertions). A family $(\psi_n)_{n \in \mathbb{N}}$ of outcome assertions diverges to ψ_∞ (written $(\psi_n)_{n \in \mathbb{N}} \uparrow \psi_\infty$) iff for all $(m_n)_{n \in \mathbb{N}}$, if $m_n \models \psi_n$ for all $n \in \mathbb{N}$, then $(\inf_{n \in \mathbb{N}} |m_n| \cdot \top) \cdot \eta(\cup) \models \psi_\infty$.

$$\begin{array}{c}
\frac{}{\langle \varphi \rangle \text{ skip } \langle \varphi \rangle} \text{SKIP} \qquad \frac{\langle \varphi \rangle C_1 \langle \zeta \rangle \quad \langle \zeta \rangle C_2 \langle \psi \rangle}{\langle \varphi \rangle C_1 ; C_2 \langle \psi \rangle} \text{SEQ} \\
\\
\frac{\varphi \models \text{true} \quad \langle \varphi \rangle C_1 \langle \psi_1 \rangle \quad \langle \varphi \rangle C_2 \langle \psi_2 \rangle}{\langle \varphi \rangle C_1 + C_2 \langle \psi_1 \oplus \psi_2 \rangle} \text{PLUS} \qquad \frac{\varphi \models e = u}{\langle \varphi \rangle \text{ assume } e \langle \varphi \odot u \rangle} \text{ASSUME} \\
\\
\frac{\varphi_n \models \text{true} \quad \zeta_n \models \text{div} \quad \begin{array}{c} (\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty \quad (\varphi_n \oplus \zeta_n)_{n \in \mathbb{N}} \Uparrow \zeta_\infty \quad \forall n \in \mathbb{N}. \\ \langle \varphi_n \oplus \zeta_n \rangle \text{ assume } e ; C \langle \varphi_{n+1} \oplus \zeta_{n+1} \rangle \quad \langle \varphi_n \rangle \text{ assume } e' \langle \psi_n \rangle \end{array}}{\langle \varphi_0 \oplus \zeta_0 \rangle C^{(e,e')} \langle \psi_\infty \oplus \zeta_\infty \rangle} \text{ITER}
\end{array}$$

Fig. 4. Inference rules for program commands.

In other words, the total mass in $\varphi_n \oplus \zeta_n$ in the limit is transferred to the nontermination outcome $\cup$. Intuitively, nontermination in the loop has two possible sources: nonterminating weight from nested loops, which is accounted for by ζ_n 's, and weight leftover from Σ -computations that never end, accounted for by the φ_n 's. This rule allows us to explain the paradox that we introduced in Section 1, in which terminating nondeterministic programs cannot make unboundedly many choices, but almost surely terminating probabilistic programs can. Consider the two programs below, which both increment x in a loop, but the program on the left is interpreted in the Boolean semiring, whereas the one on the right is interpreted in the probabilistic semiring.

$$x := 0 ; (x := x + 1)^{(1,1)} \qquad x := 0 ; (x := x + 1)^{(p,1-p)}$$

For the nondeterministic program, we set $\varphi_n \triangleq \lceil x = n \rceil$, $\psi_n \triangleq \lceil x = n \rceil$, and $\zeta_n \triangleq \lceil \text{true} \rceil^{(0)}$ for all $n \in \mathbb{N}$. Letting $\psi_\infty \triangleq \bigoplus_{k \in \mathbb{N}} \lceil x = k \rceil$, clearly $\lceil x = n \rceil_{n \in \mathbb{N}} \rightsquigarrow \bigoplus_{k \in \mathbb{N}} \lceil x = k \rceil$, indicating that there is a terminating outcome where $x = k$ for any natural number k . Letting $\zeta_\infty \triangleq \Uparrow$, we also have $\lceil x = n \rceil_{n \in \mathbb{N}} \Uparrow \zeta_\infty$; the weight of continuing to iterate remains to be 1 for all $n \in \mathbb{N}$, so there is a nonterminating trace of weight 1, giving us the final specification.

$$\langle \lceil \text{true} \rceil \rangle x := 0 ; (x := x + 1)^{(1,1)} \langle \left(\bigoplus_{k \in \mathbb{N}} \lceil x = k \rceil \right) \oplus \Uparrow \rangle$$

By contrast, in the probabilistic program we instead set $\varphi_n \triangleq \lceil x = n \rceil^{(p^n)}$ and $\psi_n \triangleq \lceil x = n \rceil^{(p^n \cdot (1-p))}$. The terminating outcomes then form a geometric distribution $\bigoplus_{k \in \mathbb{N}} \lceil x = k \rceil^{(p^k \cdot (1-p))}$. Since the weight captured by each φ_n decreases towards 0, the infimum is 0, giving us $\zeta_\infty \triangleq \lceil \text{true} \rceil^{(0)}$, so the infinite execution occurs with zero probability, and it is therefore omitted from the specification.

$$\langle \lceil \text{true} \rceil \rangle x := 0 ; (x := x + 1)^{(p,1-p)} \langle \bigoplus_{k \in \mathbb{N}} \lceil x = k \rceil^{(p^k \cdot (1-p))} \rangle$$

REMARK 1 (ACCOMMODATING NONTERMINATION). *In general, applying rules with some assertion entailment $\varphi \models e = u$ as a premise involves decomposing the precondition into components that describe terminating vs. nonterminating outcomes – the latter of which are handled by **DIV** or some derivative of it. As an example, take $\varphi = \varphi_t \oplus \varphi_\cup$ where $\varphi_t \models \text{true}$ and $\varphi_\cup \models \text{div}$. We show a derivation involving **PLUS** across program $C_1 + C_2$, in which we are given premises $\langle \varphi_t \rangle C_1 \langle \psi_{t_1} \rangle$ and $\langle \varphi_t \rangle C_2 \langle \psi_{t_2} \rangle$:*

$$\frac{\frac{\langle \varphi_t \rangle C_1 \langle \psi_{t_1} \rangle \quad \langle \varphi_t \rangle C_2 \langle \psi_{t_2} \rangle}{\langle \varphi_t \rangle C_1 + C_2 \langle \psi_{t_1} \oplus \psi_{t_2} \rangle} \text{PLUS} \quad \frac{\varphi_\cup \models \text{div}}{\langle \varphi_\cup \rangle C_1 + C_2 \langle \varphi_\cup \rangle} \text{DIV}^*}{\langle \varphi_t \oplus \varphi_\cup \rangle C_1 + C_2 \langle \psi_{t_1} \oplus \psi_{t_2} \oplus \varphi_\cup \rangle} \text{CHOICE}$$

Soundness and Relative Completeness. The proof system that we defined for Total Outcome Logic is sound and complete. Soundness means that every triple that is derivable via the inference rules is semantically valid. We state this theorem below, and prove it in the appendix.

THEOREM 3.6 (SOUNDNESS). $\vdash \langle \varphi \rangle C \langle \psi \rangle \implies \models \langle \varphi \rangle C \langle \psi \rangle$.

Relative completeness means that any valid triple can be derived in our proof system, provided an oracle that decides semantic properties such as the implications used in the rule of **CONSEQUENCE** and the assertion entailments used in the **ASSUME** rule.

THEOREM 3.7 (RELATIVE COMPLETENESS). $\models \langle \varphi \rangle C \langle \psi \rangle \implies \vdash \langle \varphi \rangle C \langle \psi \rangle$

4 The Verscht and Kaminski Taxonomy

Starting with Hoare Logic, there is an expansive body of work in developing new program logics to reason across different computational domains and effects. More recently, an opposite impulse to unify these theories has taken shape in work on taxonomies of logic, which seek to abstract from these specific frameworks and uncover underlying formalisms that relate them.

One such example is **Verscht and Kaminski’s** [2025] taxonomy of *Hoare-like* logics, a collection of 16 logics for expressing different properties about nondeterministic, looping programs. These logics are roughly organized along three axes: *correctness vs incorrectness*, *partiality vs totality*, and *angelic vs demonic* nondeterminism. The taxonomy shows how 16 logics relate to each other in terms of weakening, contraposition, and Galois connections, while also proposing as-of-yet unexplored logics, including certain *in-between* ones. We have adapted the taxonomy into a double cube structure, shown in Figure 5. Our taxonomy contains only 14 logics; two of **Verscht and Kaminski’s** logics are redundant due to Galois connections, as we explain in Section 4.2. The *in-between* logics, which appear in the interior of the upper cube, are added in Section 4.3.

The taxonomy formulates logics in terms of predicate transformers—variants of *weakest precondition* and *strongest postcondition*. Some of these transformers are well-known, while others are entirely new, neatly completing the picture. In this section, we show that TOL subsumes every logic in this taxonomy. Interestingly, TOL being lifted to reason directly on the sets of outcomes eliminates the need for rigid distinctions between logics, and instead allows us to specify many different kinds of properties just by altering the postcondition. This is why, as we see in Section 4.3, TOL is readily suited for the *in-between* cases, and provides robust proof theory for a shared analysis across different modes of reasoning (Section 6). Throughout the remaining sections, we will work in a restricted domain that excludes the empty weighting function, meaning that running a program must always result in some outcome, whether it be a reachable state or nontermination.

4.1 Weakest Precondition Logics: The Upper Cube

The upper cube in Figure 5 is comprised of logics defined by different flavors of *weakest precondition* transformers. Originally due to **Dijkstra** [1976], these transformers initially had two variants: the weakest precondition of a program C and a postcondition Q is the set of states that always terminate in Q after C is run, whereas the weakest *liberal* precondition is the set of start states that end in Q if the program terminates. Both of these transformers treat nondeterminism demonically, so **Verscht and Kaminski** [2025] refer to them as the demonic weakest precondition (dwp) and demonic weakest liberal precondition (dwlp), respectively.

$$\text{dwp}(C, Q) \triangleq \{\sigma \mid \text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q\} \quad \text{dwlp}(C, Q) \triangleq \{\sigma \mid \text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q_{\cup}\}$$

It is well known that total and partial correctness Hoare Logic [**Hoare** 1969; **Manna and Pnueli** 1974] can be framed in terms of these transformers. That is, the total correctness Hoare Triple

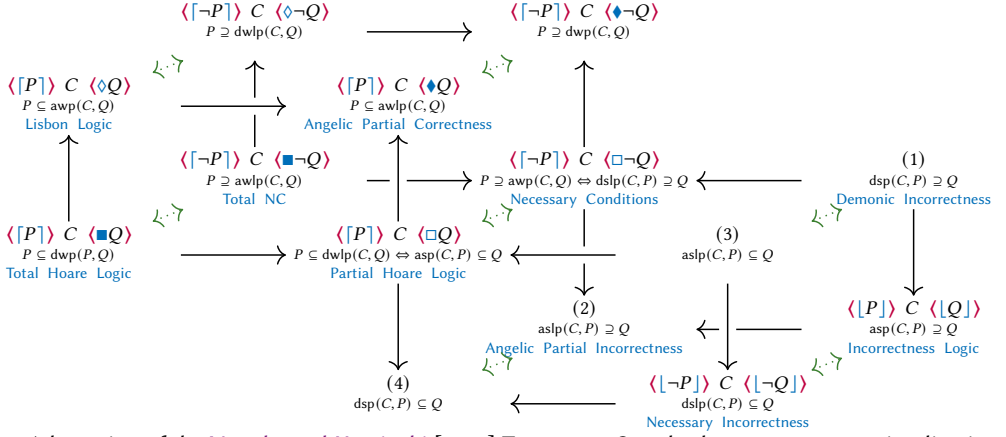


Fig. 5. Adaptation of the Verscht and Kaminski [2025] Taxonomy. Standard arrows represent implications and green dotted arrows represent contrapositives. The numbered logics are represented as two triples: (1) $\langle [\neg P] \rangle C \langle \square \neg Q \rangle \wedge \langle [P] \rangle C \langle \blacklozenge Q \rangle$, (2) $\langle [\neg P] \rangle C \langle \square \neg Q \rangle \vee \langle [P] \rangle C \langle \blacklozenge Q \rangle$, (3) $\langle [P] \rangle C \langle \square Q \rangle \wedge \langle [\neg P] \rangle C \langle \blacklozenge \neg Q \rangle$, and (4) $\langle [P] \rangle C \langle \square Q \rangle \vee \langle [\neg P] \rangle C \langle \blacklozenge \neg Q \rangle$.

$\{P\} C \{Q\}$ is equivalent to $P \subseteq \text{dwp}(C, Q)$ and the partial correctness triple $\{P\} C \{Q\}$ is equivalent to $P \subseteq \text{dwlpl}(C, Q)$. Of course, the above transformers can also be formulated angelically.

$$\text{awp}(C, Q) \triangleq \{\sigma \mid \text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q \neq \emptyset\} \quad \text{awlp}(C, Q) \triangleq \{\sigma \mid \text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q_{\cup} \neq \emptyset\}$$

The angelic weakest precondition (awp), originally due to Hoare [1978] under the name *weakest possible precondition*, is the set of states that reach Q via *some* trace in C . The resulting logic $P \subseteq \text{awp}(C, Q)$ —known as Lisbon Logic [Zilberstein et al. 2023], Backwards Under-Approximation [Möller et al. 2021], and Sufficient Incorrectness Logic [Ascari et al. 2023]—is useful for proving incorrectness, i.e., that certain bugs are reachable. The angelic weakest liberal precondition is less well studied. It is the set of states that either reach the postcondition or diverge, and was introduced by Verscht and Kaminski [2025] in order to complete the front face of the upper cube in Figure 5. It is related to the logic UNTer, a logic for proving possible nontermination [Raad et al. 2024]. Indeed, $P \subseteq \text{awlp}(C, \text{false})$ means that for each state in P , there exists a diverging trace. We encode these logics in TOL using the following modalities; recall that here we assume that $\text{supp}(m) \neq \emptyset$.

$$\begin{aligned} \square P &\triangleq \exists(u, v) : U^2 \setminus \{(\emptyset, \emptyset)\}. [P]^{(u)} \oplus \uparrow^{(v)} &= \{m \mid \text{supp}(m) \subseteq P_{\cup}\} \\ \diamond P &\triangleq \exists u : U \setminus \{\emptyset\}. [P]^{(u)} \oplus \top &= \{m \mid \text{supp}(m) \cap P \neq \emptyset\} \\ \blacksquare P &\triangleq \exists u : U \setminus \{\emptyset\}. [P]^{(u)} &= \{m \mid \text{supp}(m) \subseteq P\} \\ \blacklozenge P &\triangleq \exists(u, v) : U^2 \setminus \{(\emptyset, \emptyset)\}. [P]^{(u)} \oplus \uparrow^{(v)} \oplus \top &= \{m \mid \text{supp}(m) \cap P_{\cup} \neq \emptyset\} \end{aligned}$$

The $\square$ and $\diamond$ modalities are inspired by Dynamic Logic [Harel et al. 2001; Pratt 1976] and correspond to dwlpl and awp , respectively. That is, $\square Q$ states that Q covers all the terminating states in some weighting function m , and $\diamond Q$ states that m contains some state in Q . As was shown by Zilberstein [2024a], $\square$ and $\diamond$ can be used to encode partial correctness Hoare Logic and Lisbon Logic:

THEOREM 4.1 (SUBSUMPTION OF HOARE AND LISBON LOGIC).

$$P \subseteq \text{dwlpl}(C, Q) \iff \models \langle [P] \rangle C \langle \square Q \rangle \quad \text{and} \quad P \subseteq \text{awp}(C, Q) \iff \models \langle [P] \rangle C \langle \diamond Q \rangle$$

The $\blacksquare$ and $\blacklozenge$ modalities witness a particular (zero or nonzero) amount of termination mass, and were therefore not expressible in standard Outcome Logic [Zilberstein 2024a; Zilberstein et al. 2023, 2024a]. More precisely $\blacksquare Q$ states not only that Q covers all the reachable states, but also that $\cup$ has

weight zero. Finally, $\blacklozenge Q$ states that either Q or $\cup$ occur with nonzero weight. These new modalities can be used to express the final logics:

THEOREM 4.2 (SUBSUMPTION OF TOTAL HOARE LOGIC AND ANGELIC PARTIAL CORRECTNESS).

$$P \subseteq \text{dwp}(C, Q) \iff \models \langle [P] \rangle C \langle \blacksquare Q \rangle \quad \text{and} \quad P \subseteq \text{awlp}(C, Q) \iff \langle [P] \rangle C \langle \blacklozenge Q \rangle$$

Just as the logics in Figure 5 form a commuting square, so do the modalities:

$$\begin{array}{ccc} P \subseteq \text{awp}(C, Q) & \longrightarrow & P \subseteq \text{awlp}(C, Q) \\ \uparrow & & \uparrow \\ P \subseteq \text{dwp}(C, Q) & \longrightarrow & P \subseteq \text{dwl}(C, Q) \end{array} \quad \begin{array}{cc} \blacklozenge Q \Rightarrow \blacklozenge Q & \\ \uparrow & \uparrow \\ \blacksquare Q \Rightarrow \blacksquare Q & \end{array}$$

Lisbon Logic
Angelic Partial Correctness
Partial Hoare Logic

The back face of the upper cube is obtained by taking contrapositives of the four aforementioned logics, *i.e.*, by negating the pre- and postconditions. Because of the de Morgan style dualities between the modalities and predicate transformers (*e.g.*, $\square P$ iff $\neg \blacklozenge \neg P$ and $\blacksquare P$ iff $\neg \blacklozenge \neg P$), these logics can also be expressed by flipping the $\subseteq$ to be a $\supseteq$.

These contrapositive logics have not been studied, with the exception of $P \supseteq \text{awp}(C, Q)$, which was named *Necessary Conditions* (NC) by Cousot et al. [2013]. Indeed, in an NC triple $\{P\} C \{Q\}$, P is a necessary condition to reach Q , which means that all states outside of P must not reach Q . It was observed by Zhang and Kaminski [2022] and Ascari et al. [2023] that NC can be defined in terms of the angelic weakest precondition.

4.2 Strongest Postcondition Logics: The Lower Cube

The lower cube in Figure 5 is framed in terms of strongest postconditions [Dijkstra and Schönten 1990], which give the set of states reachable from the precondition. Verscht and Kaminski refer to the classical strongest postcondition as the *angelic* strongest postcondition, as the states must only be reachable from *some* start state, not *all* start states. There is a well known Galois connection between $\text{dwl}(C, P)$ and $\text{asp}(C, P)$, meaning that partial Hoare Logic can be framed in terms of both:

$$P \subseteq \text{dwl}(C, Q) \iff \text{asp}(C, P) \subseteq Q \quad \text{asp}(C, P) \triangleq \{\tau \mid \exists \sigma \in P. \tau \in \text{supp}(\llbracket C \rrbracket(\sigma))\}$$

Cousot and Zhang and Kaminski [2022] observed another Galois connection between awp and the demonic strongest *liberal* postcondition (dslp), meaning that NC also has two characterizations. Here, liberality corresponds to unreachability of the postcondition rather than nontermination from the precondition, so $\text{dslp}(C, P)$ contains states that are unreachable from outside of P .

$$P \supseteq \text{awp}(C, Q) \iff \text{dslp}(C, P) \supseteq Q \quad \text{dslp}(C, P) \triangleq \{\tau \mid \forall \sigma \notin P. \tau \notin \text{supp}(\llbracket C \rrbracket(\sigma))\}$$

For completeness, we include the remaining two transformers, although they do not have obvious intuitions or use cases, and as Verscht and Kaminski [2025, §3.2.4] point out, they cannot be defined inductively. As such, we do not define them directly, but rather represent them equivalently as combinations of other transformers that we have seen.

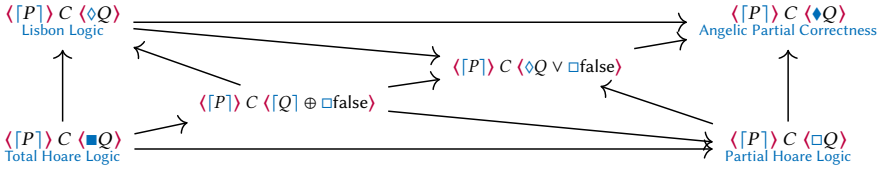
$$\text{aslp}(C, P) = \text{asp}(C, P) \cup \text{dslp}(C, P) \quad \text{dsp}(C, P) = \text{asp}(C, P) \cap \text{dslp}(C, P)$$

We can now describe the logics in the lower cube of Figure 5. Due to the aforementioned Galois connections, the top left edge overlaps with the upper cube, and describes partial Hoare Logic and NC. The only other logic in the cube that has been studied is Incorrectness Logic (IL) (originally introduced under the name Reverse Hoare Logic [de Vries and Koutavas 2011]), in the back right, which is obtained by swapping the $\subseteq$ for a $\supseteq$ in the asp definition of Hoare Logic, *i.e.*, an IL postcondition under-approximates the reachable states whereas a Hoare Logic postcondition over-approximates [O'Hearn 2020]. As was shown by Dardinier and Müller [2024, Proposition 6], IL can be encoded using under-approximate assertions $\lfloor P \rfloor$, *i.e.*, $m \models \lfloor P \rfloor$ iff $\text{supp}(m) \supseteq P$.

The remaining logics in the lower cube are not particularly well-understood. Indeed, [Verscht and Kaminski \[2025\]](#) showed that these logics have no inductive calculi, and accordingly we encode them in TOL using two triples. For example, Demonic Incorrectness states both that P is a necessary condition to reach Q ($\langle \lceil \neg P \rceil \rangle C \langle \sqcap \neg Q \rangle$), and also that every state in Q is reachable from P ($\langle \lfloor P \rfloor \rangle C \langle \lfloor Q \rfloor \rangle$), i.e., it is similar to standard incorrectness logic, but with the additional stipulation that P covers *all* the states that could reach the bug. No concrete applications have been found for these logics; as of now, they exist only to neatly complete the cube.

4.3 In-Between Logics

Although Figure 5 appears comprehensive, more logics can be formed by taking unions and intersections of the aforementioned predicate transformers. Focusing on the front face of the upper cube, [Verscht and Kaminski \[2025\]](#) form new logics by taking the conjunction and disjunction of Lisbon and Partial Hoare Logic. These *in-between* logics can be encoded in TOL as follows:



The first new logic offers triples of the form $\langle \lceil P \rceil \rangle C \langle \lceil Q \rceil \oplus \sqcap \text{false} \rangle$ — Q is reachable and is the only terminating outcome, but nontermination is possible too. This is the precise meaning of $\langle \lceil P \rceil \rangle C \langle \lceil Q \rceil \rangle$ in standard Outcome Logic [\[Zilberstein 2024a\]](#), as its semantic model could not identify the existence of nonterminating traces. This logic has a clear application in that it unifies partial correctness and incorrectness reasoning within a single logic [\[Zilberstein et al. 2023\]](#).

The second new logic has the form $\langle \lceil P \rceil \rangle C \langle \lceil Q \rceil \vee \sqcap \text{false} \rangle$, meaning that either Q is reachable or C never terminates. The applications for this second logic are less clear, being weaker than both Hoare Logic and Lisbon Logic. Still, expressing it in TOL at least gives us compositional rules to derive specifications, whereas [Verscht and Kaminski](#) must derive it using two separate calculi. More broadly, the existence of these in-between logics show that Figure 5 does not tell the complete story; *how many other logics are in-between?* In Section 6, we explore this question further and see how TOL is more expressive than any one of the predicate transformers that we have seen thus far.

5 The Cousot Cube

A second effort to build taxonomies of program logics was made by [Cousot \[2024\]](#). In this framework, 12 logics are arranged along a cube, as shown in Figure 6. The goals of constructing this cube were orthogonal to the taxonomy of [Verscht and Kaminski \[2025\]](#). Whereas the latter demonstrates weakening and contrapositive relationships, Cousot’s cube is intended to demonstrate how different logics can be constructed by composing abstractions, and how proof systems can be calculationaly derived using fixpoint induction. Nevertheless, it is interesting to compare the two taxonomies and see how Cousot’s cube can be expressed in Total Outcome Logic.

The corners of the cube in Figure 6 contain predicate transformers, similar to those that we saw in Section 4. We write, e.g., $P \overset{?}{\leftrightarrow} \text{pre}(C, Q)$, to indicate that P is either a subset or superset of $\text{pre}(C, Q)$. Two nodes protrude from each corner, denoting the $\subseteq$ and $\supseteq$ case. This makes for 16 logics, although 4 pairs of them collapse due to Galois connections. The top face of the cube is comprised of partial correctness logics, where termination and nontermination cannot be precisely quantified. Cousot’s predicate transformers are a subset of those of [Verscht and Kaminski \[2025\]](#),

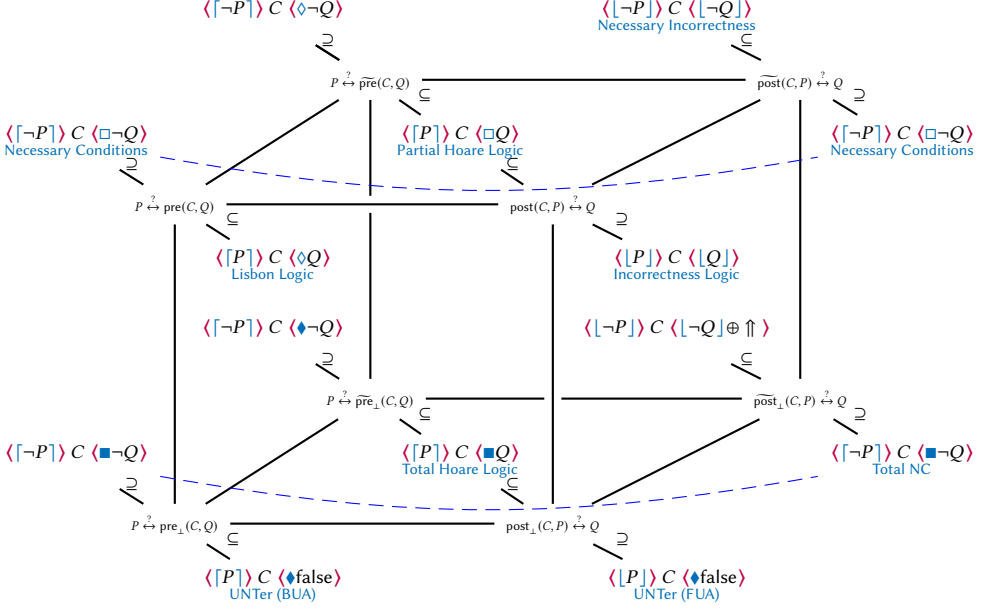


Fig. 6. The Cousot Cube, with logics expressed as Total Outcome Logic triples. The blue dashed lines represent Galois Connections.

and accordingly, all of the logics on the top face also appear in Figure 5.

$$\begin{aligned}
 \text{post}(C, P) &= \{\tau \mid \exists \sigma \in P. \tau \in \text{supp}(\llbracket C \rrbracket(\sigma))\} &= \text{asp}(C, P) \\
 \widetilde{\text{post}}(C, P) &= \{\tau \mid \forall \sigma \notin P. \tau \notin \text{supp}(\llbracket C \rrbracket(\sigma))\} &= \text{dslp}(C, P) \\
 \text{pre}(C, Q) &= \{\sigma \mid \exists \tau \in \text{supp}(\llbracket C \rrbracket(\sigma)). \tau \in Q\} &= \text{awp}(C, Q) \\
 \widetilde{\text{pre}}(C, Q) &= \{\sigma \mid \forall \tau \in \text{supp}(\llbracket C \rrbracket(\sigma)). \tau \in Q_\cup\} &= \text{dwlpl}(C, Q)
 \end{aligned}$$

The bottom face of the cube is where the two taxonomies diverge. The $\perp$ subscripted variants of the transformers are defined similarly to above, except that postconditions can range over $\Sigma_\cup$, meaning that the postcondition can express properties about (non)termination. As such, the Total Outcome Logic triples on the bottom face are only examples of what can be expressed in these logics, and each logic on the bottom face is strictly more expressive than the corresponding logic on top. For example, Total Hoare Logic appears in the middle of the square, but we could also express partial correctness there as $P \subseteq \widetilde{\text{pre}}_\perp(C, Q_\cup)$ instead of $P \subseteq \widetilde{\text{pre}}_\perp(C, Q)$.

Similarly, at the front of the bottom face we have the *backward* and *forward* under-approximate (BUA and FUA) variants of the nontermination logic UNTer [Raad et al. 2024], stating that every state in P has a nonterminating trace and that there exists a nonterminating trace starting in P , respectively. However, those nodes also subsume Lisbon Logic and Incorrectness Logic, respectively. By contrast, the FUA variant of UNTer does not appear in Figure 5, since there is no strongest postcondition transformer that includes nontermination information.

Cousot includes one more logic that does not fit into the hierarchy—*Hoare Incorrectness Logic*—which is the negation of a Hoare Triple $\neg\langle[P]\rangle C \langle\Box Q\rangle$. In TOL, we can express Hoare Incorrectness Logic as $\langle[P]\rangle C \langle\Diamond\neg Q\rangle$, which is also equivalent to $\exists \sigma \in P. \exists \tau \in \text{supp}(\llbracket C \rrbracket(\sigma)). \tau \notin Q$. This logic identifies a single trace—i.e., a counterexample—invalidating the correctness specification.

6 A New Taxonomy for Correctness and Incorrectness

We have shown that TOL has the expressive power to subsume all the logics in both the [Verscht and Kaminski \[2025\]](#) taxonomy of Hoare-like logics and the [Cousot \[2024\]](#) cube. However, although the prior two sections have explored more than 16 different program logics, only a few of them have found practical use. The front face of the upper cube in Figure 5—containing (partial and total) Hoare Logic and Lisbon Logic—is well understood, and serves as the foundation of decades of program analysis research. Incorrectness Logic has served as a semantic justification for real-world static analysis systems [[Le et al. 2022](#); [Raad et al. 2020](#)], although it was more recently shown that Lisbon Logic also models those systems [[Raad et al. 2024](#); [Zilberstein et al. 2024a](#)] and has some advantages in its ability to identify the cause of a bug [[Ascari et al. 2023](#); [Zilberstein et al. 2023](#)].

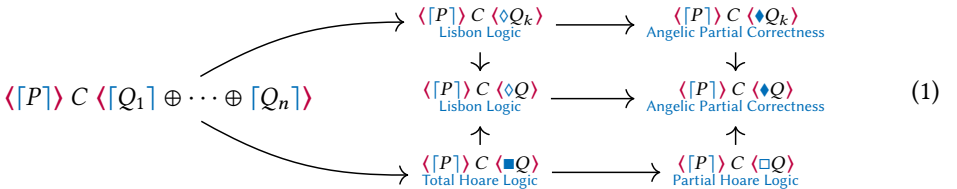
The remaining logics are not readily applicable to real static analysis problems. Their underlying concepts—demonic strongest postconditions (being reachable from *all* start states) and liberal postconditions (being unreachable from any start state)—neatly form Galois connections and complete the cubes, but defy intuitions and do not map to any of the properties about programs that have arisen in the past several decades of static analysis research. Even worse, some of these properties cannot be described using inductive rules [[Verscht and Kaminski 2025](#)].

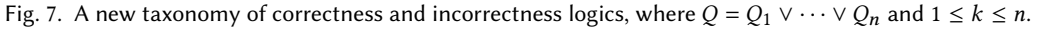
By contrast, Total Outcome Logic offers more expressiveness to reason about properties in that upper square. This expressivity stems from lifting reasoning to the level of sets of outcomes, where we can encode reachability and termination requirements as outcome assertions rather than building them into the interpretation of our triples. This lends TOL far greater versatility than simply instantiating the 16+ different logics, as we are no longer confined to the corners of the cube, but can navigate freely in the volume within.

Importantly, this enables shared program analyses in TOL for different properties. While Figure 5 suggests that Total Hoare Logic is the *strongest* logic in that upper front square, this gives only a limited, and perhaps misleading, picture as to the compatibility of the logics in that square.

Total Hoare triples give over-approximate correctness specifications, while Lisbon triples target the existence of a more select set of outcomes—potential bugs, for instance. More precisely, suppose that Q_{ok} represents some collection of good states and Q_{er} represents bugs states. It may be the case that the bug described by Q_{er} *sometimes* occurs and so the Lisbon triple $\langle [P] \rangle C \langle \Diamond Q_{er} \rangle$ is valid. But since the bug does not *always* occur, total Hoare Logic can only use a weaker postcondition $\langle [P] \rangle C \langle \blacksquare (Q_{ok} \vee Q_{er}) \rangle$. These two specifications are incompatible; there is no weakening relation between them. Similar issues arise between the other logics when postconditions differ. Some logics are wholly incompatible—*i.e.*, one type of triple cannot be used in a subderivation of the other whatsoever, such as between partial correctness $\langle [P] \rangle C \langle \Box Q \rangle$ and Lisbon Logic $\langle [P] \rangle C \langle \Diamond Q \rangle$ —rendering them isolated and disconnected for the purpose of analysis.

In TOL, we can instead perform the bulk of our program analysis on finer-grained intermediate specifications, where we track *reachable* outcomes of interest with $\oplus$ in the form of assertions: $[Q_1] \oplus \dots \oplus [Q_n]$. Then, letting $Q = Q_1 \vee \dots \vee Q_n$ and $1 \leq k \leq n$, we can weaken these to our desired property with an application of [CONSEQUENCE](#), as shown below.





By contrast, TOL offers more versatility to navigate between correctness and incorrectness specifications. With this in mind, we turn our attention to the portion of the taxonomy containing Partial and Total Hoare Logic, along with everything to the right. This is an expanded version of (1) above, where the unifying power of TOL shines. In the back there are three forms of TOL triples. Programs in the first row always terminate, programs in the second row sometimes terminate, and programs in the third row never terminate. A single TOL triple can subsume partial correctness, total correctness, Lisbon Logic, and nontermination, all of which have been successfully deployed in industrial static analysis tools. TOL unifies all these kinds of correctness and incorrectness, making

it an ideal logical foundation. In next sections, we will develop reasoning principles for TOL and illustrate their versatility through a few simple case studies.

7 Derived Rules

Throughout Sections 4 to 6, we have seen how TOL is able to express the specifications of many types of logics. However, it is also important that TOL provides convenient reasoning principles for those types of specifications. In this section, we derive new rules for common patterns arising from the logics that we have previously seen.

If Statements, While Loops, and Divergence. Recall from Section 2.1 that we gave syntactic sugar to define if statements and while loops in terms of the branching, iteration, and **assume** b constructs of our language. Whereas we previously only gave rules for standard branching and iteration, we now give rules for if and while. The if rule is the same as the one from standard Outcome Logic, and requires the precondition to be split into two parts to satisfy the guard and its negation. The two branches can then be analyzed compositionally.

$$\frac{\varphi_1 \models b \quad \langle \varphi_1 \rangle C_1 \langle \psi_1 \rangle \quad \varphi_2 \models \neg b \quad \langle \varphi_2 \rangle C_2 \langle \psi_2 \rangle}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \psi_1 \oplus \psi_2 \rangle} \text{IF}$$

The rule for while loops differs from standard Outcome Logic, as it includes nontermination information. We use three families of assertions: φ_n represents the outcomes where b remains true, ψ_n represents the outcomes where b is false, and ζ_n represents nontermination in nested loops. Compared to the ITER rule that we saw previously, the WHILE rule only has a single premise, as the entailments for φ_n and ψ_n remove the need to derive the behaviors of the ASSUME commands.

$$\frac{\forall n \in \mathbb{N}. \quad \varphi_n \models b \quad \psi_n \models \neg b \quad \zeta_n \models \text{div} \quad \langle \varphi_n \oplus \zeta_n \rangle C \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle}{\langle \varphi_0 \oplus \psi_0 \oplus \zeta_0 \rangle \text{ while } b \text{ do } C \langle \psi_\infty \oplus \zeta_\infty \rangle} \text{WHILE}$$

Standard Outcome Logic can only reason about terminating outcomes, and is thus only able to detect nontermination when no terminating executions are witnessed, *i.e.*, via triples of the form $\langle \varphi \rangle_{\text{OL}} C \langle \emptyset \odot \top \rangle_{\text{OL}}$. Whereas this restricts proofs to *possible termination* and *guaranteed nontermination*, TOL now allows us to demonstrate *sure-termination* or *possible nontermination*.

In terms of triples, the former is often bundled with a safety property as a total correctness specification. We can verify total correctness using *variants*, which track the number of iterations a loop makes until reaching a zero-variant – a point of sure-termination where the loop guard fails. In addition, a program cannot interfere with the existing weight on nontermination $\cup$. This means that any outcome assertion that only describes nontermination should be preserved across triples. It is useful to expand the DIV rule to reflect this.

$$\frac{\forall n < N. \quad \varphi_{n+1} \models b \quad \varphi_0 \models \neg b \quad \langle \varphi_{n+1} \rangle C \langle \varphi_n \rangle}{\langle \exists N : \mathbb{N}. \varphi_N \rangle \text{ while } b \text{ do } C \langle \varphi_0 \rangle} \text{VARIANT} \quad \frac{\zeta \models \text{div}}{\langle \zeta \rangle C \langle \zeta \rangle} \text{DIV}^*$$

Hoare Logic and Lisbon Logic. We will now derive the rules of Lisbon Logic and partial and total correctness Hoare Logic. We will present most of the rules as total correctness triples (of the form $\langle [P] \rangle C \langle \blacksquare Q \rangle$), since $\blacksquare Q \Rightarrow \Box Q$, and therefore a total correctness triple can be weakened to be a partial correctness one. We first give rules for sequencing total specifications, since the modalities used in pre- and postconditions do not exactly match.

$$\frac{\langle [P] \rangle C_1 \langle \blacksquare Q \rangle \quad \langle [Q] \rangle C_2 \langle \blacksquare R \rangle}{\langle [P] \rangle C_1 ; C_2 \langle \blacksquare R \rangle} \text{SEQ-TOTAL-HOARE} \quad \frac{\langle [P] \rangle C_1 \langle \Diamond Q \rangle \quad \langle [Q] \rangle C_2 \langle \Diamond R \rangle}{\langle [P] \rangle C_1 ; C_2 \langle \Diamond R \rangle} \text{SEQ-LISBON}$$

Similarly, we give a rule for if statements in Hoare and Lisbon Logic, which do not require entailments since the premises of the rules directly specify whether the guard is true or not.

$$\frac{\langle [P \wedge b] \rangle C_1 \langle \blacksquare Q \rangle \quad \langle [P \wedge \neg b] \rangle C_2 \langle \blacksquare Q \rangle}{\langle [P] \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \blacksquare Q \rangle} \text{IF-HOARE}$$

$$\frac{\langle [P \wedge b] \rangle C_1 \langle \Diamond Q \rangle \quad \langle [P \wedge \neg b] \rangle C_2 \langle \Diamond Q \rangle}{\langle [P] \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \Diamond Q \rangle} \text{IF-LISBON}$$

The inference rules for different variants of these logics differs the most when it comes to loops. It is well known that the invariant rule is complete for reasoning about loops in partial correctness Hoare Logic [Cook 1978]. We derive the invariant rule in our embedded logic.

$$\frac{\langle [P \wedge b] \rangle C \langle \Box P \rangle}{\langle [P] \rangle \text{ while } b \text{ do } C \langle \Box (P \wedge \neg b) \rangle} \text{INVARIANT}$$

We can also derive the standard variant rule employed in total Hoare Logic. The rule uses an integer-valued *ranking* expression R , which represents how close the program is to termination. In the premise, we are given that each iteration of the loop strictly decreases R , and the loop guard fails once R reaches 0. This allows us to conclude that the loop surely terminates:

$$\frac{P \wedge b \implies R > 0 \quad \forall n \in \mathbb{N}. \langle [P \wedge b \wedge R = n] \rangle C \langle \blacksquare (P \wedge R < n) \rangle}{\langle [P \wedge R \leq N] \rangle \text{ while } b \text{ do } C \langle \blacksquare (P \wedge \neg b) \rangle} \text{HOARE-VARIANT}$$

Nontermination. Bug-finding is often a more practical goal than correctness verification [O'Hearn 2020]. We can thus shift our sight from proving total correctness to proving the presence of nontermination by establishing a *quasi-invariant* [Larraz et al. 2014]. This is a property preserved by the loop body which ensures that the loop guard holds; the quasi-invariant forces indefinite reiteration, and any computation that satisfies the quasi-invariant once is trapped within the loop. Taken semantically, this aligns closely with the notion of a *recurrent set* [Gupta et al. 2008] of program states, visited infinitely often by the loop. In a nondeterministic setting, we can define this either angelically or demonically to show that the program *sometimes* or *always* diverges, respectively. Using the modalities from Section 4, we get:

$$\Diamond \text{false} = \exists u : U \setminus \{\emptyset\}. \uparrow^{(u)} \oplus \top \quad \Box \text{false} = \exists u : U. \uparrow^{(u)}$$

Then, the following two rules can be derived to prove nontermination.

$$\frac{[P] \models b \quad \langle [P] \rangle C \langle \Diamond P \rangle}{\langle [P] \rangle \text{ while } b \text{ do } C \langle \Diamond \text{false} \rangle} \text{QINV-ANGEL} \quad \frac{[P] \models b \quad \langle [P] \rangle C \langle \Box P \rangle}{\langle [P] \rangle \text{ while } b \text{ do } C \langle \Box \text{false} \rangle} \text{QINV-DEMON}$$

8 Case Studies

We want to emphasize that the power afforded by Total Outcome Logic lies in its subsumption of different types of reasoning, especially pertaining to termination and nontermination. To demonstrate the breadth of program properties that can be proven in TOL, we deploy it on a few examples. In these examples, we use variable assignments $x := E$ as basic actions. These actions are defined in the usual way, as in [Zilberstein 2024a].

8.1 Total Correctness: Quicksort Partition

In the classical formulation of Hoare logic for total correctness, [Manna and Pnueli 1974] derive the correctness of the Quicksort partition algorithm, where A is an integer array of size $n + 1$ and p

is the pre-computed pivot value:

$$\text{PARTITION} \triangleq \left\{ \begin{array}{l} i := 0 \ ; \ j := n \\ \text{while } i \leq j \text{ do} \\ \quad \text{if } A[i] \leq p \text{ then } i := i + 1 \\ \quad \text{else if } A[j] \geq p \text{ then } j := j - 1 \\ \quad \text{else swap}(A, i, j) \ ; \ i := i + 1 \ ; \ j := j - 1 \end{array} \right.$$

Total correctness of PARTITION can similarly be established in TOL. Again, we can encode a total correctness triple $[P] C [Q]$ as outcome triple $\langle \lceil P \rceil \rangle C \langle \blacksquare Q \rangle$. As the same inference rules available in total Hoare logic can be derived in TOL, our desired proof follows essentially the same steps. First, for a given pivot value p , define predicates α and β over integers:

$$\alpha(i) \triangleq \bigwedge_{0 \leq k < i} A[k] \leq p \quad \beta(j) \triangleq \bigwedge_{j < k \leq n} A[k] \geq p$$

We want to derive $\langle \lceil \text{true} \rceil \rangle \text{PARTITION} \langle \blacksquare (\alpha(i) \wedge \beta(j) \wedge i > j) \rangle$, which states that any run of the program must terminate having divided the array into two sections: elements $\leq p$ and those $\geq p$. We decorate PARTITION in Figure 10 in Appendix G.2 to sketch the full proof.

The last step of the proof uses HOARE-VARIANT. Our *invariant* $\alpha(i) \wedge \beta(j)$ enforces two boundaries in the array; every element behind index i must be $\leq p$, while every element after j must be $\geq p$. A natural choice for the *variant* is then the distance between the boundaries $j - i$, which we show must approach one another after each iteration. That is, we need to derive the following across the loop body C :

$$\langle \lceil \alpha(i) \wedge \beta(i) \wedge i \leq j \wedge j - i = m \rceil \rangle C \langle \blacksquare (\alpha(i) \wedge \beta(i) \wedge j - i < m) \rangle$$

Clearly, the loop condition $i \leq j$ implies $j - i > 0$. This provides us with all the premises needed to conclude $\langle \lceil \alpha(i) \wedge \beta(j) \rceil \rangle \text{PARTITION} \langle \blacksquare (\alpha(i) \wedge \beta(j) \wedge i > j) \rangle$.

8.2 Probabilistic Nontermination

Whereas much of the work on probabilistic verification focuses on *almost sure termination* [Kaminski 2019; McIver and Morgan 2005; McIver et al. 2018]—programs that terminate with probability 1—there are many programs simulating physical phenomena, which do not almost surely terminate, but are still important to reason about [Icard 2017]. An example of such programs is a scenario where a tortoise marches forward at a constant speed, and an erratic hare attempts to catch up.

The program below models such a scenario, where t represents the position of the tortoise, h represents the position of the hare, and k is the number of steps that have been taken. The tortoise starts one step ahead of the hare. Each step, the hare either takes a step forward, or leaps forward. However, the hare gets tired over time, so it can only leap a distance of $1 + \frac{1}{2^k}$. The program terminates if the hare catches up to the tortoise. Note that $C_1 +_p C_2$ is syntactic sugar for $(\text{assume } p \ ; \ C_1) + (\text{assume } 1 - p \ ; \ C_2)$.

$$C_{\text{body}} \triangleq \left\{ \begin{array}{l} t := 1 \ ; \ h := 0 \ ; \ k := 0 \\ \text{while } h < t \text{ do} \\ \quad t := t + 1 \ ; \\ \quad (h := h + 1) +_{\frac{1}{2}} (h := h + 1 + \frac{1}{2^k}) \ ; \\ \quad k := k + 1 \end{array} \right.$$

The hare catches the tortoise with probability $\frac{1}{2}$, when its initial move is a leap forward. If the hare's first move is not a leap, then it catches the tortoise with probability 0. We can see this informally, since in the best case, the hare catches up each round by $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$, which converges to 1, but

$$\begin{aligned}
& \langle \varphi_n \rangle \Leftrightarrow \langle \lceil t - h = \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{1}{2^n}\right)} \oplus \lceil t - h > \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^n}\right)} \rangle \\
& \quad t := t + 1 \text{;} \\
& \left\langle \lceil t - h = 1 + \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{1}{2^n}\right)} \oplus \lceil t - h > 1 + \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^n}\right)} \right\rangle \\
& \quad (h := h + 1) + \frac{1}{2} (h := h + 1 + \frac{1}{2^k}) \text{;} \\
& \left\langle \lceil t - h = \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \lceil t - h = \frac{1}{2^{n-1}} - \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \right. \\
& \quad \left. \lceil t - h > \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^{n+1}}\right)} \oplus \lceil t - h > \frac{1}{2^{n-1}} - \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^{n+1}}\right)} \right\rangle \\
& \Rightarrow \left\langle \lceil t - h > \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \lceil t - h = \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \right. \\
& \quad \left. \lceil t - h > \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^{n+1}}\right)} \oplus \lceil t - h > \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^{n+1}}\right)} \right\rangle \\
& \Rightarrow \langle \lceil t - h = \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \lceil t - h > \frac{1}{2^n} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^{n+1}}\right)} \rangle \\
& \quad k := k + 1 \\
& \langle \lceil t - h = \frac{1}{2^n} \wedge k = n + 1 \rceil^{\left(\frac{1}{2^{n+1}}\right)} \oplus \lceil t - h > \frac{1}{2^n} \wedge k = n + 1 \rceil^{\left(\frac{2^n-1}{2^{n+1}}\right)} \rangle \Leftrightarrow \langle \varphi_{n+1} \oplus \psi_{n+1} \rangle
\end{aligned}$$

Fig. 8. Derivation of the $n \geq 2$ case.

that occurs only if the hare *always* leaps forward, an event with probability $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdots = 0$. We will make this formal using the `RULE:WHILE` While rule. For this, we need to define the three families of assertions φ_n , ψ_n and ζ_n . We start with φ_n , which describes the outcomes where execution continues. If $n = 0$, then φ_0 simply describes the start state of the program. If $n = 1$, then φ_n describes the outcome where the hare's first move was a regular step forward. For $n \geq 2$, we only describe two outcomes. With probability $\frac{1}{2^n}$, the hare leaps forward on every step after the first one, and in that case, the difference in position is exactly $\frac{1}{2^n}$. If not, then the hare did not leap on at least one step, in which case the difference in position is strictly greater than $\frac{1}{2^n}$.

$$\begin{aligned}
\varphi_0 &\triangleq \lceil t = 1 \wedge h = 0 \wedge k = 0 \rceil & \varphi_1 &\triangleq \lceil t = 2 \wedge h = 1 \wedge k = 1 \rceil^{\left(\frac{1}{2}\right)} \\
\varphi_n &\triangleq \lceil t - h = \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{1}{2^n}\right)} \oplus \lceil t - h > \frac{1}{2^{n-1}} \wedge k = n \rceil^{\left(\frac{2^{n-1}-1}{2^n}\right)} & \text{if } n \geq 2
\end{aligned}$$

The ψ_n assertions describe the terminating outcomes. The program can terminate after the first step, so $\psi_1 \triangleq \lceil t = 2 \wedge h = 2 \wedge k = 1 \rceil^{\left(\frac{1}{2}\right)}$, but $\psi_n \triangleq \lceil \text{true} \rceil^{(0)}$ for every other $n \neq 1$. Finally, $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty \triangleq \lceil h = t \rceil^{\left(\frac{1}{2}\right)}$, the only terminating outcome. The ζ_n assertions describe nonterminating outcomes. The program only diverges in the limit, so we get $\zeta_n \triangleq \lceil \text{true} \rceil^{(0)}$ for all $n \in \mathbb{N}$ and $\zeta_\infty \triangleq \lceil \text{true} \rceil^{\left(\frac{1}{2}\right)}$. Clearly $(\varphi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty$, since according to φ_n , the probability that $t > h$ after n iterations is $\frac{2^{n-1}-1}{2^n}$, which converges to $\frac{1}{2}$. We now establish the premises of the **WHILE** rule, ignoring the ζ_n terms since they are vacuous. We show the cases where $n = 0$ and $n = 1$ below on the left and right, respectively.

$ \begin{aligned} & \langle \varphi_0 \rangle \Leftrightarrow \langle \lceil t = 1 \wedge h = 0 \wedge k = 0 \rceil \rangle \\ & \quad t := t + 1 \text{;} \\ & \langle \lceil t = 2 \wedge h = 0 \wedge k = 0 \rceil \rangle \\ & \quad (h := h + 1) + \frac{1}{2} (h := h + 1 + \frac{1}{2^k}) \text{;} \\ & \langle \bigoplus_{i \in \{1,2\}} \lceil t = 2 \wedge h = i \wedge k = 0 \rceil^{\left(\frac{1}{2}\right)} \rangle \\ & \quad k := k + 1 \\ & \langle \bigoplus_{i \in \{1,2\}} \lceil t = 2 \wedge h = i \wedge k = 1 \rceil^{\left(\frac{1}{2}\right)} \rangle \\ & \Leftrightarrow \langle \varphi_1 \oplus \psi_1 \rangle \end{aligned} $	$ \begin{aligned} & \langle \varphi_1 \rangle \Leftrightarrow \langle \lceil t = 2 \wedge h = 1 \wedge k = 1 \rceil^{\left(\frac{1}{2}\right)} \rangle \\ & \quad t := t + 1 \text{;} \\ & \langle \lceil t = 3 \wedge h = 1 \wedge k = 1 \rceil^{\left(\frac{1}{2}\right)} \rangle \\ & \quad (h := h + 1) + \frac{1}{2} (h := h + 1 + \frac{1}{2^k}) \text{;} \\ & \langle \lceil t = 3 \wedge h = 2 \wedge k = 1 \rceil^{\left(\frac{1}{4}\right)} \oplus \lceil t = 3 \wedge h = 2 + \frac{1}{2} \wedge k = 1 \rceil^{\left(\frac{1}{4}\right)} \rangle \\ & \quad k := k + 1 \\ & \langle \lceil t = 3 \wedge h = 2 \wedge k = 2 \rceil^{\left(\frac{1}{4}\right)} \oplus \lceil t = 3 \wedge h = 2 + \frac{1}{2} \wedge k = 2 \rceil^{\left(\frac{1}{4}\right)} \rangle \\ & \Rightarrow \langle \lceil t - h = \frac{1}{2} \wedge k = 2 \rceil^{\left(\frac{1}{4}\right)} \oplus \lceil t - h > \frac{1}{2} \wedge k = 2 \rceil^{\left(\frac{1}{4}\right)} \rangle \\ & \Leftrightarrow \langle \varphi_2 \oplus \psi_2 \rangle \end{aligned} $
--	--

Finally, we show the case where $n \geq 2$ in Figure 8. We complete the proof with a straightforward application of the **WHILE** rule.

$$\frac{\forall n \in \mathbb{N}. \langle \varphi_n \rangle C_{\text{body}} \langle \varphi_{n+1} \oplus \psi_{n+1} \rangle}{\langle [t = 1 \wedge h = 0 \wedge k = 0] \rangle \text{ while } h < t \text{ do } C_{\text{body}} \langle [h = t]^{\left(\frac{1}{2}\right)} \oplus \uparrow^{\left(\frac{1}{2}\right)} \rangle} \text{WHILE}$$

This tells us that the program terminates in a state where $h = t$ with probability $\frac{1}{2}$ and it diverges (the hare never catches the tortoise) with probability $\frac{1}{2}$.

9 Related Work

Correctness, Incorrectness, and Taxonomies of Program Logics. Recent years have seen increased interests in program logics with abilities to reason about nondeterminism in varying ways. This was spurred by Incorrectness Logic [O’Hearn 2020], which advocated for using angelic nondeterminism to identify programs that *sometimes* have undesirable behavior. Subsequently, new logics such as Outcome Logic [Zilberstein et al. 2023, 2025, 2024a,b], Hyper Hoare Logic [Dardinier and Müller 2024], Exact Separation Logic [Maksimović et al. 2023], Local Completeness Logic [Bruni et al. 2021, 2023], and Quantitative Hyper Weakest Pre [Zhang et al. 2024] emerged with the ability to reason about both angelic and demonic nondeterminism in a single framework.

However, none of the above logics handle nontermination; while they can be used to prove that programs *sometimes* terminate or *always* diverge, they cannot be used to prove that programs *always* terminate, or *sometimes* diverge. The former is the well-known total correctness property [Manna and Pnueli 1974]. The latter is useful in program analysis, as many codebases have nontermination bugs that can be found with new static analysis tools [Raad et al. 2024]. To our knowledge, Total Outcome Logic is the only logic that subsumes all of the aforementioned abilities.

This proliferation of logics sparked efforts to *taxonomy* them in terms of weakening relations, Galois connections, and contrapositives. For instance, Zhang and Kaminski [2022] began to organize logics by their characterizations in terms of predicate transformers. Through this lens, they discovered new logics such as *partial incorrectness logic*. The taxonomy of Zhang and Kaminski [2022] was extended by Verscht and Kaminski [2025], who characterize 16+ Hoare-like logics spanning three dimensions of program analysis: (1) correctness vs. incorrectness, (2) totality vs. partiality, and (3) angelic vs. demonic nondeterminism. Cousot [2024] described a framework for generating program logics calculational by composing abstractions, resulting in a similar taxonomy of 12 logics, arranged in a cube, where (non)termination reasoning forms a major axis.

While both taxonomies support partial and total correctness as well as incorrectness, their focus is to show how these emerge separately as specialized capabilities of disjoint logics. TOL subsumes both taxonomies, uniting these different forms of reasoning under a single metatheoretic foundation. Moreover, intermediate specifications in TOL enable reusable proof fragments, as they can be weakened to prove the various triples featured in correctness and incorrectness logics.

Weighted Programming. Batz et al. [2022] proposed weighted programming as a paradigm in which each outcome of a program is weighted by an element of a semiring. The initial formulation defined a weakest precondition style calculus for deriving the weight of a single outcome. It used an operational semantics based on total semirings to encode the bounded models that we have in this paper, such as probabilistic programs. While the operational semantics could identify infinite traces, the semantics was not derived from a fixed point.

Outcome Logic extended the model of weighted programming to partial semirings, to encode more models [Zilberstein 2024a]. Outcome Logic also has the ability to reason about multiple executions in a single derivation, so that it can characterize the weights of many outcomes at once. Quantitative Weakest Hyper Pre [Zhang et al. 2024] is the predicate transformer analogue of

Outcome Logic, as weakest preconditions is to Hoare Logic. However, neither Outcome Logic nor Quantitative Weakest Hyper Pre can identify the weight of nonterminating traces.

Powerdomains and Unbounded Nondeterminism. Powerdomains [Plotkin 1976; Smyth 1978] provide a mechanism to lift a dcpo $\langle X, \leq \rangle$ to a dcpo on $\mathcal{P}(X)$ (the powerset of X). There are three standard powerdomains with different properties and tradeoffs, built from the Hoare and Smyth [1978] orders below, and the Egli-Milner order, which combines the two: $S \sqsubseteq_{EM} T$ iff $S \sqsubseteq_H T$ and $S \sqsubseteq_S T$.

$$S \sqsubseteq_H T \quad \text{iff} \quad \forall x \in S. \exists y \in T. x \leq y \qquad S \sqsubseteq_S T \quad \text{iff} \quad \forall y \in T. \exists x \in S. x \leq y$$

When applied to a flat poset $\langle \Sigma_{\cup}, \leq \rangle$ ², we obtain the dcpos below [Søndergaard and Sestoft 1992].

$$\begin{aligned} \text{Hoare} &\triangleq \langle \{S \cup \{\cup\} \mid S \subseteq \Sigma\}, \sqsubseteq_H \rangle \cong \langle \mathcal{P}(\Sigma), \subseteq \rangle \\ \text{Smyth} &\triangleq \langle \mathcal{P}_{\text{fin}}(\Sigma) \cup \{\{\cup\}\}, \sqsubseteq_S \rangle \\ \text{Plotkin} &\triangleq \langle \mathcal{P}_{\text{fin}}(\Sigma) \cup \{S \subseteq \Sigma \mid \cup \in S\}, \sqsubseteq_{EM} \rangle \end{aligned}$$

Only the Hoare powerdomain supports unbounded nondeterminism, however it cannot identify when nontermination may occur (except when there are no terminating outcomes at all), making it a good semantic basis for partial correctness. Since $\cup$ is always present, the Hoare powerdomain can equivalently be defined as the dcpo on $\mathcal{P}(\Sigma)$ ordered by subset inclusion. The semantic model of Outcome Logic [Zilberstein 2024a] can be seen as a generalization of the Hoare powerdomain.

The other two powerdomains can identify nontermination, but do not allow unbounded nondeterminism, stemming from the impossibility result of Apt and Plotkin [1986]. The Smyth [1978] powerdomain consists of all *finite* sets of terminating outcomes and $\{\cup\}$. If nontermination is possible at all, then the semantics is $\{\cup\}$, so we cannot distinguish a program that sometimes diverges from one that always does. This makes it a good semantic basis for total correctness.

Finally, the Plotkin [1976] Powerdomain—derived from the Egli-Milner order—contains all finite subsets of Σ and infinite subsets containing $\cup$. This is reminiscent of the TOL semantic model, which maximizes the weight of nontermination for infinite sets. Indeed, in the powerset interpretation, the maximum possible weight is 1, and $m(\cup) = 1$ means that $\cup$ is in the set of outcomes. As such, our dcpo $\langle \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma_{\cup}), \sqsubseteq \rangle$ can be viewed as a generalization of the Plotkin powerdomain.

10 Conclusion

Recent interest in new program logics for correctness and incorrectness has led to the development of unified reasoning frameworks. Despite being quite extensive, one dimension that was missing in existing frameworks was the ability to reason about both termination and nontermination, leaving the picture incomplete. In this paper, we proposed Total Outcome Logic as a logic to derive precise specifications about both terminating and nonterminating traces. Total Outcome Logic is generalized over a weighted programming model, meaning that reasoning about (non)termination orthogonally extends to programs with other computational effects, such as probabilistic programs.

Outcome Logic also provides an adequate theory to unify correctness and incorrectness analyses [Zilberstein et al. 2023] and Total Outcome Logic extends this theory further to more types of correctness and incorrectness properties. We showed that TOL subsumes and extends entire taxonomies of program logics [Cousot 2024; Verscht and Kaminski 2025], rendering it a powerful metatheoretic foundation for shared program analysis.

As a next step, it would be interesting to build static analysis algorithms that take advantage of TOL’s expressive power. Inspired by techniques such as bi-abduction [Calcagno et al. 2009; Le et al. 2022; Raad et al. 2020; Zilberstein et al. 2024a], our new tool would use formulae of the form shown in Section 6 to increase the amount of specification sharing over prior work, while supporting

²The flat poset has $\cup \leq \sigma$ and $\sigma \leq \sigma$ for all $\sigma \in \Sigma_{\cup}$, but $\sigma \not\leq \tau$ for all $\sigma \neq \tau$.

partial correctness, total correctness, incorrectness, nontermination, and more all within a single tool. We believe that this extensible approach will help to scale the implementation and execution of static analysis for many different types of programs.

References

- Samson Abramsky and Achim Jung. 1995. *Domain theory*. Oxford University Press, Inc., USA, 1–168.
- K. R. Apt and G. D. Plotkin. 1986. Countable nondeterminism and random assignment. *J. ACM* 33, 4 (aug 1986), 724–767. <https://doi.org/10.1145/6490.6494>
- Flavio Ascari, Roberto Bruni, and Roberta Gori. 2022. Limits and difficulties in the design of under-approximation abstract domains. In *Foundations of Software Science and Computation Structures*. Springer International Publishing, Cham, 21–39. https://doi.org/10.1007/978-3-030-99253-8_2
- Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. 2023. Sufficient Incorrectness Logic: SIL and Separation SIL. arXiv:2310.18156 [cs.LO]
- Ralph-Johan Back. 1983. A Continuous Semantics for Unbounded Nondeterminism. *Theoretical Computer Science* 23 (12 1983), 187–210. [https://doi.org/10.1016/0304-3975\(83\)90055-5](https://doi.org/10.1016/0304-3975(83)90055-5)
- Kevin Batz, Adrian Gallus, Benjamin Lucien Kaminski, Joost-Pieter Katoen, and Tobias Winkler. 2022. Weighted Programming. arXiv:2202.07577 [cs.PL]
- Jon Beck. 1969. Distributive laws. In *Seminar on Triples and Categorical Homology Theory*, B. Eckmann (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 119–140. <https://doi.org/10.1007/BFb0083084>
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2021. A Logic for Locally Complete Abstract Interpretations. In *2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 1–13. <https://doi.org/10.1109/LICS52264.2021.9470608>
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2023. A Correctness and Incorrectness Program Logic. *J. ACM* 70, 2, Article 15 (mar 2023), 45 pages. <https://doi.org/10.1145/3582267>
- Cristiano Calcagno, Dino Distefano, Peter O’Hearn, and Hongseok Yang. 2009. Compositional Shape Analysis by Means of Bi-Abduction. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) (POPL ’09). Association for Computing Machinery, New York, NY, USA, 289–300. <https://doi.org/10.1145/1480881.1480917>
- Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (feb 1978), 70–90. <https://doi.org/10.1137/0207005>
- Patrick Cousot. 2002. Constructive design of a hierarchy of semantics of a transition system by abstract interpretation. *Theoretical Computer Science* 277, 1 (2002), 47–103. [https://doi.org/10.1016/S0304-3975\(00\)00313-3](https://doi.org/10.1016/S0304-3975(00)00313-3) Static Analysis.
- Patrick Cousot. 2024. Calculational Design of [In]Correctness Transformational Program Logics by Abstract Interpretation. *Proc. ACM Program. Lang.* 8, POPL, Article 7 (jan 2024), 34 pages. <https://doi.org/10.1145/3632849>
- Patrick Cousot, Radhia Cousot, Manuel Fähndrich, and Francesco Logozzo. 2013. Automatic Inference of Necessary Preconditions. In *Verification, Model Checking, and Abstract Interpretation*, Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 128–148.
- Thibault Dardinier and Peter Müller. 2024. Hyper Hoare Logic: (Dis-)Proving Program Hyperproperties. *Proc. ACM Program. Lang.* 8, PLDI, Article 207 (jun 2024), 25 pages. <https://doi.org/10.1145/3656437>
- Edsko de Vries and Vasileios Koutavas. 2011. Reverse Hoare Logic. In *Software Engineering and Formal Methods*. Springer Berlin Heidelberg, Berlin, Heidelberg, 155–171. https://doi.org/10.1007/978-3-642-24690-6_12
- Edsger W. Dijkstra. 1976. *A Discipline of Programming*. Prentice-Hall. I–XVII, 1–217 pages.
- Edsger W. Dijkstra and Carel S. Schölten. 1990. *The strongest postcondition*. Springer New York, New York, NY, 209–215. https://doi.org/10.1007/978-1-4612-3228-5_12
- Shenghua Feng, Mingshuai Chen, Han Su, Benjamin Lucien Kaminski, Joost-Pieter Katoen, and Naijun Zhan. 2023. Lower Bounds for Possibly Divergent Probabilistic Programs. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 99 (April 2023), 31 pages. <https://doi.org/10.1145/3586051>
- Robert W. Floyd. 1967. Assigning Meanings to Programs. In *Mathematical Aspects of Computer Science (Proceedings of Symposia in Applied Mathematics, Vol. 19)*. American Mathematical Society, Providence, Rhode Island, 19–32.
- Jonathan S. Golan. 1999. Semirings and their applications. <https://api.semanticscholar.org/CorpusID:122727231>
- Ashutosh Gupta, Thomas A. Henzinger, Rupak Majumdar, Andrey Rybalchenko, and Ru-Gang Xu. 2008. Proving non-termination. *SIGPLAN Not.* 43, 1 (Jan. 2008), 147–158. <https://doi.org/10.1145/1328897.1328459>
- David Harel, Dexter Kozen, and Jerzy Tiuryn. 2001. Dynamic logic. *SIGACT News* 32, 1 (2001), 66–69. <https://doi.org/10.1145/568438.568456>
- Charles Antony Richard Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (Oct. 1969), 576–580. <https://doi.org/10.1145/363235.363259>

- C. A. R. Hoare. 1978. Some Properties of Predicate Transformers. *J. ACM* 25, 3 (Jul 1978), 461–480. <https://doi.org/10.1145/322077.322088>
- Thomas Icard. 2017. Beyond Almost-Sure Termination. In *Proceedings of the 39th Annual Meeting of the Cognitive Science Society, CogSci 2017, London, UK, 16-29 July 2017*, Glenn Gunzelmann, Andrew Howes, Thora Tenbrink, and Eddy J. Davelaar (Eds.). cognitivesciencesociety.org. <https://mindmodeling.org/cogsci2017/papers/0430/index.html>
- Benjamin Lucien Kaminski. 2019. *Advanced weakest precondition calculi for probabilistic programs*. Dissertation. RWTH Aachen University, Aachen. <https://doi.org/10.18154/RWTH-2019-01829>
- Werner Kuich. 2011. Algebraic systems and pushdown automata. In *Algebraic foundations in computer science. Essays dedicated to Symeon Bozapalidis on the occasion of his retirement*. Berlin: Springer, 228–256. https://doi.org/10.1007/978-3-642-24897-9_11
- L. Lamport. 1977. Proving the Correctness of Multiprocess Programs. *IEEE Transactions on Software Engineering* SE-3, 2 (1977), 125–143. <https://doi.org/10.1109/TSE.1977.229904>
- Daniel Larraz, Kaustubh Nimkar, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. 2014. Proving Non-termination Using Max-SMT. In *Computer Aided Verification*, Armin Biere and Roderick Bloem (Eds.). Springer International Publishing, Cham.
- Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. 2022. Finding Real Bugs in Big Programs with Incorrectness Logic. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 81 (Apr 2022), 27 pages. <https://doi.org/10.1145/3527325>
- Andreas Löw, Daniele Nantes-Sobrinho, Sacha-Élie Ayoun, Caroline Cronjäger, Petar Maksimović, and Philippa Gardner. 2024. Compositional Symbolic Execution for Correctness and Incorrectness Reasoning. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 25:1–25:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.25>
- Christoph Lüth and Neil Ghani. 2002. Composing Monads Using Coproducts. In *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (Pittsburgh, PA, USA) (ICFP ’02)*. Association for Computing Machinery, New York, NY, USA, 133–144. <https://doi.org/10.1145/581478.581492>
- Rupak Majumdar and V.R. Sathiyarayanan. 2025. Sound and Complete Proof Rules for Probabilistic Termination. *Proc. ACM Program. Lang.* 9, POPL, Article 63 (Jan. 2025), 32 pages. <https://doi.org/10.1145/3704899>
- Petar Maksimović, Caroline Cronjäger, Andreas Löw, Julian Sutherland, and Philippa Gardner. 2023. Exact Separation Logic: Towards Bridging the Gap Between Verification and Bug-Finding. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 19:1–19:27. <https://doi.org/10.4230/LIPIcs.ECOOP.2023.19>
- Ernest G. Manes. 1976. *Algebraic Theories*. Springer New York. <https://doi.org/10.1007/978-1-4612-9860-1>
- Zohar Manna and Amir Pnueli. 1974. Axiomatic Approach to Total Correctness of Programs. *Acta Inf.* 3, 3 (sep 1974), 243–263. <https://doi.org/10.1007/BF00288637>
- Annabelle McIver and Carroll Morgan. 2005. *Abstraction, Refinement and Proof for Probabilistic Systems*. Springer. <https://doi.org/10.1007/b138392>
- Annabelle McIver, Carroll Morgan, Benjamin Lucien Kaminski, and Joost-Pieter Katoen. 2018. A New Proof Rule for Almost-Sure Termination. *Proc. ACM Program. Lang.* 2, POPL, Article 33 (Jan 2018), 28 pages. <https://doi.org/10.1145/3158121>
- Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)
- Bernhard Möller, Peter O’Hearn, and Tony Hoare. 2021. On Algebra of Program Correctness and & Incorrectness. In *Relational and Algebraic Methods in Computer Science: 19th International Conference, RAMiCS 2021, Marseille, France, November 2–5, 2021, Proceedings* (Marseille, France). Springer-Verlag, Berlin, Heidelberg, 325–343. https://doi.org/10.1007/978-3-030-88701-8_20
- Peter W. O’Hearn. 2020. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (Jan. 2020), 32 pages. <https://doi.org/10.1145/3371078>
- Gordon Plotkin. 1976. A Powerdomain Construction. *SIAM J. Comput.* 5, 3 (1976), 452–487. <https://doi.org/10.1137/0205035> arXiv:<https://doi.org/10.1137/0205035>
- Vaughan R. Pratt. 1976. Semantical Considerations on Floyd-Hoare Logic. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*. 109–121. <https://doi.org/10.1109/SFCS.1976.27>
- Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter O’Hearn, and Jules Villard. 2020. Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic. In *Computer Aided Verification*. Springer International Publishing, Cham, 225–252. https://doi.org/10.1007/978-3-030-53291-8_14
- Azalea Raad, Julien Vanegue, and Peter O’Hearn. 2024. Non-termination Proving at Scale. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 280 (Oct. 2024), 29 pages. <https://doi.org/10.1145/3689720>

- Michael Smyth. 1978. Power domains. *J. Comput. System Sci.* 16, 1 (1978), 23–36. [https://doi.org/10.1016/0022-0000\(78\)90048-X](https://doi.org/10.1016/0022-0000(78)90048-X)
- Harald Søndergaard and Peter Sestoft. 1992. Non-determinism in Functional Languages. *Comput. J.* 35, 5 (10 1992), 514–523. <https://doi.org/10.1093/comjnl/35.5.514> arXiv:<https://academic.oup.com/comjnl/article-pdf/35/5/514/1125580/35-5-514.pdf>
- Lena Verscht and Benjamin Lucien Kaminski. 2025. A Taxonomy of Hoare-Like Logics: Towards a Holistic View using Predicate Transformers and Kleene Algebras with Top and Tests. *Proc. ACM Program. Lang.* 9, POPL, Article 60 (Jan. 2025), 30 pages. <https://doi.org/10.1145/3704896>
- Linpeng Zhang and Benjamin Lucien Kaminski. 2022. Quantitative strongest post: a calculus for reasoning about the flow of quantitative information. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 87 (apr 2022), 29 pages. <https://doi.org/10.1145/3527331>
- Linpeng Zhang, Noam Zilberstein, Benjamin Lucien Kaminski, and Alexandra Silva. 2024. Quantitative Weakest Hyper Pre: Unifying Correctness and Incorrectness Hyperproperties via Predicate Transformers. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 300 (oct 2024), 30 pages. <https://doi.org/10.1145/3689740>
- Noam Zilberstein. 2024a. A Relatively Complete Program Logic for Effectful Branching. arXiv:2401.04594 [cs.LO] <https://arxiv.org/abs/2401.04594>
- Noam Zilberstein. 2024b. Unified Analysis Techniques for Programs with Outcomes. In *Companion Proceedings of the 2024 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity* (Pasadena, CA, USA) (*SPLASH Companion '24*). Association for Computing Machinery, New York, NY, USA, 4–6. <https://doi.org/10.1145/3689491.3691814>
- Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2023. Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 93 (Apr 2023), 29 pages. <https://doi.org/10.1145/3586045>
- Noam Zilberstein, Dexter Kozen, Alexandra Silva, and Joseph Tassarotti. 2025. A Demonic Outcome Logic for Randomized Nondeterminism. *Proc. ACM Program. Lang.* 9, POPL, Article 19 (Jan 2025), 30 pages. <https://doi.org/10.1145/3704855>
- Noam Zilberstein, Angelina Saliling, and Alexandra Silva. 2024a. Outcome Separation Logic: Local Reasoning for Correctness and Incorrectness with Computational Effects. *Proc. ACM Program. Lang.* 8, OOPSLA1 (Apr 2024). <https://doi.org/10.1145/3649821>
- Noam Zilberstein, Alexandra Silva, and Joseph Tassarotti. 2024b. Probabilistic Concurrent Reasoning in Outcome Logic: Independence, Conditioning, and Invariants. arXiv:2411.11662 [cs.LO] <https://arxiv.org/abs/2411.11662>

Appendix

A Semantics of Tests

This definition for the semantics of tests $\llbracket b \rrbracket_{\text{Test}} : \Sigma \rightarrow \{0, 1\}$ was omitted from Section 2.2.

$$\begin{aligned}
 \llbracket \text{true} \rrbracket_{\text{Test}}(\sigma) &\triangleq 1 \\
 \llbracket \text{false} \rrbracket_{\text{Test}}(\sigma) &\triangleq 0 \\
 \llbracket b_1 \vee b_2 \rrbracket_{\text{Test}}(\sigma) &\triangleq \begin{cases} 1 & \text{if } \llbracket b_1 \rrbracket_{\text{Test}}(\sigma) = 1 \text{ or } \llbracket b_2 \rrbracket_{\text{Test}}(\sigma) = 1 \\ 0 & \text{otherwise} \end{cases} \\
 \llbracket b_1 \wedge b_2 \rrbracket_{\text{Test}}(\sigma) &\triangleq \begin{cases} 1 & \text{if } \llbracket b_1 \rrbracket_{\text{Test}}(\sigma) = \llbracket b_2 \rrbracket_{\text{Test}}(\sigma) = 1 \\ 0 & \text{otherwise} \end{cases} \\
 \llbracket \neg b \rrbracket_{\text{Test}}(\sigma) &\triangleq \begin{cases} 1 & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = 0 \\ 0 & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = 1 \end{cases} \\
 \llbracket t \rrbracket_{\text{Test}}(\sigma) &\triangleq \begin{cases} 1 & \text{if } \sigma \in t \\ 0 & \text{if } \sigma \notin t \end{cases}
 \end{aligned}$$

B Domain Properties

Here, we prove a few preliminary results on our semantic domain $\mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$. The first of these describes how the Kleisli extension $(-)^{\dagger} : (X \rightarrow \mathcal{W}_{\mathcal{A}}(Y)) \rightarrow \mathcal{W}_{\mathcal{A}}(X) \rightarrow \mathcal{W}_{\mathcal{A}}(Y)$ interacts with $+$, scalar multiplication $\cdot$, and divergence:

LEMMA B.1. *For any $f, g \in (X \rightarrow \mathcal{W}_{\mathcal{A}}(Y))$, $m, m' \in \mathcal{W}_{\mathcal{A}}(X)$, and $u \in U$:*

- (1) $f^{\dagger}(m + m') = f^{\dagger}(m) + f^{\dagger}(m')$
- (2) If $\text{supp}(m) \subseteq \Sigma$, $(f + g)^{\dagger}(m) = f^{\dagger}(m) + g^{\dagger}(m)$
- (3) $f^{\dagger}(u \cdot m) = u \cdot f^{\dagger}(m)$
- (4) If $\text{supp}(m) \subseteq \Sigma$, $f^{\dagger}(m \cdot u) = f^{\dagger}(m) \cdot u$
- (5) $f^{\dagger}(\eta(\cup)) = \eta(\cup)$

PROOF. (1), (3), and (4) are left to the reader. We show:

- (2) If $\text{supp}(m) \subseteq \Sigma$, then $m(\cup) = 0$. We have:

$$\begin{aligned}
 (f + g)^{\dagger}(m) &= \left(\sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot (f + g)(\sigma) \right) + m(\cup) \cdot \eta(\cup) \\
 &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot (f(\sigma) + g(\sigma)) \\
 &= \left(\sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma) \right) + \left(\sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot g(\sigma) \right) \\
 &= f^{\dagger}(m) + g^{\dagger}(m)
 \end{aligned}$$

- (5) Note that $\text{supp}(\eta(\cup)) = \{\cup\}$, so $\text{supp}(\eta(\cup)) \cap \Sigma = \emptyset$. Then:

$$\begin{aligned}
 f^{\dagger}(\eta(\cup)) &= \left(\sum_{\sigma \in \text{supp}(\eta(\cup)) \cap \Sigma} m(\sigma) \cdot f(\sigma) \right) + \eta(\cup)(\cup) \cdot \eta(\cup) \\
 &= \eta(\cup)(\cup) \cdot \eta(\cup)
 \end{aligned}$$

$$= \eta(\cup)$$

□

C Totality of Language Semantics

C.1 $\mathcal{D}$ -Closure

To accommodate both a continuous semantics and a (countably) infinite state set, it was necessary to restrict our domain to a subset $\mathcal{D}$ of weighting functions. In particular, we identified two possible classes of semantics with corresponding restrictions on the semiring of weights $\mathcal{A} = \langle X, +, \cdot, \emptyset, \top \rangle$ used:

- **Conservative:** $\mathcal{A}$ is partial and bounded. Then, we take $\mathcal{D} \triangleq \{m \in \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) \mid |m| = \top\}$.
- **Indicative:** $\mathcal{A}$ is bounded with strongly-infinite top element $\top$. For any other semiring, we can adjoin an element $w \notin X$ and extend $+$ and $\cdot$ such that w is a strongly-infinite upper bound:
 - $w + x = x + w = w$ for all $x \in X$;
 - $w \cdot x = x \cdot w = w$ for all $x \in X \setminus \{\emptyset\}$;
 - $w \cdot \emptyset = \emptyset \cdot w = \emptyset$.

In this case, take $\mathcal{D} \triangleq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)$.

In this section we demonstrate that the program semantics is closed in $\mathcal{D}$ for each of the classes described above.

C.1.1 Conservative .

LEMMA C.1. *For any collection $(x_i \in X)_{i \in I}$ such that $\sum_{i \in I} x_i = \top$, if $(m_i)_{i \in I}$ is a family of weighting functions with $|m_i| = \top$ for each $i \in I$, then $|\sum_{i \in I} x_i \cdot m_i| = \top$.*

PROOF. It holds that

$$\left| \sum_{i \in I} x_i \cdot m_i \right| = \sum_{i \in I} x_i \cdot |m_i| = \sum_{i \in I} x_i \cdot \top = \left(\sum_{i \in I} x_i \right) \cdot \top = \top \cdot \top = \top$$

□

LEMMA C.2. *For any $m \in \mathcal{D}$ and $f \in \Sigma \rightarrow \mathcal{D}$, $\text{bind}(m, f) \in \mathcal{D}$.*

PROOF. Recall that $\mathcal{D} \subseteq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) = \mathcal{W}_{\mathcal{A}}^+(\Sigma) \cup \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$. First, suppose that $m, f(\sigma) \in \mathcal{W}_{\mathcal{A}}^+(\Sigma)$ (i.e. m and $f(\sigma)$ have finite support) for all $\sigma \in \text{supp}(m) \cap \Sigma$. Recall:

$$\text{bind}(m, f) = m(\cup) \cdot \top \cdot \text{unit}(\cup) + \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma)$$

So,

$$\text{supp}(\text{bind}(m, f)) = (\text{supp}(m) \cap \{\cup\}) \cup \bigcup_{\sigma \in \text{supp}(m) \cap \Sigma} \text{supp}(f(\sigma))$$

Since $\text{supp}(m)$ is finite, and $\text{supp}(f(\sigma))$ is finite for each $\sigma \in \text{supp}(m) \cap \Sigma$, the above expression consists of a finite union of finite sets, which is itself finite. Thus, $\text{bind}(m, f) \in \mathcal{W}_{\mathcal{A}}^+(\Sigma) \subseteq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)$.

Next, we consider the case in which $m \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$ or $f(\sigma) \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$ for some $\tau \in \text{supp}(m) \cap \Sigma$. We split our analysis across the two weighting schemes defined (and, accordingly, the two definitions for $\mathcal{D}$ provided for each):

- **Conservative Weighting:** $\mathcal{D} = \{m \in \mathcal{W}_{\mathcal{A}}^{\infty} \mid |m| = \top\}$.

Since $m, f(\sigma) \in \mathcal{D}$ for every $\sigma \in \Sigma$, it holds that

- $(m(\sigma) \in X)_{\sigma \in \text{supp}(m)}$ is a family of weights such that $|\sum_{\sigma \in \text{supp}(m)} m(\sigma)| = |m| = \top$;
- $|f(\sigma)| = \top$ for each $\sigma \in \text{supp}(m) \cap \Sigma$;
- $|\top \cdot \text{unit}(\cup)| = \top$.

Thus, by lemma C.1, $|\text{bind}(m, f)| = \top$.

- **Indicative Weighting:** $\mathcal{D} = \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma)$.

If $m \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$, then $m(\cup) = \sup_{m' \in E(m)} m'(\cup) = \top$, indicating the presence of nontermination. As a strong infinity, $\top$ is absorbing and thus propagates through $\text{bind}(m, f)$:

$$\begin{aligned}
 \text{bind}(m, f)(\cup) &= m(\cup) \cdot \top \cdot \text{unit}(\cup)(\cup) + \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= \top \cdot \top \cdot \mathbf{1} + \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= \top + \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= \top = \sup_{m' \in E(\text{bind}(m, f))} m'(\cup)
 \end{aligned}$$

Similarly, if $f(\tau) \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$ for some $\tau \in \text{supp}(m) \cap \Sigma$:

$$\begin{aligned}
 \text{bind}(m, f)(\cup) &= m(\cup) \cdot \top \cdot \text{unit}(\cup)(\cup) + \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= (m(\cup) \cdot \top \cdot \mathbf{1}) + (m(\tau) \cdot f(\tau)(\cup)) + \sum_{\sigma \in \text{supp}(m) \cap \Sigma / \{\tau\}} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= (m(\cup) \cdot \top \cdot \mathbf{1}) + (m(\tau) \cdot \top) + \sum_{\sigma \in \text{supp}(m) \cap \Sigma / \{\tau\}} m(\sigma) \cdot f(\sigma)(\cup)
 \end{aligned}$$

Since $\tau \in \text{supp}(m)$, we know $m(\tau) \neq 0$, so $m(\tau) \cdot \top = \top$:

$$\begin{aligned}
 &= (m(\cup) \cdot \top \cdot \mathbf{1}) + \top + \sum_{\sigma \in \text{supp}(m) \cap \Sigma / \{\tau\}} m(\sigma) \cdot f(\sigma)(\cup) \\
 &= \top = \sup_{m' \in E(\text{bind}(m, f))} m'(\cup)
 \end{aligned}$$

Therefore, $\text{bind}(m, f) \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma) \subseteq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) = \mathcal{D}$.

□

C.2 Fixpoint Existence

THEOREM C.3 (SCOTT-CONTINUITY OF $\mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$). *Suppose $\mathcal{A}$ is a partial semiring which is bounded, finitary, Scott-continuous, and lower Scott-continuous. Then, $+ : \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})^2 \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$ on weighting functions is Scott-continuous with respect to the **fusion order** $\sqsubseteq$. That is, for any directed subset*

$$D \subseteq \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}),$$

$$\sup_{m_1 \in D} (m_1 + m_2) = \sup D + m_2$$

PROOF. Let $\sigma \in \Sigma$. Then,

$$\begin{aligned} \sup_{m_1 \in D} ((m_1 + m_2)(\sigma)) &= \sup_{m_1 \in D} (m_1(\sigma) + m_2(\sigma)) \\ &= \sup_{m_1 \in D} (m_1(\sigma)) + m_2(\sigma) && \text{(Scott-continuity of } \mathcal{A} \text{)} \\ &= \left(\sup_{m_1 \in D} m_1(\sigma) \right) + m_2(\sigma) \end{aligned}$$

Dually,

$$\begin{aligned} \inf_{m_1 \in D} ((m_1 + m_2)(\cup)) &= \inf_{m_1 \in D} (m_1(\cup) + m_2(\cup)) \\ &= \inf_{m_1 \in D} (m_1(\cup)) + m_2(\cup) && \text{(Lower Scott-continuity of } \mathcal{A} \text{)} \\ &= \left(\inf_{m_1 \in D} m_1(\cup) \right) + m_2(\cup) \end{aligned}$$

Now, since fusion order extends $\leq$ pointwise on Σ and $\geq$ on $\cup$, we have that $\sup_{m_1 \in D} (m_1 + m_2) = \sup D + m_2$. $\square$

LEMMA C.4 (JOINT CONTINUITY). *Suppose X is a dcpo. For any function $f : X \times X \rightarrow X$ that is Scott-continuous in the variables separately, f must be continuous in the variables jointly as well. I.e. for directed subsets $D_1, D_2 \subseteq X$,*

$$\sup_{x \in D_1, y \in D_2} f(x, y) = f(\sup D_1, \sup D_2)$$

PROOF. Since f is continuous separately in the first and second variables:

$$f(\sup D_1, \sup D_2) = \sup_{x \in D_1} f(x, \sup D_2) = \sup_{x \in D_1} \sup_{y \in D_2} f(x, y) = \sup_{x \in D_1, y \in D_2} f(x, y)$$

$\square$

LEMMA C.5. *Let $\langle X, +, \cdot, 0, 1 \rangle$ be a finitary (complete), continuous, partial semiring. For any family of Scott-continuous functions $(f_i : X \rightarrow X)_{i \in I}$ and directed set $D \subseteq X$:*

$$\sup_{x \in D} \sum_{i \in I} f_i(x) = \sum_{i \in I} f_i(\sup D)$$

PROOF. Since each f_i is Scott-continuous, we have $\{f_i(x) \mid x \in D\}$ is a directed set. We now proceed by induction on I .

- Base Case: $I = \{i_1\}$. Then, we only need to show $\sup_{x \in D} f_{i_1}(x) = f_{i_1}(\sup D)$, which follows from the Scott-continuity of f_{i_1} .
- Successor Case: Suppose the claim holds for all set smaller than I , for I finite. We can partition I into disjoint non-empty parts I_1 and I_2 , with $I = I_1 \cup I_2$, $I_1 \cap I_2 = \emptyset$, and $I_1, I_2 \neq \emptyset$. So,

$$\sup_{x \in D} \sum_{i \in I} f_i(x) = \sup_{x \in D} \left(\sum_{i \in I_1} f_i(x) + \sum_{i \in I_2} f_i(x) \right)$$

By the induction hypothesis, each of $\lambda x. \sum_{i \in I_1} f_i(x)$ and $\lambda x. \sum_{i \in I_2} f_i(x)$ are continuous, so $\{\sum_{i \in I_1} f_i(x) \mid x \in D\}$ and $\{\sum_{i \in I_2} f_i(x) \mid x \in D\}$ are directed. We apply joint continuity of $+: X^2 \rightarrow X$ to get:

$$\begin{aligned}
 &= \sup_{x \in D} \sum_{i \in I_1} f_i(x) + \sup_{x \in D} \sum_{i \in I_2} f_i(x) \\
 &= \sum_{i \in I_1} f_i(\sup D) + \sum_{i \in I_2} f_i(\sup D) \quad (\text{IH}) \\
 &= \sum_{i \in I} f_i(\sup D)
 \end{aligned}$$

- Limit Case: Suppose the claim holds for all finite indexing sets. Since the semiring is finitary:

$$\begin{aligned}
 \sup_{x \in D} \sum_{i \in I} f_i(x) &= \sup_{x \in D} \left(\sup_{J \subseteq I, J \text{ finite}} \sum_{i \in J} f_i(x) \right) \\
 &= \sup_{J \subseteq I, J \text{ finite}} \left(\sup_{x \in D} \sum_{i \in J} f_i(x) \right) \\
 &= \sup_{J \subseteq I, J \text{ finite}} \left(\sum_{i \in J} f_i(\sup D) \right) \\
 &= \sum_{i \in I} f_i(\sup D)
 \end{aligned}$$

□

COROLLARY C.6. For any **finite** family of Scott-continuous functions $(f_i : \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}) \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}))_{i \in I}$ and directed set $D \subseteq \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$:

$$\sup_{m \in D} \sum_{i \in I} f_i(m) = \sum_{i \in I} f_i(\sup D)$$

PROOF. See the proof of C.5 up to the successor case. □

LEMMA C.7. For any $\sigma \in \Sigma$, the function $g(f) = m(\sigma) \cdot f(\sigma)(\tau)$ is Scott-continuous in the semiring $\mathcal{A}$.

PROOF.

$$\sup_{f \in D} g(f) = \sup_{f \in D} m(\sigma) \cdot f(\sigma)(\tau)$$

By Scott-continuity of the $\cdot$ operator in the semiring:

$$= m(\sigma) \cdot \sup_{f \in D} f(\sigma)(\tau)$$

Since we are using the pointwise* ordering:

$$= m(\sigma) \cdot (\sup D)(\sigma)(\tau) = g(\sup D)$$

□

COROLLARY C.8. For $D \subseteq (\Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}))$ directed,

$$\sup_{f \in D} \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot (\sup D)(\sigma)(\tau) = \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot (\sup D)(\sigma)(\tau)$$

PROOF. By Lemma C.5. □

LEMMA C.9. If $m \in \mathcal{D}$ and $f \in (\Sigma \rightarrow \mathcal{D})$, $\text{bind}(\llbracket C \rrbracket(\sigma), f)$ is Scott-continuous with respect to its second argument f .

PROOF. Take $D \subseteq \Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma \cup)$ directed. Let $\sigma \in \Sigma$ and $\llbracket C \rrbracket(\sigma) \in \mathcal{D}$. Recall that $\mathcal{D} \subseteq \mathcal{W}_{\mathcal{A}}^{\infty}(\Sigma) = \mathcal{W}_{\mathcal{A}}^+(\Sigma) \cup \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$, so we can split our analysis across two cases:

- Suppose $\llbracket C \rrbracket(\sigma) \in \mathcal{W}_{\mathcal{A}}^+(\Sigma)$. Then, $\text{supp}(\llbracket C \rrbracket(\sigma))$ is finite, as is $\text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma$.

$$\begin{aligned} & \sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f) \\ &= \sup_{f \in D} \left(\llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup) + \sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot f(\tau) \right) \\ &= \llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup) + \sup_{f \in D} \left(\sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot f(\tau) \right) \end{aligned}$$

Since the above summation is finite (by assumption), we can apply (Corollary):

$$\begin{aligned} &= \llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup) + \sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot (\sup D)(\tau) \\ &= \text{bind}(\llbracket C \rrbracket(\sigma), \sup D) \end{aligned}$$

- Otherwise, $\llbracket C \rrbracket(\sigma) \in \mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$. We know that for any program state $\tau' \in \Sigma$,

$$\begin{aligned} & \sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f)(\tau') \\ &= \sup_{f \in D} \left(\llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup)(\tau') + \sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot f(\tau)(\tau') \right) \\ &= \llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup)(\tau') + \sup_{f \in D} \left(\sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot f(\tau)(\tau') \right) \\ &= \llbracket C \rrbracket(\sigma)(\cup) \cdot \top \cdot \text{unit}(\cup)(\tau') + \sum_{\tau \in \text{supp}(\llbracket C \rrbracket(\sigma) \cap \Sigma)} \llbracket C \rrbracket(\sigma)(\tau) \cdot (\sup D)(\tau)(\tau') \\ &= \text{bind}(\llbracket C \rrbracket(\sigma), \sup D)(\tau') \end{aligned}$$

In other words, $\sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f) \in E(\text{bind}(\llbracket C \rrbracket(\sigma), \sup D))$, so by the definition of $\mathcal{W}_{\mathcal{A}}^{\omega}(\Sigma)$,

$$\sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f)(\cup) \sqsubseteq \text{bind}(\llbracket C \rrbracket(\sigma), \sup D)(\cup)$$

Conversely, $\text{bind}(\llbracket C \rrbracket(\sigma), \sup D) \in E(\sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f))$ implies

$$\sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f)(\cup) \sqsupseteq \text{bind}(\llbracket C \rrbracket(\sigma), \sup D)(\cup)$$

Thus, $\sup_{f \in D} \text{bind}(\llbracket C \rrbracket(\sigma), f)(\cup) = \text{bind}(\llbracket C \rrbracket(\sigma), \sup D)(\cup)$. □

LEMMA C.10. Let $\Phi_{\langle C, e_1, e_2 \rangle}(f)(\sigma) = \llbracket e_1 \rrbracket(\sigma) \cdot \text{bind}_{\text{nt}}(\llbracket C \rrbracket(\sigma), f) + \llbracket e_2 \rrbracket(\sigma) \cdot \text{unit}(\sigma)$ and suppose that it is a total function, then $\Phi_{\langle C, e_1, e_2 \rangle}$ is Scott continuous with respect to the pointwise order: $f_1 \sqsubseteq f_2$ iff $f_1(\sigma) \sqsubseteq f_2(\sigma)$ for all $\sigma \in \Sigma$.

PROOF. For all directed sets $D \subseteq (\Sigma \rightarrow \mathcal{W}(\Sigma \cup \{\cup\}))$ and $\sigma \in \Sigma$, we have

$$\sup_{f \in D} \Phi_{\langle C, e_1, e_2 \rangle}(f)(\sigma) = \sup_{f \in D} (\llbracket e_1 \rrbracket(\sigma) \cdot \text{bind}_{\text{nt}}(\llbracket C \rrbracket(\sigma), f) + \llbracket e_2 \rrbracket(\sigma) \cdot \text{unit}(\sigma))$$

By the continuity of $+$ and $\cdot$ in $\mathcal{W}_{\mathcal{A}}$:

$$= \llbracket e_1 \rrbracket(\sigma) \cdot \left(\sup_{f \in D} \text{bind}_{\text{nt}}(\llbracket C \rrbracket(\sigma), f) \right) + \llbracket e_2 \rrbracket(\sigma) \cdot \text{unit}(\sigma)$$

By Lemma 6 (the previous Lemma):

$$\begin{aligned} &= \llbracket e_1 \rrbracket(\sigma) \cdot \text{bind}_{\text{nt}}(\llbracket C \rrbracket(\sigma), \sup D) + \llbracket e_2 \rrbracket(\sigma) \cdot \text{unit}(\sigma) \\ &= \Phi_{\langle C, e_1, e_2 \rangle}(\sup D)(\sigma) \end{aligned}$$

As this holds for all $\sigma \in \Sigma$, we also have that

$$\sup_{f \in D} \Phi_{\langle C, e_1, e_2 \rangle}(f) = \Phi_{\langle C, e_1, e_2 \rangle}(\sup D)$$

□

D Subsumption of Program Logics

Here, we prove the results of Section 4 on the subsumption of classical program logics. For this, recall that we limit ourselves to a nondeterministic interpretation of TOL, instantiated on the Boolean semiring. First, we show that the encodings of modalities $\Box$ and $\Diamond$ of Dynamic Logic in TOL are DeMorgan duals.

LEMMA D.1 (MODAL DUALITY).

$$\Diamond P = \neg \Box \neg P \quad \text{and} \quad \Box P = \neg \Diamond \neg P$$

PROOF.

$$\begin{aligned} \neg \Box \neg P &= \neg \{m \mid \text{supp}(m) \subseteq (\neg P)_{\cup}\} \\ &= \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}) \setminus \{m \mid \text{supp}(m) \subseteq (\neg P)_{\cup}\} \\ &= \{m \mid \text{supp}(m) \not\subseteq (\neg P)_{\cup}\} \\ &= \{m \mid \text{supp}(m) \cap P \neq \emptyset\} = \Diamond P \end{aligned}$$

$$\begin{aligned} \neg \Diamond \neg P &= \neg \{m \mid \text{supp}(m) \cap \neg P \neq \emptyset\} \\ &= \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}) \setminus \{m \mid \text{supp}(m) \cap \neg P \neq \emptyset\} \\ &= \{m \mid \text{supp}(m) \cap \neg P = \emptyset\} \\ &= \{m \mid \text{supp}(m) \subseteq P_{\cup}\} = \Box P \end{aligned}$$

□

Moreover, the alternative modalities $\blacksquare$ and $\blacklozenge$ exhibit the same duality. Proof of this is nearly identical to the above, and so is omitted.

Theorems 4.1 and 4.2 state that Hoare, Lisbon, Total Hoare, as well as angelic partial correctness triples can be encoded using the above modalities. These correspond to the logics in the front upper cube of Figure 5 which are formulated in terms of weakest-precondition transformers.

THEOREM 4.1 (SUBSUMPTION OF HOARE LOGIC). $\models \langle \lceil P \rceil \rangle C \langle \Box Q \rangle$ iff $P \subseteq \text{dwlp}(C, Q)$.

PROOF. $\implies$: Suppose $\sigma \in P$. Then, $\eta(\sigma) \models P$. We assume $\langle \lceil P \rceil \rangle C \langle \Box Q \rangle$, so

$$\llbracket C \rrbracket^\dagger(\eta(\sigma)) \models \Box Q = \exists (u, v) \in U^2. \lceil Q \rceil^{(u)} \oplus \uparrow^{(v)} = \{m \mid \text{supp}(m) \subseteq Q_\cup\}$$

That is, $\text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q_\cup$. By definition $\sigma \in \text{dwlp}(C, Q)$.

$\Leftarrow$: We assume $\models P \subseteq \text{dwlp}(C, Q)$. Suppose $m \models \lceil P \rceil$. So, $|m| = 1$ and $\text{supp}(m) \subseteq P \subseteq \text{dwlp}(C, Q)$. Note that $\cup \notin \text{supp}(m)$. Then, since $\text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q_\cup$ for all $\sigma \in \text{supp}(m)$, we have

$$\text{supp}(\llbracket C \rrbracket^\dagger(m)) = \bigcup_{\sigma \in \text{supp}(m)} \text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q_\cup$$

By definition of $\Box$, $\llbracket C \rrbracket^\dagger(m) \models \Box Q$. □

THEOREM 4.1 (SUBSUMPTION OF LISBON LOGIC). $\models \langle \lceil P \rceil \rangle C \langle \Diamond Q \rangle$ iff $P \subseteq \text{awp}(C, Q)$.

PROOF. $\implies$: Suppose $\sigma \in P$. Then, $\eta(\sigma) \models P$. Since $\models \langle \lceil P \rceil \rangle C \langle \Diamond Q \rangle$, we have that

$$\llbracket C \rrbracket^\dagger(\eta(\sigma)) \models \Diamond Q = \exists u : U \setminus \{\emptyset\}. \lceil Q \rceil^{(u)} \oplus \top = \{m \mid \text{supp}(m) \cap Q \neq \emptyset\}$$

This means $\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q \neq \emptyset$, so $\sigma \in \text{awp}(C, Q)$ by definition.

$\Leftarrow$: Assume $P \subseteq \langle Q \rangle$. Suppose $m \models \lceil P \rceil$, i.e. $|m| = 1$, $\text{supp}(m) \subseteq P \subseteq \text{awp}(C, Q)$. Note that $\cup \notin \text{supp}(m)$. So, for any $\sigma \in \text{supp}(m)$, $\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q \neq \emptyset$. It holds that

$$\text{supp}(\llbracket C \rrbracket^\dagger(m)) \cap Q = \left(\bigcup_{\sigma \in \text{supp}(m)} \text{supp}(\llbracket C \rrbracket(\sigma)) \right) \cap Q = \bigcup_{\sigma \in \text{supp}(m)} (\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q) \neq \emptyset$$

By definition, $\llbracket C \rrbracket^\dagger(m) \models \Diamond Q$. □

THEOREM 4.2 (SUBSUMPTION OF TOTAL HOARE LOGIC). $\models \langle \lceil P \rceil \rangle C \langle \blacksquare Q \rangle$ iff $P \subseteq \text{dwp}(C, Q)$.

PROOF. $\implies$: Suppose $\sigma \in P$. Then, $\eta(\sigma) \models \lceil P \rceil$ and since $\models \langle \lceil P \rceil \rangle C \langle \blacksquare Q \rangle$,

$$\llbracket C \rrbracket^\dagger(\eta(\sigma)) \models \blacksquare Q = \exists u : U \setminus \{\emptyset\}. \lceil Q \rceil^{(u)} = \{m \mid \text{supp}(m) \subseteq Q\}$$

That is, $\text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q$, so we have $\sigma \in \text{dwp}(C, Q)$.

$\Leftarrow$: Suppose $m \models \lceil P \rceil$, i.e. $|m| = 1$ and $\text{supp}(m) \subseteq P \subseteq \text{dwp}(C, Q)$. Note that $\cup \notin \text{supp}(m)$, and

$$\text{supp}(\llbracket C \rrbracket^\dagger(m)) = \bigcup_{\sigma \in \text{supp}(m)} \text{supp}(\llbracket C \rrbracket(\sigma))$$

For each $\sigma \in \text{supp}(m)$, $\text{supp}(\llbracket C \rrbracket(\sigma)) \subseteq Q$. This gives us that $\llbracket C \rrbracket^\dagger(m) \models \blacksquare Q$, as desired. □

THEOREM 4.2 (SUBSUMPTION OF ANGELIC PARTIAL CORRECTNESS).

$$\models \langle \lceil P \rceil \rangle C \langle \blacklozenge Q \rangle \text{ iff } P \subseteq \text{awlp}(P, Q)$$

PROOF. $\implies$: Suppose $\sigma \in P$. Then, $\eta(\sigma) \models [P]$ and since $\models \langle [P] \rangle C \langle \blacklozenge Q \rangle$,

$$\begin{aligned} \llbracket C \rrbracket^\dagger(\eta(\sigma)) \models \blacklozenge Q &= \exists(u, v) : U^2 \setminus \{(\emptyset, \emptyset)\}. \llbracket Q \rrbracket^{(u)} \oplus \uparrow^{(v)} \oplus \top \\ &= \{m \mid \text{supp}(m) \cap Q_\cup \neq \emptyset\} \end{aligned}$$

That is, $\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q_\cup \neq \emptyset$, so we have $\sigma \in \text{awlp}(C, Q)$.

$\Leftarrow$: Suppose $m \models [P]$, i.e. $|m| = 1$ and $\text{supp}(m) \subseteq P \subseteq \text{awlp}(C, Q)$. By definition, for any $\sigma \in \text{supp}(m)$, $\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q_\cup \neq \emptyset$. It holds then that

$$\text{supp}(\llbracket C \rrbracket^\dagger(m)) \cap Q_\cup = \left(\bigcup_{\sigma \in \text{supp}(m)} \text{supp}(\llbracket C \rrbracket(\sigma)) \right) \cap Q_\cup = \bigcup_{\sigma \in \text{supp}(m)} (\text{supp}(\llbracket C \rrbracket(\sigma)) \cap Q_\cup) \neq \emptyset$$

This gives us $\llbracket C \rrbracket^\dagger(m) \models \blacklozenge Q$, as desired. □

E Soundness and Relative Completeness

Here, we prove the soundness and (relative) completeness of TOL. First, some preliminary definitions:

Definition E.1. For any test $b \in 2^\Sigma$ and $m \in \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$, we define the *projection* of m onto b to be the following weighting function:

$$(b ? m)(\sigma) \triangleq \begin{cases} m(\sigma) & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = 1 \\ 0 & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = \emptyset \text{ or } \sigma = \cup \end{cases}$$

In our proofs, it will be useful to decompose a program configuration $m \in \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$ into two – one part which solely describes weights on program states Σ , and another which solely describes the degree of nontermination. We introduce the following notational devices:

Definition E.2. Suppose $m \in \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$. We define $\text{term} ? m \triangleq \text{true} ? m$ and $\text{div} ? m \triangleq m(\cup) \cdot \eta(\cup)$ to be the stateful and nonterminating components of m , respectively. These are weighting functions:

$$(\text{term} ? m)(\sigma) = \begin{cases} m(\sigma) & \text{if } \sigma \in \Sigma \\ 0 & \text{if } \sigma = \cup \end{cases} \quad (\text{div} ? m)(\sigma) = \begin{cases} 0 & \text{if } \sigma \in \Sigma \\ m(\sigma) & \text{if } \sigma = \cup \end{cases}$$

We give some properties for the projection operator:

LEMMA E.3. *The projection operator $(b ? -)$ commutes with the following operations:*

- (i) $\sum_{i \in I} (b ? m_i) = b ? \sum_{i \in I} m_i$
- (ii) $u \cdot (b ? m) = b ? u \cdot m$ and $(b ? m) \cdot u = b ? m \cdot u$

Moreover, for any outcome assertion φ :

- (ii) if for all $m \in \varphi$, $m = b ? m'$ for some m' , then $\varphi \models b$
- (iv) if for all $m \in \varphi$, $m = \text{div} ? m'$ for some m' , then $\varphi \models \text{div}$.

PROOF. (i) and (ii): We show that $(u \cdot \sum_{i \in I} (b ? m_i) \cdot v)(\sigma) = (b ? u \cdot \sum_{i \in I} m_i \cdot v)(\sigma)$. From the definition of *projection*, it is straightforward to see that these are equal to

$$\begin{cases} u \cdot \sum_{i \in I} m_i(\sigma) \cdot v & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = 1 \\ 0 & \text{if } \llbracket b \rrbracket_{\text{Test}}(\sigma) = \emptyset \text{ or } \sigma = \cup \end{cases}$$

(iii): Suppose that for all $m \in \varphi$, $m = b ? m'$ for some m' . Then, it should be clear that for all $\sigma \in \text{supp}(m) = \text{supp}(b ? m')$, $\llbracket b \rrbracket_{\text{Test}}(\sigma) = 1$. By definition, $\varphi \models b$. (iv) holds by a similar argument. $\square$

COROLLARY E.4. $\llbracket \text{assume } e \rrbracket^\dagger(b ? m) = b ? (\llbracket \text{assume } e \rrbracket^\dagger(m))$

PROOF.

$$\begin{aligned}
 & b ? \llbracket \text{assume } e \rrbracket^\dagger(m)(\tau) \\
 &= b ? \left[\sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot \llbracket \text{assume } e \rrbracket(\sigma) + m(\cup) \cdot \eta(\cup) \right] \\
 &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} b ? [m(\sigma) \cdot \llbracket \text{assume } e \rrbracket(\sigma)(\tau)] + b ? m(\cup) \cdot \eta(\cup) \\
 &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} b ? [m(\sigma) \cdot \llbracket e \rrbracket(\sigma) \cdot \eta(\sigma)] \\
 &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} b ? [m(\sigma)] \cdot \llbracket e \rrbracket(\sigma) \cdot \eta(\sigma) \\
 &= \sum_{\sigma \in \text{supp}(b ? m) \cap \Sigma} b ? [m(\sigma)] \cdot \llbracket \text{assume } e \rrbracket(\sigma)
 \end{aligned}$$

Note that this is precisely $\llbracket \text{assume } e \rrbracket^\dagger(b ? m)$. $\square$

We proceed by proving some results on the semantics of iteration. Recall that the semantics for a command $C^{(e,e')}$ is given in terms of the characteristic function $\Phi_{\langle C, e, e' \rangle} : (\Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)) \rightarrow \Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_\cup)$:

$$\Phi_{\langle C, e, e' \rangle}(f)(\sigma) = \llbracket e \rrbracket(\sigma) \cdot f^\dagger(\llbracket C \rrbracket(\sigma)) + \llbracket e' \rrbracket(\sigma) \cdot \eta(\sigma)$$

The following Lemma states that applications of $\Phi_{\langle C, e, e' \rangle}$ starting on $\perp$ are equivalent to *unrolling* $C^{(e,e')}$ a corresponding number of times.

LEMMA E.5. For all $n \in \mathbb{N}$, $\sigma \in \Sigma$, and $\tau \in \Sigma_\cup$,³

$$\Phi_{\langle C, e, e' \rangle}^{n+1}(\perp)(\sigma)(\tau) = \begin{cases} \sum_{k=0}^n \llbracket (\text{assume } e \ ; C)^n \ ; \text{assume } e' \rrbracket(\sigma)(\tau) & \text{if } \tau \in \Sigma \\ \perp^\dagger(\llbracket (\text{assume } e \ ; C)^{n+1} \rrbracket(\sigma))(\cup) & \text{if } \tau = \cup \end{cases}$$

Note: For a command C , we define $C^0 \triangleq \text{skip}$ and $C^{n+1} \triangleq C^n \ ; C$.

PROOF. We proceed by induction on n .

- $n = 0$. We have that

$$\begin{aligned}
 \Phi_{\langle C, e, e' \rangle}(\perp)(\sigma) &= \llbracket e \rrbracket(\sigma) \cdot \perp^\dagger(\llbracket C \rrbracket(\sigma)) + \llbracket e' \rrbracket(\sigma) \cdot \eta(\sigma) \\
 &= \perp^\dagger(\llbracket \text{assume } e \ ; C \rrbracket(\sigma)) + \llbracket \text{assume } e' \rrbracket(\sigma)
 \end{aligned}$$

Now, if $\tau \in \Sigma$, then $\perp^\dagger(\llbracket \text{assume } e \ ; C \rrbracket(\sigma))(\tau) = \emptyset$, since $\text{supp}(\perp^\dagger(\llbracket \text{assume } e \ ; C \rrbracket(\sigma))) = \{\cup\}$. On the other hand, $\llbracket \text{assume } e' \rrbracket(\sigma)(\cup) = \emptyset$. This gives us the two cases, as desired.

³Our semantics must capture two computational effects – weighted execution and nontermination – each of which compose differently when programs are sequenced. As a result, the unrolling is expressed differently depending on whether we want the final weight on a program state in Σ or the final weight of the nontermination outcome. In the case of the former, we aggregate the weight collected for traces terminating within the first n iterations. In the latter, we push any weight of traces not yet terminated (including any divergent weight from potential inner loops) onto $\cup$.

- *Inductive step.* Suppose the claim holds for n . First,

$$\begin{aligned}\Phi_{\langle C, e, e' \rangle}^{n+2}(\perp)(\sigma) &= \llbracket e \rrbracket(\sigma) \cdot (\Phi_{\langle C, e, e' \rangle}^{n+1}(\perp))^{\dagger}(\llbracket C \rrbracket(\sigma)) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma) \\ &= \llbracket e \rrbracket(\sigma) \cdot \left(\sum_{\tau' \in \text{supp}(\llbracket C \rrbracket(\sigma))} \llbracket C \rrbracket(\sigma)(\tau') \cdot \Phi_{\langle C, e, e' \rangle}^{n+1}(\perp)(\tau') \right) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)\end{aligned}$$

Now, for $\tau \in \Sigma$, the induction hypothesis gives us:

$$\begin{aligned}\Phi_{\langle C, e, e' \rangle}^{n+2}(\perp)(\sigma)(\tau) &= \llbracket e \rrbracket(\sigma) \cdot \left(\sum_{\tau' \in \text{supp}(\llbracket C \rrbracket(\sigma))} \llbracket C \rrbracket(\sigma)(\tau') \cdot \sum_{k=0}^n \llbracket (\text{assume } e \mathbin{;} C)^k \mathbin{;} \text{assume } e' \rrbracket(\tau')(\tau) \right) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)(\tau) \\ &= \left(\sum_{\tau' \in \text{supp}(\llbracket C \rrbracket(\sigma))} \sum_{k=0}^n \llbracket e \rrbracket(\sigma) \cdot \llbracket C \rrbracket(\sigma)(\tau') \cdot \llbracket (\text{assume } e \mathbin{;} C)^k \mathbin{;} \text{assume } e' \rrbracket(\tau')(\tau) \right) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)(\tau) \\ &= \left(\sum_{k=0}^n \sum_{\tau' \in \text{supp}(\llbracket C \rrbracket(\sigma))} \llbracket e \rrbracket(\sigma) \cdot \llbracket C \rrbracket(\sigma)(\tau') \cdot \llbracket (\text{assume } e \mathbin{;} C)^k \mathbin{;} \text{assume } e' \rrbracket(\tau')(\tau) \right) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)(\tau) \\ &= \left(\sum_{k=1}^{n+1} \llbracket (\text{assume } e \mathbin{;} C)^k \mathbin{;} \text{assume } e' \rrbracket(\sigma)(\tau) \right) + \llbracket \text{assume } e' \rrbracket(\sigma)(\tau) \\ &= \sum_{k=0}^{n+1} \llbracket (\text{assume } e \mathbin{;} C)^k \mathbin{;} \text{assume } e' \rrbracket(\sigma)(\tau)\end{aligned}$$

Applying the other case of the induction hypothesis:

$$\begin{aligned}\Phi_{\langle C, e, e' \rangle}^{n+2}(\perp)(\sigma)(\cup) &= \llbracket e \rrbracket(\sigma) \cdot \left(\sum_{\tau' \in \text{supp}(\llbracket C \rrbracket(\sigma))} \llbracket C \rrbracket(\sigma)(\tau') \cdot \perp^{\dagger}(\llbracket (\text{assume } e \mathbin{;} C)^{n+1} \rrbracket(\tau'))(\cup) \right) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)(\cup) \\ &= \perp^{\dagger}(\llbracket (\text{assume } e \mathbin{;} C)^{n+2} \rrbracket(\sigma))(\cup) + \llbracket e' \rrbracket(\sigma) + \eta(\sigma)(\cup) \\ &= \perp^{\dagger}(\llbracket (\text{assume } e \mathbin{;} C)^{n+2} \rrbracket(\sigma))(\cup)\end{aligned}$$

□

We can now reformulate the semantics of $C^{(e, e')}$ by unrolling the command iteration-by-iteration:

LEMMA E.6 (UNROLLING). *For all $\sigma \in \Sigma$ and $\tau \in \Sigma_{\cup}$,*

$$\llbracket C^{(e, e')} \rrbracket(\sigma)(\tau) = \begin{cases} \sum_{n \in \mathbb{N}} \llbracket (\text{assume } e \mathbin{;} C)^n \mathbin{;} \text{assume } e' \rrbracket(\sigma)(\tau) & \text{if } \tau \in \Sigma \\ \inf_{n \in \mathbb{N}} \perp^{\dagger}(\llbracket (\text{assume } e \mathbin{;} C)^n \rrbracket(\sigma))(\tau) & \text{if } \tau = \cup \end{cases}$$

PROOF. By the program semantics (fig. 4) and Kleene's fixed point theorem,

$$\llbracket C^{(e, e')} \rrbracket(\sigma) = \text{lfp } f. \Phi_{\langle C, e, e' \rangle}(f)(\sigma) = \sup_{n \in \mathbb{N}} \Phi_{\langle C, e, e' \rangle}^n(\perp)(\sigma)$$

Now, suppose $\tau \in \Sigma$. Since $\Phi_{\langle C, e, e' \rangle}^0(\perp) = \perp$, or the bottom of the order on $(\Sigma \rightarrow \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup}))$, we can rewrite the supremum as:

$$\sup_{n \in \mathbb{N}} \Phi_{\langle C, e, e' \rangle}^n(\perp)(\sigma)(\tau) = \sup_{n \in \mathbb{N}} \Phi_{\langle C, e, e' \rangle}^{n+1}(\perp)(\sigma)(\tau)$$

By lemma E.5:

$$= \sup_{n \in \mathbb{N}} \sum_{k=0}^n \llbracket (\text{assume } e \ ; C)^k \ ; \text{assume } e' \rrbracket (\sigma)(\tau)$$

By the definition of infinite sums:

$$= \sum_{n \in \mathbb{N}} \llbracket (\text{assume } e \ ; C)^n \ ; \text{assume } e' \rrbracket (\sigma)(\tau)$$

Finally, we can obtain the remaining case using Lemma E.5:

$$\begin{aligned} \sup_{n \in \mathbb{N}} \Phi_{\langle C, e, e' \rangle}^n(\sigma)(\cup) &= \sup_{n \in \mathbb{N}} (\perp^\dagger(\llbracket (\text{assume } e \ ; C)^n \rrbracket)(\cup)) \\ &= \inf_{n \in \mathbb{N}} \perp^\dagger(\llbracket (\text{assume } e \ ; C)^n \rrbracket)(\cup) \end{aligned}$$

□

THEOREM E.7 (SOUNDNESS).

$$\Omega \vdash \langle \varphi \rangle C \langle \psi \rangle \quad \Longrightarrow \quad \models \langle \varphi \rangle C \langle \psi \rangle$$

PROOF. The triple $\langle \varphi \rangle C \langle \psi \rangle$ is proved using inference rules given in fig. 4 and fig. 3. If the last step in this proof makes use of an axiom, then we are done since we took all axioms in Ω to be semantically valid. Otherwise, we proceed by induction on the derivation of $\Omega \vdash \langle \varphi \rangle C \langle \psi \rangle$.

- FALSE. We need to show $\models \langle \perp \rangle C \langle \varphi \rangle$. Suppose $m \models \perp$. But this is impossible, so the claim holds vacuously.
- TRUE. We need to show $\models \langle \varphi \rangle C \langle \top \rangle$. Suppose $m \models \varphi$. It is trivial that $\llbracket C \rrbracket^\dagger(m) \models \top$, so we are done.
- DRV. We want to demonstrate $\models \langle \uparrow^{(u)} \rangle C \langle \uparrow^{(u)} \rangle$. Suppose $m \models \uparrow^{(u)}$. Then, $m = u \cdot \eta(\cup)$ by definition. We have

$$\begin{aligned} \llbracket C \rrbracket^\dagger(m) &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} (u \cdot \eta(\cup))(\sigma) \cdot \llbracket C \rrbracket(\sigma) + (u \cdot \eta(\cup))(\cup) \cdot \eta(\cup) \\ &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} \emptyset \cdot \llbracket C \rrbracket(\sigma) + u \cdot \eta(\cup) = u \cdot \eta(\cup) \end{aligned}$$

So, $\llbracket C \rrbracket^\dagger(m) \models \uparrow^{(u)}$.

- SCALE. By the induction hypothesis, we have $\models \langle \varphi \rangle C \langle \psi \rangle$. We need to show $\models \langle u \odot \varphi \rangle C \langle u \odot \psi \rangle$. Suppose $m \models u \odot \varphi$. By definition, $m = u \cdot m'$ where $m' \models \varphi$. Using (iii) of Lemma B.1, it follows that

$$\llbracket C \rrbracket^\dagger(m) = \llbracket C \rrbracket^\dagger(u \cdot m') = u \cdot \llbracket C \rrbracket^\dagger(m')$$

Since $\llbracket C \rrbracket^\dagger(m') \models \psi$, we can conclude that $\llbracket C \rrbracket^\dagger(m) \models u \odot \psi$.

- DISJ. By the induction hypothesis, $\models \langle \varphi_1 \rangle C \langle \psi_1 \rangle$ and $\models \langle \varphi_2 \rangle C \langle \psi_2 \rangle$. We want to show $\models \langle \varphi_1 \vee \varphi_2 \rangle C \langle \psi_1 \vee \psi_2 \rangle$. Suppose $m \models \varphi_1 \vee \varphi_2$. Without loss of generality, take $m \models \varphi_1$. The induction hypothesis gives us $\llbracket C \rrbracket^\dagger(m) \models \psi_1$, which can be weakened to $\llbracket C \rrbracket^\dagger(m) \models \psi_1 \vee \psi_2$. The case for $m \models \varphi_2$ is symmetric.
- CONJ. We have $\models \langle \varphi_1 \rangle C \langle \psi_1 \rangle$ and $\models \langle \varphi_2 \rangle C \langle \psi_2 \rangle$ by the induction hypothesis and we want to show $\models \langle \varphi_1 \wedge \varphi_2 \rangle C \langle \psi_1 \wedge \psi_2 \rangle$. Supposing $m \models \varphi_1 \wedge \varphi_2$, it holds that $m \models \varphi_1$ and $m \models \varphi_2$. Then, $\llbracket C \rrbracket^\dagger(m) \models \psi_1$ and $\llbracket C \rrbracket^\dagger(m) \models \psi_2$. By definition, $\llbracket C \rrbracket^\dagger(m) \models \psi_1 \wedge \psi_2$, as desired.

- **CHOICE.** By the induction hypothesis, we have that $\models \langle \phi(t) \rangle C \langle \phi'(t) \rangle$ for all $t \in T$. We now show that $\models \langle \bigoplus_{t \in T} \phi(t) \rangle C \langle \bigoplus_{t \in T} \phi'(t) \rangle$. Suppose $m \models \bigoplus_{t \in T} \phi(t)$. By definition, this means $m = \sum_{t \in T} m_t$ such that for all $t \in T$, $m_t \models \phi(t)$. Since $\sum$ commutes with $(-)^{\dagger}$ in the first argument by [Lemma B.1](#),

$$\llbracket C \rrbracket^{\dagger}(m) = \llbracket C \rrbracket^{\dagger}\left(\sum_{t \in T} m_t\right) = \sum_{t \in T} \llbracket C \rrbracket^{\dagger}(m_t)$$

For each m_t , $\llbracket C \rrbracket^{\dagger}(m_t) \models \phi'(t)$, giving us $\llbracket C \rrbracket^{\dagger}(m) \models \bigoplus_{t \in T} \phi'(t)$.

- **EXISTS.** By the induction hypothesis, we are given $\forall t \in T. \models \langle \phi(t) \rangle C \langle \phi'(t) \rangle$. We need to show $\models \langle \exists t : T. \phi(t) \rangle C \langle \exists t : T. \phi'(t) \rangle$. Suppose $m \models \exists t : T. \phi(t)$. By definition, this means $m \in \bigcup_{t \in T} \phi(t)$, so there must be some $t \in T$ for which $m \in \phi(t)$. Then, by the induction hypothesis, $\llbracket C \rrbracket^{\dagger}(m) \models \phi'(t)$, we have that $\llbracket C \rrbracket^{\dagger}(m) \models \exists t : T. \phi'(t)$.
- **CONSEQUENCE.** Given to us are $\models \varphi' \implies \varphi$ and $\models \psi \implies \psi'$, and we have $\models \langle \varphi \rangle C \langle \psi \rangle$ by the induction hypothesis. We need to show $\models \langle \varphi' \rangle C \langle \psi' \rangle$. Suppose $m \models \varphi'$. Then, $m \models \varphi$ and $\llbracket C \rrbracket^{\dagger}(m) \models \psi$. Finally, this gives us $\llbracket C \rrbracket^{\dagger}(m) \models \psi'$ as desired.
- **SKIP.** We need to show that $\models \langle \varphi \rangle \text{skip} \langle \varphi \rangle$. Suppose that $m \models \varphi$. We have that $\llbracket \text{skip} \rrbracket^{\dagger}(m) = m$, so $\llbracket \text{skip} \rrbracket^{\dagger}(m) \models \varphi$ as desired.
- **SEQ.** Given $\Omega \vdash \langle \varphi \rangle C_1 \langle \vartheta \rangle$ and $\Omega \vdash \langle \vartheta \rangle C_2 \langle \psi \rangle$, we need to show $\models \langle \varphi \rangle C_1 \circ C_2 \langle \psi \rangle$. Suppose $m \models \varphi$, so by the induction hypothesis, $\llbracket C_1 \rrbracket^{\dagger}(m) \models \vartheta$ and $\llbracket C_2 \rrbracket^{\dagger}(\llbracket C_1 \rrbracket^{\dagger}(m)) \models \psi$. Since $\llbracket C_2 \rrbracket^{\dagger}(\llbracket C_1 \rrbracket^{\dagger}(m)) = (\llbracket C_1 \rrbracket^{\dagger} \circ \llbracket C_2 \rrbracket^{\dagger})(m) = \llbracket C_1 \circ C_2 \rrbracket^{\dagger}(m)$, we are done.
- **PLUS.** Given $\varphi \models \mathbb{1}$, $\Omega \vdash \langle \varphi \rangle C_1 \langle \psi_1 \rangle$, and $\Omega \vdash \langle \varphi \rangle C_2 \langle \psi_2 \rangle$, we need to show $\models \langle \varphi \rangle C_1 + C_2 \langle \psi_1 \oplus \psi_2 \rangle$. First, suppose $m \models \varphi$. Since $\varphi \models \mathbb{1}$, it must be that $\text{supp}(m) \subseteq \Sigma$. By [Lemma B.1](#), we know that the $(-)^{\dagger}$ operator commutes with $+$ in its first argument:

$$\llbracket C_1 + C_2 \rrbracket^{\dagger}(m) = \llbracket C_1 \rrbracket^{\dagger}(m) + \llbracket C_2 \rrbracket^{\dagger}(m)$$

By the induction hypothesis, $\llbracket C_1 \rrbracket^{\dagger}(m) \models \psi_1$ and $\llbracket C_2 \rrbracket^{\dagger}(m) \models \psi_2$, so we have $\llbracket C_1 + C_2 \rrbracket^{\dagger}(m) \models \psi_1 \oplus \psi_2$.

- **ASSUME.** Suppose $\varphi \models e = u$. Recall that this holds iff

$$\forall m \in \varphi. m(\cup) = 0 \text{ and } \forall \sigma \in \text{supp}(m) \cap \Sigma. \llbracket e \rrbracket(\sigma) = u.$$

We need to show $\models \langle \varphi \rangle \text{assume } e \langle \varphi u \rangle$. Take $m \models \varphi$. Then,

$$\begin{aligned} \llbracket \text{assume } e \rrbracket^{\dagger}(m) &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot \llbracket \text{assume } e \rrbracket(\sigma) + m(\cup) \cdot \eta(\cup) \\ &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot \llbracket e \rrbracket(\sigma) + m(\cup) \cdot \eta(\cup) \\ &= \sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot u \cdot \eta(\sigma) + 0 \\ &= \left(\sum_{\sigma \in \text{supp}(m) \cap \Sigma} m(\sigma) \cdot \eta(\sigma) \right) \cdot u \\ &= m \cdot u \end{aligned}$$

By definition, $m \cdot u \models \varphi \odot u$, as desired.

- **ITER.** We need to show that $\models \langle \varphi_0 \oplus \zeta_0 \rangle C^{(e, e')} \langle \psi_{\infty} \oplus \zeta_{\infty} \rangle$, given the premises. Suppose $m \models \varphi_0 \oplus \zeta_0$. By the induction hypothesis, we have that for all $n \in \mathbb{N}$,

$$\models \langle \varphi_n \oplus \zeta_n \rangle \text{assume } e \circ \zeta \langle \varphi_{n+1} \oplus \zeta_{n+1} \rangle \quad \models \langle \varphi_n \rangle \text{assume } e' \langle \psi_n \rangle$$

By mathematical induction on n , it is straightforward to show that

$$\llbracket (\mathbf{assume} \ e \ ; \ C)^n \rrbracket^\dagger(m) \models \varphi_n \oplus \zeta_n \quad (2)$$

$$\llbracket (\mathbf{assume} \ e \ ; \ C)^n \ ; \ \mathbf{assume} \ e' \rrbracket^\dagger(m) \models \psi_n \oplus \zeta_n \quad (3)$$

for all n . Note that $\models \langle \zeta_n \rangle \mathbf{assume} \ e' \langle \zeta_n \rangle$ since $\zeta_n \models \text{div}$. In particular, we have that for any $m \models \zeta_n$, $\text{supp}(m) \subseteq \{\cup\}$, so $\llbracket \mathbf{assume} \ e' \rrbracket^\dagger(m) = m$.

Consider each of these results separately:

- (1) For all n , $\llbracket (\mathbf{assume} \ e \ ; \ C)^n \ ; \ \mathbf{assume} \ e' \rrbracket^\dagger(m) \models \psi_n \oplus \zeta_n$. Then, there exist $p_n, q_n \in \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$ such that

$$\llbracket (\mathbf{assume} \ e \ ; \ C)^n \ ; \ \mathbf{assume} \ e' \rrbracket^\dagger(m) = p_n + q_n$$

for $p_n \models \psi_n$ and $q_n \models \zeta_n$. Since $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$, it must be that $\sum_{n \in \mathbb{N}} p_n \models \psi_\infty$.

- (2) For all n , $\llbracket (\mathbf{assume} \ e \ ; \ C)^n \ ; \ \mathbf{assume} \ e' \rrbracket^\dagger(m) \models \psi_n \oplus \zeta_n$. Since $(\varphi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty$, we have

$$\left(\inf_{n \in \mathbb{N}} \left| \llbracket (\mathbf{assume} \ e \ ; \ C)^n \rrbracket^\dagger(m) \right| \cdot \top \right) \cdot \eta(\cup) \models \zeta_\infty$$

We can now combine the two parts. By [Lemma E.6](#), the unrolling of $\llbracket C^{(e,e')} \rrbracket^\dagger(m)$ is

$$\begin{aligned} \llbracket C^{(e,e')} \rrbracket^\dagger &= \sum_{n \in \mathbb{N}} \text{term} \ ? \ \llbracket (\mathbf{assume} \ e \ ; \ C)^n \ ; \ \mathbf{assume} \ e' \rrbracket^\dagger(m) \\ &\quad + \left(\inf_{n \in \mathbb{N}} \left| \llbracket (\mathbf{assume} \ e \ ; \ C)^n \rrbracket^\dagger(m) \right| \cdot \top \right) \cdot \eta(\cup) \end{aligned}$$

Therefore, $\llbracket C^{(e,e')} \rrbracket^\dagger(m) \models \psi_\infty \oplus \zeta_\infty$.

□

Paving a path to relative completeness, we first prove that triples of the form $\langle \varphi \rangle C \langle \text{post}(C, \varphi) \rangle$ can be derived, where $\text{post}(C, \varphi)$ is the *strongest postcondition* that makes $\langle \varphi \rangle C \langle \psi \rangle$ true. Defined otherwise:

Definition E.8 (Strongest Postcondition). $\text{post}(C, \varphi)$ consists of all program configurations reachable from φ by executing command C :

$$\text{post}(C, \varphi) \triangleq \{ \llbracket C \rrbracket^\dagger(m) \mid m \in \varphi \}$$

LEMMA E.9.

$$\Omega \vdash \langle \varphi \rangle C \langle \text{post}(C, \varphi) \rangle$$

PROOF. By induction on the structure of the program C .

- $C = \mathbf{skip}$. $\llbracket \mathbf{skip} \rrbracket^\dagger(m) = m$ for all $m \in \mathcal{W}_{\mathcal{A}}(\Sigma_{\cup})$, so $\text{post}(\mathbf{skip}, \varphi) = \varphi$. The desired triple is derived by one application of the [SKIP](#) rule:

$$\frac{}{\langle \varphi \rangle C \langle \varphi \rangle} \text{SKIP}$$

- $C = C_1 \ ; \ C_2$. First, observe that

$$\begin{aligned} \text{post}(C_1 \ ; \ C_2, \varphi) &= \{ \llbracket C_1 \ ; \ C_2 \rrbracket^\dagger(m) \mid m \in \varphi \} \\ &= \{ \llbracket C_2 \rrbracket^\dagger(\llbracket C_1 \rrbracket^\dagger(m)) \mid m \in \varphi \} \\ &= \{ \llbracket C_2 \rrbracket^\dagger(m') \mid m' \in \{ \llbracket C_1 \rrbracket^\dagger(m) \mid m \in \varphi \} \} \end{aligned}$$

$$= \text{post}(C_2, \text{post}(C_1, \varphi))$$

By the induction hypothesis, we have

$$\Omega \vdash \langle \varphi \rangle C_1 \langle \text{post}(C_1, \varphi) \rangle$$

$$\Omega \vdash \langle \text{post}(C_1, \varphi) \rangle C_2 \langle \text{post}(C_2, \text{post}(C_1, \varphi)) \rangle$$

The derivation is completed with the **SEQ** rule:

$$\frac{\frac{\Omega}{\langle \varphi \rangle C_1 \langle \text{post}(C_1, \varphi) \rangle} \quad \frac{\Omega}{\langle \text{post}(C_1, \varphi) \rangle C_2 \langle \text{post}(C_2, \text{post}(C_1, \varphi)) \rangle}}{\langle \varphi \rangle C_1 \circ C_2 \langle \text{post}(C_1 \circ C_2, \varphi) \rangle} \text{SEQ}$$

- $C = C_1 + C_2$. It holds that

$$\begin{aligned} \text{post}(C_1 + C_2, \varphi) &= \{ \llbracket C_1 + C_2 \rrbracket^\dagger(m) \mid m \in \varphi \} \\ &= \{ \llbracket C_1 + C_2 \rrbracket^\dagger(\text{term} ? m + m(\cup) \cdot \eta(\cup)) \mid m \in \varphi \} \end{aligned}$$

By **Lemma B.1** (i), this is equal to:

$$= \{ \llbracket C_1 + C_2 \rrbracket^\dagger(\text{term} ? m) + \llbracket C_1 + C_2 \rrbracket^\dagger(m(\cup) \cdot \eta(\cup)) \mid m \in \varphi \}$$

Observe that $\text{supp}(\text{term} ? m) \subseteq \Sigma$. Applying (ii) and (iv) of **Lemma B.1**:

$$\begin{aligned} &= \{ \llbracket C_1 \rrbracket^\dagger(\text{term} ? m) + \llbracket C_2 \rrbracket^\dagger(\text{term} ? m) + m(\cup) \cdot \eta(\cup) \mid m \in \varphi \} \\ &= \bigcup_{m \in \varphi} \{ \llbracket C_1 \rrbracket^\dagger(m') + \llbracket C_2 \rrbracket^\dagger(m') + m(\cup) \cdot \eta(\cup) \mid m' \in \mathbf{1}(\text{term} ? m) \} \\ &= \exists m : \varphi. \text{post}(C_1, \text{term} ? m) \oplus \text{post}(C_2, \text{term} ? m) \oplus \uparrow (m(\cup)) \end{aligned}$$

We break the derivation into parts. First, note that $\mathbf{1}(\text{term} ? m) \models \text{term}$ for any m . Then, we have the subderivation (*):

$$\frac{\frac{\Omega}{\langle \mathbf{1}(\text{term} ? m) \rangle C_2 \langle \text{post}(C_2, \mathbf{1}(\text{term} ? m)) \rangle} \quad \frac{\Omega}{\langle \mathbf{1}(\text{term} ? m) \rangle C_1 \langle \text{post}(C_1, \mathbf{1}(\text{term} ? m)) \rangle}}{\langle \mathbf{1}(\text{term} ? m) \rangle C_1 + C_2 \langle \text{post}(C_1, \mathbf{1}(\text{term} ? m)) \oplus \text{post}(C_2, \mathbf{1}(\text{term} ? m)) \rangle} \text{PLUS}$$

The full derivation is completed as follows:

$$\frac{\frac{(*)}{\forall m \in \varphi. \langle \mathbf{1}(m) \rangle C_1 + C_2 \langle \text{post}(C_1, \mathbf{1}(\text{term} ? m)) \oplus \text{post}(C_2, \mathbf{1}(\text{term} ? m)) \oplus \uparrow (m(\cup)) \rangle} \quad \frac{\frac{\text{PLUS}}{\langle \mathbf{1}(\text{term} ? m) \rangle C_1 + C_2 \langle \dots \rangle} \quad \frac{\text{Div}}{\langle \uparrow (m(\cup)) \rangle C_1 + C_2 \langle \uparrow (m(\cup)) \rangle}}{\langle \varphi \rangle C_1 + C_2 \langle \text{post}(C_1 + C_2, \varphi) \rangle} \text{CHOICE EXISTS}$$

- $C = \text{assume } e$, where e must either be a test $b \in \text{Test}$ or weight $u \in U$. First, we see that

$$\begin{aligned} \text{post}(\text{assume } e, \varphi) &= \{ \llbracket \text{assume } e \rrbracket^\dagger(m) \mid m \in \varphi \} \\ &= \{ \llbracket \text{assume } e \rrbracket^\dagger(\text{term} ? m + m(\cup) \cdot \eta(\cup)) \mid m \in \varphi \} \\ &= \{ \llbracket \text{assume } e \rrbracket^\dagger(\text{term} ? m) + m(\cup) \cdot \eta(\cup) \mid m \in \varphi \} \end{aligned}$$

Recall that, by **Definition E.1**,

$$\text{term} ? m = b ? (\text{term} ? m) + \neg b ? (\text{term} ? m) = b ? m + \neg b ? m$$

So $1(\text{term} ? m) = 1(b ? m) + 1(\neg b ? m)$. Then, we have that:

$$\begin{aligned} \text{post}(\mathbf{assume} \ b, \varphi) &= \{ \llbracket \mathbf{assume} \ b \rrbracket^\dagger(b ? m + \neg b ? m) + m(\cup) \cdot \eta(\cup) \mid m \in \varphi \} \\ &= \{ b ? m + m(\cup) \cdot \eta(\cup) \mid m \in \varphi \} \\ &= \exists m : \varphi. 1(b ? m) + \uparrow(m(\cup)) \end{aligned}$$

It should be clear that $b ? m \models b = \mathbb{1}$ and $\neg b ? m \models b = \mathbb{0}$. Then, we have (*):

$$\frac{\text{ASSUME} \quad \frac{}{1(b ? m) \models b = \mathbb{1}} \quad \frac{}{1(\neg b ? m) \models b = \mathbb{0}}}{\frac{\langle 1(b ? m) \rangle \mathbf{assume} \ b \ \langle 1(b ? m) \odot \mathbb{1} \rangle \quad \langle 1(\neg b ? m) \rangle \mathbf{assume} \ b \ \langle 1(\neg b ? m) \odot \mathbb{0} \rangle}{\langle 1(b ? m) \oplus 1(\neg b ? m) \rangle \mathbf{assume} \ b \ \langle 1(\neg b ? m) \rangle} \text{CHOICE}}$$

The derivation is completed as follows:

$$\frac{\frac{\frac{}{\langle \uparrow(m(\cup)) \rangle \mathbf{assume} \ b \ \langle \uparrow(m(\cup)) \rangle} \text{Div}}{\vdash m \in \varphi. \quad \frac{\langle 1(b ? m) \oplus 1(\neg b ? m) \rangle \mathbf{assume} \ b \ \langle 1(\neg b ? m) \rangle \quad \langle \uparrow(m(\cup)) \rangle \mathbf{assume} \ b \ \langle \uparrow(m(\cup)) \rangle}{\langle 1(b ? m) \oplus 1(\neg b ? m) \oplus \uparrow(m(\cup)) \rangle \mathbf{assume} \ b \ \langle 1(b ? m) \oplus \uparrow(m(\cup)) \rangle} \text{CHOICE}}{\langle \varphi \rangle \mathbf{assume} \ b \ \langle \text{post}(\mathbf{assume} \ b, \varphi) \rangle} \text{EXISTS}}$$

Suppose instead that e is a weight $u \in U$. Since $\text{supp}(\text{term} ? m) \subseteq \Sigma$, $\llbracket \mathbf{assume} \ u \rrbracket^\dagger(\text{term} ? m) = (\text{term} ? m) \cdot u$ by [Lemma B.1](#). So,

$$\begin{aligned} \text{post}(\mathbf{assume} \ u, \varphi) &= \{ (\text{term} ? m) \cdot u + m(\cup) \cdot \eta(\cup) \mid m \in \varphi \} \\ &= \exists m : \varphi. 1((\text{term} ? m) \odot u) \oplus \uparrow(m(\cup)) \end{aligned}$$

We know $1(\text{term} ? m) \models u = u$, so the full derivation is given by:

$$\frac{\frac{\frac{}{1(\text{term} ? m) \models u = u} \quad \frac{}{\langle \uparrow(m(\cup)) \rangle \mathbf{assume} \ u \ \langle \uparrow(m(\cup)) \rangle} \text{Div}}{\vdash m \in \varphi. \quad \frac{\langle 1(\text{term} ? m) \rangle \mathbf{assume} \ u \ \langle 1(\text{term} ? m) \odot u \rangle \quad \langle \uparrow(m(\cup)) \rangle \mathbf{assume} \ u \ \langle \uparrow(m(\cup)) \rangle}{\langle 1(\text{term} ? m) \oplus \uparrow(m(\cup)) \rangle \mathbf{assume} \ u \ \langle 1(\text{term} ? m) \odot u \oplus \uparrow(m(\cup)) \rangle} \text{CHOICE}}{\langle \varphi \rangle \mathbf{assume} \ u \ \langle \text{post}(\mathbf{assume} \ u, \varphi) \rangle} \text{EXISTS}}$$

- $C = C^{(e, e')}$. Note that $\varphi = \exists m. \varphi. 1(m)$. To use the [ITER](#) rule, we define the corresponding families of assertions, parametric on a weighting function m :

$$\begin{aligned} \varphi_n(m) &\triangleq 1(\text{term} ? \llbracket (\mathbf{assume} \ e ; C)^n \rrbracket^\dagger(m)) \\ \psi_n(m) &\triangleq 1(\text{term} ? \llbracket (\mathbf{assume} \ e ; C)^n ; \mathbf{assume} \ e' \rrbracket^\dagger(m)) \\ \zeta_n(m) &\triangleq \uparrow(\llbracket (\mathbf{assume} \ e ; C)^n \rrbracket^\dagger(m)(\cup)) \\ \psi_\infty(m) &\triangleq 1(\text{term} ? \llbracket C^{(e, e')} \rrbracket^\dagger(m)) \\ \zeta_\infty(m) &\triangleq \uparrow(\llbracket C^{(e, e')} \rrbracket^\dagger(m)(\cup)) \end{aligned}$$

Observe that

$$\begin{aligned}\varphi_n(m) \oplus \zeta_n(m) &= \mathbf{1}(\llbracket (\text{assume } e \circ C)^n \rrbracket^\dagger(m)) = \text{post}((\text{assume } e \circ C)^n, \mathbf{1}(m)) \\ \psi_\infty(m) \oplus \zeta_\infty(m) &= \mathbf{1}\left(\llbracket C^{(e,e')} \rrbracket^\dagger(m)\right) = \text{post}(C^{(e,e')}, \mathbf{1}(m))\end{aligned}$$

Next, we show that the premises of **ITER** hold for these definitions:

$$- (\psi_n(m))_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty(m).$$

Consider a family $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \psi_n(m)$ for all n . By definition of $\psi_n(m)$, m_n can only be one weighting function: $\llbracket (\text{assume } e \circ C)^n \circ \text{assume } e' \rrbracket^\dagger(m)$. Then,

$$\sum_{n \in \mathbb{N}} m_n = \sum_{n \in \mathbb{N}} \text{term ? } \llbracket (\text{assume } e \circ C)^n \circ \text{assume } e' \rrbracket^\dagger(m)$$

By **Lemma E.6**, this is precisely the stateful component of $\llbracket C^{(e,e')} \rrbracket^\dagger(m)$:

$$= \text{term ? } \llbracket C^{(e,e')} \rrbracket^\dagger(m) = \mathbf{1}\left(\llbracket C^{(e,e')} \rrbracket^\dagger(m)\right) = \psi_\infty(m)$$

$$- (\varphi_n(m) \oplus \zeta_n(m))_{n \in \mathbb{N}} \uparrow \zeta_\infty(m).$$

Take a family $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \varphi_n(m) \oplus \zeta_n(m)$ for all n . Then, $m_n \models \mathbf{1}(\llbracket (\text{assume } e \circ C)^n \rrbracket^\dagger(m))$ so $m_n = \llbracket (\text{assume } e \circ C)^n \rrbracket^\dagger(m)$. We have that

$$\left(\inf_{n \in \mathbb{N}} |m_n| \cdot \top\right) \cdot \eta(\cup) = \left(\inf_{n \in \mathbb{N}} \left|\llbracket (\text{assume } e \circ C)^n \rrbracket^\dagger(m)\right| \cdot \top\right) \cdot \eta(\cup)$$

By **Lemma E.6**, this is equal to $\text{div ? } \llbracket C^{(e,e')} \rrbracket^\dagger(m)$, which certainly satisfies $\uparrow (\llbracket C^{(e,e')} \rrbracket^\dagger(m)) = \zeta_\infty(m)$.

$$- \Omega \vdash \langle \varphi_n(m) \oplus \zeta_n(m) \rangle \text{ assume } e \circ C \langle \varphi_{n+1}(m) \oplus \zeta_{n+1}(m) \rangle.$$

$$\begin{aligned}\varphi_{n+1}(m) \oplus \zeta_{n+1}(m) &= \{\llbracket (\text{assume } e \circ C)^{n+1} \rrbracket^\dagger(m)\} \\ &= \{\llbracket \text{assume } e \circ C \rrbracket^\dagger(\llbracket (\text{assume } e \circ C)^n \rrbracket^\dagger(m))\} \\ &= \{\llbracket \text{assume } e \circ C \rrbracket^\dagger(m') \mid m' \in \varphi_n(m) \oplus \zeta_n(m)\} \\ &= \text{post}(\text{assume } e \circ C, \varphi_n(m) \oplus \zeta_n(m))\end{aligned}$$

By the induction hypothesis, this triple is derivable.

$$- \Omega \vdash \langle \varphi_n(m) \rangle \text{ assume } e' \langle \psi_n(m) \rangle.$$

$$\begin{aligned}\varphi_n(m) &= \{\text{term ? } \llbracket (\text{assume } e \circ C)^n \circ \text{assume } e' \rrbracket^\dagger(m)\} \\ &= \{\llbracket \text{assume } e' \rrbracket^\dagger(\text{term ? } (\text{assume } e \circ C)^n)\} \\ &= \{\llbracket \text{assume } e' \rrbracket^\dagger(m') \mid m' \in \varphi_n(m)\} \\ &= \text{post}(\text{assume } e', \varphi_n(m))\end{aligned}$$

By the induction hypothesis, this triple is derivable.

$$- \text{For all } n \in \mathbb{N}, \text{ it is clear that } \varphi_n(m) \models \text{term} \text{ and } \zeta_n(m) \models \text{div}.$$

By definition, $\varphi_0(m) = \mathbf{1}(\text{term ? } m)$ and $\zeta_0(m) = \uparrow (m(\cup))$, so

$$\varphi_0(m) \oplus \zeta_0(m) = \mathbf{1}(\text{term ? } (m) + \text{div ? } m) = \mathbf{1}(m)$$

Then,

$$\varphi = \exists m : \varphi. \varphi_0(m) \oplus \zeta_0(m)$$

$$\text{post}(C^{\langle e, e' \rangle}, \varphi) = \exists m : \varphi. \psi_\infty(m) \oplus \zeta_\infty(m)$$

The derivation $\Omega \vdash \langle \varphi \rangle C^{\langle e, e' \rangle} \langle \text{post}(C^{\langle e, e' \rangle}, \varphi) \rangle$ proceeds as follows:

$$\frac{\frac{\frac{\Omega}{\forall n. \langle \varphi_n(m) \oplus \zeta_n(m) \rangle \text{ assume } e \circ C \langle \varphi_{n+1}(m) \oplus \zeta_{n+1}(m) \rangle} \quad \frac{\Omega}{\langle \varphi_0(m) \oplus \zeta_0(m) \rangle C^{\langle e, e' \rangle} \langle \psi_\infty(m) \oplus \zeta_\infty(m) \rangle}}{\langle \varphi \rangle C^{\langle e, e' \rangle} \langle \text{post}(C^{\langle e, e' \rangle}, \varphi) \rangle} \text{ITER} \quad \text{EXISTS}$$

- $C = a$. We assumed that Ω contains all valid triples pertaining to atomic actions $a \in \text{Act}$, so $\Omega \vdash \langle \varphi \rangle a \langle \text{post}(a, \varphi) \rangle$ since $\models \langle \varphi \rangle a \langle \text{post}(a, \varphi) \rangle$.

□

THEOREM E.10 (RELATIVE COMPLETENESS).

$$\models \langle \varphi \rangle C \langle \psi \rangle \implies \vdash \langle \varphi \rangle C \langle \psi \rangle$$

PROOF. We show that $\text{post}(C, \varphi) \implies \psi$. Suppose that $m \models \text{post}(C, \varphi)$. Then by definition, there must exist some m' such that $m = \llbracket C \rrbracket^\dagger(m')$. Since $\models \langle \varphi \rangle C \langle \psi \rangle$, we know that $m \models \psi$ as well. Then, we can apply CONSEQUENCE to derive the desired triple:

$$\frac{\frac{\Omega}{\langle \varphi \rangle C \langle \text{post}(C, \varphi) \rangle} \text{LEMMA E.9} \quad \text{post}(C, \varphi) \implies \psi}{\langle \varphi \rangle C \langle \psi \rangle} \text{CONSEQUENCE}$$

□

F Rule Derivations

LEMMA F.1. *The following rules are derivable*

$$\text{LEMMA F.1 - PARTIAL} \quad \frac{\langle [P] \rangle C \langle \Box Q \rangle}{\langle \Box P \rangle C \langle \Box Q \rangle} \quad \frac{\langle [P] \rangle C \langle \Box Q \rangle}{\langle \Box P \rangle C \langle \Box Q \rangle} \text{LEMMA F.1 - TOTAL}$$

PROOF. Observe that $\Box P$ is equivalent to:

$$\Box P = \exists m : \Box P. \mathbf{1}(m) = \exists m : \Box P. \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot \mathbf{1}(\eta(\sigma))$$

For $\sigma \in \text{supp}(m)$, $\eta(\sigma) \models [P]$, so $\Box P$ implies $\exists m : \Box P. \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot [P]$. Moreover,

$$\begin{aligned} \Box Q &= \exists m : \Box P. \Box Q \\ &= \exists m : \Box P. \oplus_{\sigma \in \text{supp}(m)} \Box Q = \exists m : \Box P. \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot \Box Q \end{aligned}$$

The derivation is as follows:

$$\frac{\frac{\frac{\langle [P] \rangle C \langle \Box Q \rangle}{\forall \sigma \in \text{supp}(m). \langle m(\sigma) \odot [P] \rangle C \langle m(\sigma) \odot \Box Q \rangle} \text{SCALE}}{\forall m \in \Box P. \langle \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot [P] \rangle C \langle \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot \Box Q \rangle} \text{CHOICE}}{\langle \exists m : \Box P. \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot \Box P \rangle C \langle \exists m : [P]. \oplus_{\sigma \in \text{supp}(m)} m(\sigma) \odot \Box Q \rangle} \text{EXISTS} \quad \text{CONSEQUENCE}$$

□

Note that $\lceil P \rceil \implies \Box P$, so by a simple application of **CONSEQUENCE**, we can derive the converse:

$$\frac{\langle \Box P \rangle C \langle \Box Q \rangle}{\langle \lceil P \rceil \rangle C \langle \Box Q \rangle} \text{CONSEQUENCE}$$

So we see that the two triples $\langle \lceil P \rceil \rangle C \langle \Box Q \rangle$ and $\langle \Box P \rangle C \langle \Box Q \rangle$ are really interchangeable in any derivation. Very similar derivations can be obtained to show that $\langle \lceil P \rceil \rangle C \langle \blacksquare Q \rangle$ and $\langle \blacksquare P \rangle C \langle \blacksquare Q \rangle$ equivalent in the logic as well.

LEMMA F.2. *The following rule is derivable:*

$$\frac{\langle \lceil P \rceil \rangle C \langle \Diamond Q \rangle}{\langle \Diamond P \rangle C \langle \Diamond Q \rangle} \text{LEMMA F.2}$$

PROOF. For each $m \in \Diamond P$, we can fix a state $\sigma_m \in \text{supp}(m)$ such that $\sigma_m \in \lceil P \rceil$ as well. This must exist by definition of $\Diamond P$. Let $u_m = m(\sigma)$ be its assigned weight. Note that $u_m \neq \emptyset$. Then, we have as consequences:

$$\begin{aligned} \Diamond P &\iff \exists m : \Diamond P. 1(m) \\ &\iff \exists m : \Diamond P. (u_m \odot 1(\eta(\sigma_m))) \oplus \top \\ &\implies \exists m : \Diamond P. (u_m \odot \lceil P \rceil) \oplus \top \end{aligned}$$

$$\begin{aligned} \exists m : \Diamond P. (u_m \odot \Diamond Q) \oplus \top &\iff \exists m : \Diamond P. \Diamond Q \oplus \top \\ &\iff \exists m : \Diamond P. \Diamond Q \\ &\iff \Diamond Q \end{aligned}$$

The derivation is as follows:

$$\frac{\forall m \in \lceil P \rceil. \frac{\frac{\langle \lceil P \rceil \rangle C \langle \Diamond Q \rangle}{\langle u_m \odot \lceil P \rceil \rangle C \langle u_m \odot \Diamond Q \rangle} \text{SCALE} \quad \frac{}{\langle \top \rangle C \langle \top \rangle} \text{TRUE}}{\langle (u_m \odot \lceil P \rceil) \oplus \top \rangle C \langle (u_m \odot \Diamond Q) \oplus \top \rangle} \text{CHOICE}}{\langle \exists m : \Diamond P. (u_m \odot \lceil P \rceil) \oplus \top \rangle C \langle \exists m : \Diamond P. (u_m \odot \Diamond Q) \oplus \top \rangle} \text{EXISTS}}{\langle \Diamond P \rangle C \langle \Diamond Q \rangle} \text{CONSEQUENCE}$$

□

LEMMA F.3. *The following rules are derivable:*

$$\begin{array}{c} \text{SEQ-LISBON} \\ \frac{\langle \lceil P \rceil \rangle C_1 \langle \Diamond Q \rangle \quad \langle \lceil Q \rceil \rangle C_2 \langle \Diamond R \rangle}{\langle \lceil P \rceil \rangle C_1 \circ C_2 \langle \Diamond R \rangle} \end{array} \quad \begin{array}{c} \text{SEQ-TOTAL-HOARE} \\ \frac{\langle \lceil P \rceil \rangle C_1 \langle \blacksquare Q \rangle \quad \langle \lceil Q \rceil \rangle C_2 \langle \blacksquare R \rangle}{\langle \lceil P \rceil \rangle C_1 \circ C_2 \langle \blacksquare R \rangle} \end{array}$$

PROOF. We show only the derivation for **SEQ-TOTAL-HOARE** – we derive **SEQ-LISBON** nearly identically using an application of **Lemma F.2**.

$$\frac{\langle \lceil P \rceil \rangle C_1 \langle \blacksquare Q \rangle \quad \frac{\langle \lceil Q \rceil \rangle C_2 \langle \blacksquare R \rangle}{\langle \blacksquare Q \rangle C_2 \langle \blacksquare R \rangle} \text{LEMMA F.1 - TOTAL}}{\langle \lceil P \rceil \rangle C_1 \circ C_2 \langle \blacksquare R \rangle} \text{SEQ}$$

□

LEMMA F.4. *The following rule is derivable:*

$$\frac{\varphi_1 \models b \quad \langle \varphi_1 \rangle C_1 \langle \psi_1 \rangle \quad \varphi_2 \models \neg b \quad \langle \varphi_2 \rangle C_2 \langle \psi_2 \rangle}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \psi_1 \oplus \psi_2 \rangle} \text{IF}$$

PROOF. First, we give a subderivation (1):

$$\frac{\frac{\varphi_1 \models b}{\langle \varphi_1 \rangle \text{ assume } b \langle \varphi_1 \rangle} \text{ASSUME} \quad \frac{\varphi_2 \models \neg b}{\langle \varphi_2 \rangle \text{ assume } b \langle \emptyset \odot \varphi_2 \rangle} \text{ASSUME}}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ assume } b \langle \varphi_1 \rangle} \text{CHOICE} \quad \frac{\langle \varphi_1 \rangle C \langle \psi_1 \rangle}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ assume } b \circ C_1 \langle \psi_1 \rangle} \text{SEQ}$$

The proof of (2) is nearly identical. We complete the derivation with:

$$\frac{\frac{(1)}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ assume } b \circ C_1 \langle \psi_1 \rangle} \text{SEQ} \quad \frac{(2)}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ assume } \neg b \circ C_2 \langle \psi_2 \rangle} \text{SEQ}}{\langle \varphi_1 \oplus \varphi_2 \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \psi_1 \oplus \psi_2 \rangle} \text{PLUS}$$

□

LEMMA F.5. *The following rule is derivable:*

$$\frac{\langle \lceil P \wedge b \rceil \rangle C_1 \langle \blacksquare Q \rangle \quad \langle \lceil P \wedge \neg b \rceil \rangle C_2 \langle \blacksquare Q \rangle}{\langle \lceil P \rceil \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \blacksquare Q \rangle} \text{IF-HOARE}$$

PROOF. Note that $\blacksquare(P \wedge b) \models b$ and $\blacksquare(P \wedge \neg b) \models \neg b$. The derivation proceeds as follows:

$$\frac{\frac{\langle \lceil P \wedge b \rceil \rangle C_1 \langle \blacksquare Q \rangle}{\langle \blacksquare(P \wedge b) \rangle C_1 \langle \blacksquare Q \rangle} \text{LEMMA F.1} \quad \frac{\langle \lceil P \wedge \neg b \rceil \rangle C_2 \langle \blacksquare Q \rangle}{\langle \blacksquare(P \wedge \neg b) \rangle C_2 \langle \blacksquare Q \rangle} \text{LEMMA F.1}}{\langle \blacksquare(P \wedge b) \oplus \blacksquare(P \wedge \neg b) \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \blacksquare Q \oplus \blacksquare Q \rangle} \text{IF} \quad \frac{}{\langle \lceil P \rceil \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \blacksquare Q \rangle} \text{CONSEQUENCE}$$

□

LEMMA F.6. *The following rule is derivable:*

$$\frac{\langle \lceil P \wedge b \rceil \rangle C_1 \langle \diamond Q \rangle \quad \langle \lceil P \wedge \neg b \rceil \rangle C_2 \langle \diamond Q \rangle}{\langle \lceil P \rceil \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \diamond Q \rangle} \text{IF-LISBON}$$

PROOF. Subderivation (1):

$$\frac{\frac{[P \wedge b] \models b \quad \langle \lceil P \wedge b \rceil \rangle C_1 \langle \diamond Q \rangle \quad \emptyset \odot \top \models \neg b \quad \frac{\frac{}{\langle \top \rangle C_2 \langle \top \rangle} \text{TRUE}}{\langle \emptyset \odot \top \rangle C_2 \langle \emptyset \odot \top \rangle} \text{SCALE}}{\langle \lceil P \wedge b \rceil \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \diamond Q \rangle} \text{IF}}{\langle \diamond(P \wedge b) \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \diamond Q \rangle} \text{LEMMA F.2}$$

Part (2) is symmetrical.

$$\begin{array}{c}
 (2) \\
 \frac{\frac{\frac{\langle \Diamond(P \wedge \neg b) \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \Diamond Q \rangle}{\text{LEMMA F.2}}}{\langle \Diamond(P \wedge b) \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \Diamond Q \rangle} \text{LEMMA F.2} \quad \vdots}{\langle \Diamond(P \wedge b) \vee \Diamond(P \wedge \neg b) \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \Diamond Q \vee \Diamond Q \rangle} \text{DISJ} \\
 \hline
 \langle [P] \rangle \text{ if } b \text{ then } C_1 \text{ else } C_2 \langle \Diamond Q \rangle \quad \text{CONSEQUENCE}
 \end{array}$$

□

LEMMA F.7. *The following rule is derivable:*

$$\frac{\zeta \models \text{div}}{\langle \zeta \rangle C \langle \zeta \rangle} \text{DIV}^*$$

PROOF. Since $\zeta \models \text{div}$, we know that for all m such that $m \models \zeta$, $\text{supp}(m) \subseteq \{\cup\}$. Then, these are equivalent: $\zeta = \exists m : \zeta. \uparrow^{(m(\cup))}$. The derivation is as follows:

$$\frac{\frac{\forall m \in \zeta. \quad \frac{\langle \uparrow^{(m(\cup))} \rangle C \langle \uparrow^{(m(\cup))} \rangle}{\text{DIV}}}{\langle \exists m : \zeta. \uparrow^{(m(\cup))} \rangle C \langle \exists m : \zeta. \uparrow^{(m(\cup))} \rangle} \text{EXISTS} \\
 \hline
 \langle \zeta \rangle C \langle \zeta \rangle \quad \text{CONSEQUENCE}$$

□

LEMMA F.8. *The following rule is derivable:*

$$\frac{\forall n \in \mathbb{N}. \quad \varphi_n \models b \quad \frac{(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty \quad (\varphi_n \oplus \psi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty}{\zeta_n \models \text{div} \quad \langle \varphi_n \oplus \zeta_n \rangle C \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle}}{\langle \varphi_0 \oplus \psi_0 \oplus \zeta_0 \rangle \text{ while } b \text{ do } C \langle \psi_\infty \oplus \zeta_\infty \rangle} \text{WHILE}$$

PROOF. We will use **ITER** to derive this rule. First, we give the following subderivation (*):

$$\begin{array}{c}
 \text{ASSUME} \quad \frac{\varphi_n \models b}{\langle \varphi_n \rangle \text{ assume } b \langle \varphi_n \rangle} \quad \frac{\psi_n \models \neg b}{\langle \psi_n \rangle \text{ assume } b \langle \emptyset \cdot \top \rangle} \quad \text{ASSUME} \quad \frac{\zeta_n \models \text{div}}{\langle \zeta_n \rangle \text{ assume } b \langle \zeta_n \rangle} \text{DIV}^* \\
 \hline
 \langle \varphi_n \oplus \psi_n \oplus \zeta_n \rangle \text{ assume } b \langle \varphi_n \oplus \zeta_n \rangle \quad \vdots \\
 \hline
 \langle \varphi_n \oplus \psi_n \oplus \zeta_n \rangle \text{ assume } b \langle \varphi_n \oplus \zeta_n \rangle \quad \text{CHOICE}
 \end{array}$$

This gives us (1):

$$\begin{array}{c}
 (*) \\
 \hline
 \frac{\langle \varphi_n \oplus \psi_n \oplus \zeta_n \rangle \text{ assume } b \langle \varphi_n \oplus \zeta_n \rangle \quad \langle \varphi_n \oplus \zeta_n \rangle C \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle}{\langle \varphi_n \oplus \psi_n \oplus \zeta_n \rangle \text{ assume } b \circ C \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle} \text{SEQ}
 \end{array}$$

And we derive (2) similarly:

$$\begin{array}{c}
 \frac{\varphi_n \models b}{\langle \varphi_n \rangle \text{ assume } \neg b \langle \emptyset \cdot \varphi_n \rangle} \text{ASSUME} \quad \frac{\psi_n \models \neg b}{\langle \psi_n \rangle \text{ assume } \neg b \langle \psi_n \rangle} \text{ASSUME} \\
 \hline
 \langle \varphi_n \oplus \psi_n \rangle \text{ assume } \neg b \langle \psi_n \rangle \quad \text{CHOICE}
 \end{array}$$

Now, note that since $\varphi_n \models b$ and $\psi_n \models \neg b$, it must be that $\varphi_n \oplus \psi_n \models \text{true}$. Recall also that **while** b **do** C is sugar for $C^{(b, \neg b)}$. The full derivation proceeds as follows:

$$\frac{\begin{array}{c} (1) \\ \forall n \in \mathbb{N}. \quad \frac{\langle \varphi_n \oplus \psi_n \oplus \zeta_n \rangle \text{ assume } b \ ; \ C \ \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle}{\langle \varphi_0 \oplus \psi_0 \oplus \zeta_0 \rangle \text{ while } b \text{ do } C \ \langle \psi_\infty \oplus \zeta_\infty \rangle} \end{array} \quad \frac{\begin{array}{c} (2) \\ \langle \varphi_n \oplus \psi_n \rangle \text{ assume } \neg b \ \langle \psi_n \rangle \end{array}}{\text{ITER}} \quad \square$$

LEMMA F.9. *The following rule is derivable:*

$$\frac{\langle \lceil P \wedge b \rceil \rangle C \ \langle \Box P \rangle}{\langle \lceil P \rceil \rangle \text{ while } b \text{ do } C \ \langle \Box(P \wedge \neg b) \rangle} \text{ INVARIANT}$$

PROOF. We will derive this rule using **WHILE**. For all $n \in \mathbb{N}$, let

$$\varphi_n \triangleq \blacksquare(P \wedge b) \quad \psi_n = \psi_\infty \triangleq \blacksquare(P \wedge \neg b) \quad \zeta_n = \zeta_\infty \triangleq \exists u. \uparrow^{(u)}$$

We show that the necessary premises hold:

- Take any $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \psi_n$ for all n . That is, $m_n \models \blacksquare(P \wedge \neg b)$, so $\emptyset \subset \text{supp}(m_n) \subseteq (P \wedge \neg b)$. Then,

$$\emptyset \subset \text{supp} \left(\sum_{n \in \mathbb{N}} m_n \right) = \bigcup_{n \in \mathbb{N}} \text{supp}(m_n) \subseteq (P \wedge \neg b)$$

By definition, $\sum_{n \in \mathbb{N}} m_n \models \blacksquare(P \wedge \neg b) = \psi_\infty$. Thus, $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$.

- Again, take any $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \varphi_n \oplus \psi_n \oplus \zeta_n$ for all n . Note that $\zeta_\infty = \exists u : U. \uparrow^{(u)} = \{m \mid \text{supp}(m) \subseteq \{\cup\}\}$. Thus, it always holds that

$$\left(\inf_{n \in \mathbb{N}} |m_n| \cdot \top \right) \cdot \eta(\cup) \models \zeta_\infty$$

We have $(\varphi_n \oplus \psi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty$.

- Clearly, $\blacksquare(P \wedge b) \models b$, $\blacksquare(P \wedge \neg b) \models \neg b$, and $\exists u : U. \uparrow^{(u)} \models \text{div}$ for all n .

The derivation is as follows:

$$\frac{\frac{\frac{\langle \lceil P \wedge b \rceil \rangle C \ \langle \Box P \rangle}{\langle \Box(P \wedge b) \rangle C \ \langle \Box P \rangle} \text{ LEMMA F.1 - PARTIAL}}{\langle \blacksquare(P \wedge b) \oplus \exists u : U. \uparrow^{(u)} \rangle C \ \langle \blacksquare(P \wedge b) \oplus \blacksquare(P \wedge \neg b) \oplus \exists u : U. \uparrow^{(u)} \rangle} \text{ CONSEQUENCE}}{\frac{\langle \blacksquare(P \wedge b) \oplus \blacksquare(P \wedge \neg b) \oplus \exists u : U. \uparrow^{(u)} \rangle \text{ while } b \text{ do } C \ \langle \blacksquare(P \wedge \neg b) \oplus \exists u : U. \uparrow^{(u)} \rangle}{\langle \lceil P \rceil \rangle \text{ while } b \text{ do } C \ \langle \Box(P \wedge \neg b) \rangle} \text{ WHILE}} \text{ CONSEQUENCE} \quad \square$$

LEMMA F.10. *The following rule is derivable:*

$$\frac{\forall n < N. \quad \varphi_{n+1} \models b \quad \varphi_0 \models \neg b \quad \langle \varphi_{n+1} \rangle C \ \langle \varphi_n \rangle}{\langle \exists N : \mathbb{N}. \varphi_N \rangle \text{ while } b \text{ do } C \ \langle \varphi_0 \rangle} \text{ VARIANT}$$

PROOF. Once again, we use **WHILE**. For all N and n , define

$$\varphi'_n \triangleq \begin{cases} \varphi_{N-n} & \text{if } n < N \\ \emptyset \odot \top & \text{otherwise} \end{cases} \quad \psi_n \triangleq \begin{cases} \varphi_0 & \text{if } n \in \{N, \infty\} \\ \emptyset \odot \top & \text{otherwise} \end{cases} \quad \zeta_n = \zeta_\infty \triangleq 1_0$$

We give the necessary premises for the rule:

- Take any $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \psi_n$ for all $n \in \mathbb{N}$. Then, $m_N \models \varphi_0$ while $m_n \models \mathbb{0} \odot \top$ for $n \neq N$. So $\sum_{n \in \mathbb{N}} m_n = m_N \models \varphi_0$. This gives us $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$.
- Take any $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \varphi_n \oplus \psi_n \oplus \zeta_n$. We know $|m_n| = \mathbb{0}$ for all $n \geq N$, so

$$\left(\inf_{n \in \mathbb{N}} |m_n| \cdot \top \right) \cdot \eta(\cup) = \mathbb{0} \cdot \eta(\cup) \models \mathbb{0} \odot \top$$

We have $(\varphi_n \oplus \psi_n \oplus \zeta_n) \models \zeta_\infty$.

- We have as premise that for all $n < N$, $\varphi_n, (\mathbb{0} \odot \top) \models b$, so $\varphi'_n \models b$. $\varphi_0, (\mathbb{0} \odot \top) \models \neg b$, so $\psi_n \models \neg b$. And finally $\zeta_n = \mathbb{0} \odot \top \models \text{div}$.

To derive $\langle \varphi'_n \rangle C \langle \varphi'_{n+1} \oplus \psi_{n+1} \rangle$ for all n , we consider two cases: either $n < N$ or $n \geq N$. We take these subderivations as part (1):

$$\frac{\forall m < N. \langle \varphi_{m+1} \rangle C \langle \varphi_m \rangle}{\forall n < N. \langle \varphi_{N-n} \rangle C \langle \varphi_{N-(n+1)} \rangle} \quad \frac{\frac{\langle \top \rangle C \langle \top \rangle \text{ TRUE}}{\langle \mathbb{0} \cdot \top \rangle C \langle \mathbb{0} \odot \top \rangle} \text{ SCALE}}{\forall n \geq N. \langle \varphi'_n \rangle C \langle \varphi'_{n+1} \oplus \psi_{n+1} \rangle}$$

The derivation is completed as follows:

$$\frac{(1) \quad \frac{\forall n \in \mathbb{N}. \langle \varphi'_n \rangle C \langle \varphi'_{n+1} \oplus \psi_{n+1} \rangle}{\forall N \in \mathbb{N}. \langle \varphi_N \rangle \text{ while } b \text{ do } C \langle \varphi_0 \rangle} \text{ WHILE}}{\langle \exists N : \mathbb{N}. \varphi_N \rangle \text{ while } b \text{ do } C \langle \varphi_0 \rangle} \text{ EXISTS}$$

□

LEMMA F.11. *The following rule is derivable:*

$$\frac{\text{HOARE-VARIANT} \quad (P \wedge b) \Rightarrow R > 0 \quad \forall n \in \mathbb{N}. \langle \llbracket P \wedge b \wedge R = n \rrbracket \rangle C \langle \blacksquare(P \wedge R < n) \rangle}{\langle \llbracket P \wedge R \leq N \rrbracket \rangle \text{ while } b \text{ do } C \langle \blacksquare(P \wedge \neg b) \rangle}$$

PROOF. We use **WHILE** and fix families of assertions φ_n, ψ_n and ζ_n for the rule.

For all $n \in \mathbb{N}$, let $\zeta_n \triangleq \mathbb{0} \odot \top$ and

$$\varphi_n \triangleq \begin{cases} \blacksquare(P \wedge R \leq (N - n) \wedge b) & \text{if } n \leq N \\ \mathbb{0} \odot \top & \text{if } n > N \end{cases} \quad \psi_n \triangleq \begin{cases} \blacksquare(P \wedge R \leq (N - n) \wedge \neg b) & \text{if } n \leq N \\ \mathbb{0} \odot \top & \text{if } n > N \end{cases}$$

Let $\psi_\infty \triangleq \blacksquare(P \wedge \neg b)$ and $\zeta_\infty \triangleq \mathbb{0} \odot \top$.

We clearly have that $\varphi_n \models b$ and $\psi_n \models \neg b$. Next, we show that the necessary convergence/divergence conditions hold:

- Take any $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \psi_n$ for all n . Then $m_n \models \blacksquare(P \wedge \neg b)$, so $\sum_{n \in \mathbb{N}} m_n \models \blacksquare(P \wedge \neg b)$. This gives us $(\psi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$.
- Take $(m_n)_{n \in \mathbb{N}}$ such that $m_n \models \varphi_n \oplus \psi_n \oplus \zeta_n$ for all n . Then for $m > N$, $m_n \models \mathbb{0} \cdot \top$. So, $(\inf_{n \in \mathbb{N}} |m_n| \cdot \top \cdot \eta(\cup) = \mathbb{0} \cdot \eta(\cup))$. Thus, $(\varphi_n \oplus \psi_n \oplus \zeta_n) \uparrow \zeta_\infty$.

Observe that for $n > N$, $\varphi_n \oplus \psi_n \oplus \zeta_n = \llbracket P \wedge R \leq (N - n) \rrbracket$. In part (1) below, we derive the triple $\langle \varphi_n \oplus \zeta_n \rangle C \langle \varphi_{n+1} \oplus \psi_{n+1} \oplus \zeta_{n+1} \rangle$:

$$\frac{\forall m \leq N - n. \quad \frac{\frac{\langle \llbracket P \wedge b \wedge R = m \rrbracket \rangle C \langle \blacksquare(P \wedge R < m) \rangle}{\langle \blacksquare(P \wedge b \wedge R = m) \rangle C \langle \blacksquare(P \wedge R < m) \rangle} \text{LEMMA F.1 - TOTAL}}{\langle \blacksquare(P \wedge b \wedge R = m) \rangle C \langle \blacksquare(P \wedge R \leq (N - (n + 1))) \rangle} \text{CONSEQUENCE}}{\langle \blacksquare(P \wedge b \wedge R \leq (N - n)) \rangle C \langle \blacksquare(P \wedge R \leq (N - (n + 1))) \rangle} \text{EXISTS}$$

We then complete the derivation with an application of **WHILE** and **CONSEQUENCE**:

$$\frac{(1) \quad \frac{\langle \blacksquare(P \wedge b \wedge R \leq (N - n)) \rangle C \langle \blacksquare(P \wedge R \leq (N - (n + 1))) \rangle}{\langle \blacksquare(P \wedge R \leq N) \rangle \text{ while } b \text{ do } C \langle \blacksquare(P \wedge \neg b) \rangle} \text{WHILE}}{\langle \llbracket P \wedge R \leq N \rrbracket \rangle \text{ while } b \text{ do } C \langle \blacksquare(P \wedge \neg b) \rangle} \text{CONSEQUENCE}$$

□

LEMMA F.12. *The following rules are derivable:*

$$\frac{\llbracket P \rrbracket \models b \quad \langle \llbracket P \rrbracket \rangle C \langle \square P \rangle}{\langle \llbracket P \rrbracket \rangle \text{ while } b \text{ do } C \langle \square \text{false} \rangle} Q_{\text{INV-DEMON}} \quad \frac{\llbracket P \rrbracket \models b \quad \langle \llbracket P \rrbracket \rangle C \langle \diamond P \rangle}{\langle \llbracket P \rrbracket \rangle \text{ while } b \text{ do } C \langle \diamond \text{false} \rangle} Q_{\text{INV-ANGEL}}$$

PROOF. We show the full derivation for **QINV-ANGEL**, omitting that for the demonic counterpart.

We use the **WHILE** rule. For all n , take

$$\begin{aligned} \varphi_n &\triangleq (\exists u : U \setminus \{\emptyset\}. \llbracket P \rrbracket^{(u)}) \oplus \blacksquare b & \psi_n &\triangleq \blacksquare(\neg b) & \zeta_n &= \exists u : U. \uparrow^{(u)} \\ \psi_\infty &\triangleq \top & \zeta_\infty &\triangleq \exists u : U \setminus \{\emptyset\}. \uparrow^{(u)} \end{aligned}$$

Take any family of weighting functions $(m_n)_{n \in \mathbb{N}}$.

- If $m_n \models \psi_n$ for all n , it is trivial that $\sum_{n \in \mathbb{N}} m_n \models \top$. So $(\varphi_n)_{n \in \mathbb{N}} \rightsquigarrow \psi_\infty$.
- Suppose $m_n \models \varphi_n \oplus \psi_n \oplus \zeta_n$. Then, $m_n = p + q$ where $p \models \exists u : U \setminus \{\emptyset\}. \llbracket P \rrbracket^{(u)} = \varphi_n$, so $|m_n| > |p| > 0$. This means that, for weight $v > 0$,

$$\left(\inf_{n \in \mathbb{N}} |m_n \cdot \top| \right) \cdot \eta(\cup) = v \cdot \eta(\cup) \models \exists u : U \setminus \{\emptyset\}. \uparrow^{(u)}$$

Thus, $(\varphi_n \oplus \psi_n \oplus \zeta_n)_{n \in \mathbb{N}} \uparrow \zeta_\infty$.

Since $\llbracket P \rrbracket \models b$, we know $\varphi_n = \exists u : U \setminus \{\emptyset\}. \llbracket P \rrbracket^{(u)} \oplus \blacksquare b \models b$. Clearly, $\psi_n \models \neg b$ and $\zeta_n \models \text{div}$.

Note that we can partition $\top = \blacksquare b \oplus \blacksquare(\neg b) \oplus \exists u : U. \uparrow^{(u)}$. So, we have

$$\begin{aligned} \varphi_n \oplus \psi_n \oplus \zeta_n &= (\exists u : U \setminus \{\emptyset\}. \llbracket P \rrbracket^{(u)} \oplus \blacksquare b) \oplus \blacksquare(\neg b) \oplus \exists u : U. \uparrow^{(u)} \\ &= \exists u : U \setminus \{\emptyset\}. \llbracket P \rrbracket^{(u)} \oplus \top \\ &= \diamond P \\ \psi_\infty \oplus \zeta_\infty &= \exists u : U \setminus \{\emptyset\}. \uparrow^{(u)} \oplus \top = \diamond \text{false} \end{aligned}$$

$$\text{NT} \triangleq \left\{ \begin{array}{l} x := 1 \ ; \ y := 2 \ ; \\ \text{while } x + y > 1 \text{ do} \\ \quad x := 3 - x \ ; \\ \quad y := 3 - y \ ; \end{array} \right. \quad \text{MALLOCDIV} \triangleq \left\{ \begin{array}{l} \text{byte} * p \ ; \\ p := (\text{byte} *) \text{malloc}(\text{size} + 16) \ ; \\ \text{while } p = \text{NULL} \text{ do} \\ \quad p := (\text{byte} *) \text{malloc}(\text{size} + 16) \ ; \end{array} \right.$$

Fig. 9. Two non-terminating programs.

Moreover, $\Diamond P \wedge \Box b = \varphi_n \oplus \zeta_n$. The derivation is as follows:

$$\frac{\frac{\frac{\langle [P] \rangle C \langle \Diamond P \rangle}{\langle \Diamond P \rangle C \langle \Diamond P \rangle} \text{LEMMA F.2}}{\langle \Diamond P \wedge \Box b \rangle C \langle \Diamond P \rangle} \text{CONSEQUENCE}}{\langle \Diamond P \rangle \text{ while } b \text{ do } C \langle \Diamond \text{false} \rangle} \text{WHILE}$$

$$\frac{\langle \Diamond P \rangle \text{ while } b \text{ do } C \langle \Diamond \text{false} \rangle}{\langle [P] \rangle \text{ while } b \text{ do } C \langle \Diamond \text{false} \rangle} \text{CONSEQUENCE}$$

□

G Additional Case Studies

G.1 Proving Nontermination

[Raad et al. 2024] advocated the need for formal methods to prove nontermination, as many industrial codebases have bugs that cause programs to diverge and are difficult to discover with testing. Here, we use some of their examples to demonstrate how TOL can be used for nontermination proving. Consider the NT program in Figure 9. To show that this program indeed always diverges, we make use of the **QINV-DEMON** rule to derive the triple $\langle [\text{true}] \rangle \text{NT} \langle \Box \text{false} \rangle$.

$$\begin{aligned} & \langle [\text{true}] \rangle \\ & x := 1 \ ; \ y := 2 \ ; \\ & \langle [x = 1 \wedge y = 2] \rangle \\ & \implies \langle [x + y = 3] \rangle \\ & \quad x := 3 - x \ ; \\ & \quad \langle [3 - x + y = 3] \rangle \quad // \text{ASSIGN} \\ & \quad y := 3 - y \ ; \\ & \quad \langle [(3 - x) + (3 - y) = 3] \rangle \quad // \text{ASSIGN} \\ & \quad \implies \langle [x + y = 3] \rangle \\ & \quad \implies \langle \Box(x + y = 3) \rangle \\ & \langle \Box \text{false} \rangle \end{aligned}$$

As a second example, we will use the rule **QINV-ANGEL** to establish the *possibility* of nontermination in the presence of branching. Consider a loop involving the malloc function in C as in the program MALLOCDIV in Figure 9.

MALLOCDIV allocates memory in a loop, iterating until a successful call to malloc and p is set to a non-null value. Crucially, malloc *may* fail each time, giving rise to a divergent execution. We can thus see that the assertion $p = \text{NULL}$ is a (angelic) quasi-invariant for this loop; we have

$$\frac{\langle [p = \text{NULL}] \rangle p := (\text{byte} *) \text{malloc}(\text{size} + 16) \langle \Diamond(p = \text{NULL}) \rangle}{\langle [p = \text{NULL}] \rangle \text{ while } p = \text{NULL} \text{ do } \dots \langle \Diamond \text{false} \rangle} \text{QINV-ANGEL}$$

```

 $\langle \text{true} \rangle$ 
 $i := 0 \ ; \ j := n$ 
 $\langle \text{true} \rangle \implies \langle \alpha(i) \wedge \beta(j) \wedge j - i \geq 0 \rangle$ 
while  $i \leq j$  do
   $\langle \alpha(i) \wedge \beta(j) \wedge i \leq j \wedge j - i = m \rangle \implies \langle \alpha(i) \wedge \beta(j) \wedge j - i = m \rangle$ 
  if  $A[i] \leq p$  then
     $\langle \alpha(i) \wedge \beta(j) \wedge A[i] \leq p \wedge j - i = m \rangle \implies \langle \alpha(i+1) \wedge \beta(j) \wedge j - i = m \rangle$ 
     $i := i + 1$ 
     $\langle \alpha(i) \wedge \beta(j) \wedge j - i = m - 1 \rangle \quad // \text{ ASSIGN}$ 
  else if  $A[j] \geq p$  then
     $\langle \alpha(i) \wedge \beta(j) \wedge A[j] \geq p \wedge j - i = m \rangle \implies \langle \alpha(i) \wedge \beta(j-1) \wedge j - i = m \rangle$ 
     $j := j - 1$ 
     $\langle \alpha(i) \wedge \beta(j) \wedge j - i = m - 1 \rangle \quad // \text{ ASSIGN}$ 
  else
     $\langle \alpha(i) \wedge \beta(j) \wedge A[i] > p \wedge A[j] < p \wedge j - i = m \rangle$ 
    swap( $A, i, j$ )
     $\langle \alpha(i) \wedge \beta(j) \wedge A[j] > p \wedge A[i] < p \wedge j - i = m \rangle$ 
     $\implies \langle \alpha(i+1) \wedge \beta(j-1) \wedge j - i = m \rangle$ 
     $i := i + 1 \ ; \ j := j - 1$ 
     $\langle \alpha(i) \wedge \beta(j) \wedge j - i = m - 2 \rangle \quad // \text{ ASSIGN}$ 
   $\langle \alpha(i) \wedge \beta(j) \wedge j - i < m \rangle \quad // \text{ IF}$ 
   $\implies \langle \alpha(i) \wedge \beta(j) \wedge j - i < m \rangle$ 
 $\langle \alpha(i) \wedge \beta(j) \wedge i > j \rangle \quad // \text{ HOARE-VARIANT}$ 

```

Fig. 10. Decorated proof of the PARTITION program.

And thus MALLOCDIV may diverge. Similar proofs of *guaranteed nontermination* can be derived using QINV-DEMON.

G.2 Full Proof of Quicksort Partition